

Министерство науки и высшего образования Российской Федерации
КУБАНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

А.Н. ПОЛЕТАЙКИН, Н.Ю. ДОБРОВОЛЬСКАЯ

ПРОЕКТИРОВАНИЕ ПРОГРАММНЫХ СИСТЕМ

Учебное пособие

Краснодар
2025

УДК 004.41(075.8)

ББК 32.973я73

П 497

Рецензенты:

Доктор физико-математических наук, профессор

А.В. Павлова

Кандидат технических наук, доцент

Л.Ф. Данилова

Полетайкин, А.Н., Добровольская, Н.Ю.

П 497 Проектирование программных систем: учебное пособие / А.Н. Полетайкин, Н.Ю. Добровольская; Министерство науки и высшего образования Российской Федерации, Кубанский государственный университет. – Краснодар: Кубанский гос. ун-т, 2025. – 184 с. – 500 экз.
ISBN 978-5-8209-2666-2

Посвящено теории и практике создания программных систем разного назначения. Рассматриваются основные понятия, методы, подходы и инструментальные средства для системного анализа предметной области, постановки задач, формулирования требований, разработки проектных и рабочих решений по созданию программных систем. Особое внимание уделяется объектно-ориентированному анализу и моделированию процессов и систем с применением UML.

Адресуется студентам направлений подготовки 01.03.02, 01.04.02 «Прикладная математика и информатика», 02.03.03 «Математическое обеспечение и администрирование информационных систем», 09.03.03 «Прикладная информатика», 09.04.02 «Информационные системы и технологии».

УДК 004.41(075.8)

ББК 32.973я73

ISBN 978-5-8209-2666-2

© Кубанский государственный университет, 2025

© Полетайкин А.Н.,
Добровольская Н.Ю., 2025

ВВЕДЕНИЕ

Появление и интенсивное развитие ЭВМ в 1950–1960-е гг. положило начало кибернетическому направлению применения программных и технических средств для повышения эффективности труда и управления производством, породив таким образом явление, называемое компьютеризацией.

Компьютеризация – целенаправленный процесс внедрения компьютерных технологий, обеспечивающих автоматизацию информационных процессов и технологий в различных сферах человеческой деятельности для повышения эффективности этой деятельности и качества производимой в её ходе продукции.

Именно компьютеризация послужила интенсификации переработки и рациональной организации хранения данных в условиях экспоненциального роста их объемов и скорости обновления. Тем не менее проблема была решена лишь на короткое время. Уже в 1970–1980-е гг. непрерывное усложнение задач, сопровождаемое развитием вычислительной техники, поставило новые задачи перед программными инженерами в плане революционного развития существующих средств разработки ПО. В частности, избыток средств доставки информации при недостатке средств ее анализа привел к тому, что нужной для принятия решений информации или совсем нет, или ее недостаточно, в то время как ненужная (нерелевантная) информация имеется в избытке. Данное обстоятельство потребовало изыскания новых технологий, принципиально отличающихся от компьютерных широким спектром методов и средств обработки информации в ее значительном количестве и многообразии, что привело к возникновению такого явления, как информатизация.

Информатизация – это направленный процесс системной интеграции компьютерных средств, информационных и коммуникационных технологий с целью выработки релевантной информации в области ее применения и получения новых общесистемных свойств этой области, позволяющих более

эффективно организовать продуктивную деятельность человека, группы, социума.

Сегодня социальный прогресс невозможен без информатизации. Информатизация, так же как и компьютеризация, стала реальной благодаря появлению компьютеров, которые представляют собой универсальное средство для работы с информацией и обеспечивают широкие возможности для коммуникации. Однако информатизация – это более широкое понятие, чем компьютеризация. Во-первых, в основе информатизации лежит массовое применение информационных систем и технологий, бурное развитие которых в 1990-е гг. привело к ее проникновению практически во все сферы деятельности человека вплоть до актуализации такой сверхзадачи, как информатизация общества, выражающей стратегическое направление развития современной цивилизации. Во-вторых, при информатизации на первый план выходит комплекс мероприятий, целью которых является использование знаний и данных во всех сферах цивилизации.

Для успешного решения задачи информатизации создаются компьютерные информационные и программные системы. При этом компьютерные информационные системы ориентированы в основном на обработку большого количества информации и в центре их архитектуры всегда находится информационная база.

Информационная система (ИС) – взаимосвязанная совокупность средств, методов и персонала, используемых для сбора, хранения, обработки и выдачи информации в интересах достижения поставленной цели.

Программные же системы (ПС) имеют более широкое применение и характеризуются разнообразным, зачастую нетривиальным составом и структурой (архитектурой) ПО. Фактически ИС есть частный случай ПС, нацеленной на информатизацию в полном смысле этого понятия.

Программная система – это ИС, состоящая преимущественно из программных компонентов, взаимодействующих между собой и с внешней средой. Она включает в себя как отдельные программы, так и связанные с

ними структуры данных, файлы конфигурации, документацию и другие ресурсы, работающие вместе для достижения общей цели.

Архитектура ПС – это её внутренняя структура (компоненты и их связи), основы пользовательского интерфейса продукта, а также средства разработки и управления проектом.

В данном учебном пособии рассмотрены преимущественно вопросы разработки ПС и проблемы и задачи создания крупного программного продукта – ПС. Для решения таких проблем и задач нужна тщательная подготовка проекта, определение функциональных и нефункциональных требований к будущему продукту, исследование структуры системы, определение логических взаимосвязей между компонентами. По сути, необходимо качественное проектирование ПС, основанное на современных технологиях проектирования ПО. Этот процесс предполагает использование методик и технологий управления процессами жизненного цикла ПС: постановки задачи, проектирование, разработка, тестирование и развертывание системы.

Владение знаниями в области проектирования программных продуктов, практические методы проектирования необходимы квалифицированным разработчикам программного обеспечения, менеджерам программных проектов, специалистам обеспечения и контроля качества. Предлагаемое учебное пособие знакомит студентов с методами управления проектами ПС, способами планирования и оценивания качества проекта, организации работ в заданные сроки. Содержит теоретические и практические знания создания программных систем на основе использования различного инструментария управления жизненным циклом приложений на всех этапах их жизненного цикла. Все этапы от анализа предметной области до внедрения и эксплуатации готовой системы мы называем программным процессом или разработкой в широком смысле, потому что разработке подлежат не только программы, но и требования, проектные решения, тесты, планы внедрения и пр.

1. ТЕХНОЛОГИЯ ПРОЕКТИРОВАНИЯ ПРОГРАММНЫХ СИСТЕМ

1.1. ОБЩИЕ СВЕДЕНИЯ О ПРОЕКТИРОВАНИИ ПРОГРАММНЫХ СИСТЕМ

Проектирование ПО – это один из основных процессов жизненного цикла ПО и важнейшая составляющая программного процесса.

1.1.1. Понятие проектирования

Проектирование ПО можно понимать в узком и широком смысле. В широком смысле это реализация всего программного процесса, проектирование отождествляется с разработкой ПО. В узком же смысле проектирование ПО сводится к реализации одной стадии жизненного цикла (ЖЦ) ПО.

Проектирование – это процесс создания описания, необходимого для построения в заданных условиях еще не существующего объекта, а также преобразования исходного описания в окончательное на основе выполнения работ конструкторского, исследовательского, расчетного характера.

Проектирование как в узком, так и в широком смысле осуществляется в рамках проектной деятельности.

Проектная деятельность – это учебно-познавательная, творческая деятельность, результатом которой является решение какой-либо проблемы или задачи, представленное в виде его подробного описания (проекта).

Совокупность проектных документов в соответствии с установленным перечнем, в которых представлен результат проектирования, называется проектом.

Проект – это детально описанный прообраз будущего объекта или способа деятельности.

Это также узкое понимание. В широком же смысле проект ПО отождествляется с программным процессом и проектной деятельностью по созданию программного продукта.

Проектирование так или иначе предполагает формализованный подход к решению проблемы или задачи. Формализованное описание расчетной задачи было рассмотрено в работах академика В.М. Глушкова на основе кибернетического подхода. Реализация каждого этапа проектирования осуществляется человеком, при этом для каждой конкретной задачи весь процесс выполняется заново.

По степени участия человека в процессе проектирования выделяют три вида проектирования.

1. Если весь процесс проектирования осуществляется человеком, то проектирование называют неавтоматизированным.

2. Проектирование, при котором происходит взаимодействие человека и ЭВМ, называется автоматизированным. Автоматизированное проектирование, как правило, осуществляется в режиме диалога человека с ЭВМ.

3. Проектирование, при котором все преобразования описаний объекта и алгоритма его функционирования осуществляются без участия человека, называется автоматическим.

1.1.2. Системный подход к описанию предметной области

В основе системного подхода лежит рассмотрение объекта или процесса как системы: целостного комплекса взаимосвязанных элементов.

Прежде чем перейти к рассмотрению проектирования ПС, рассмотрим само понятие программной системы.

Под системой в общем смысле понимают любой объект, который одновременно рассматривается и как единое целое, и как объединенная в интересах достижения поставленных целей совокупность разнородных элементов.

Система — это совокупность взаимодействующих разнородных компонентов, объединенных во имя достижения общей цели.

Системы отличаются между собой как по составу, так и по главным целям. В табл. 1.1 показаны примеры некоторых систем и их основные характеристики.

Таблица 1.1

Пример систем и их характеристики

Система	Элементы системы	Главная цель системы
Фирма	Люди, оборудование, материалы, здания и др.	Производство товаров
ЭВМ	Электронные и электромеханические элементы, линии связи и др.	Обработка данных
Телекоммуникационная система	ЭВМ, модемы, кабели, сетевое программное обеспечение и др.	Передача информации
Информационная (программная) система	ЭВМ, сети ЭВМ, люди, информационное и программное обеспечение	Производство профессиональной информации

Как информационные, так и программные системы обеспечивают сбор, хранение, обработку, поиск, выдачу информации, необходимой в процессе принятия решений задач в некоторой предметной области.

Предметная область – часть реального мира, подлежащая изучению с целью организации управления и в конечном счете информатизации.

Предметная область представляется множеством фрагментов, например, машиностроение – цехами, дирекцией, бухгалтерией и т.д.; управление дорожным движением – центральным управляющим пунктом, участками дорожно-транспортной сети, светофорными объектами и т.д. В организационных и социально-экономических системах компоненты предметной области принято отождествлять с бизнес-процессом либо понимать их как комплекс взаимосвязанных бизнес-процессов, к которому также применим системный подход.

Бизнес-процесс – логически завершенная цепочка взаимосвязанных и повторяющихся видов деятельности, в результате которых ресурсы организационного объекта используются для переработки других ресурсов (физически или виртуально) с целью достижения определенных измеримых результатов или создания продукции для удовлетворения внутренних или внешних потребителей.

В качестве клиента бизнес-процесса может выступать другой бизнес-процесс. Бизнес-процесс можно рассматривать как совокупность различных видов деятельности компании, в рамках которой «на входе» используются один или более видов ресурсов, и в результате этой деятельности «на выходе» создается продукт, представляющий ценность для потребителя.

В цепочку обычно входят операции, которые выполняются по определенным бизнес-правилам.

Бизнес-правила – способы реализации бизнес-функций в рамках бизнес-процесса, а также характеристики и условия выполнения бизнес-процесса.

Составляющие бизнес-процесс действия могут выполняться людьми (вручную или с применением компьютерных средств или механизмов) или быть полностью автоматизированы. Порядок выполнения действий и эффективность работы того, кто выполняет действие, определяют общую эффективность бизнес-процесса. Задачей каждого организационного объекта, стремящегося к совершенствованию своей деятельности, является построение таких бизнес-процессов, которые были бы эффективны и включали только действительно необходимые действия.

Примеры бизнес-процессов:

- разработка продукта – от выработки концепции до создания прототипа;
- производство продукции – от момента закупки сырья до момента отгрузки готовой продукции;
- продажа – от выявления потенциального клиента до получения заказа;

- выполнение заказа – от оформления заказа до осуществления платежа за поставленный товар или оказанную услугу;

- обслуживание – от получения запроса до разрешения возникшей проблемы;

- разработка ПС – от намерения заказать систему до ее внедрения.

Для системного представления и анализа бизнес-процесса прибегают к моделированию бизнес-процессов.

Модель бизнес-процесса – это его формализованное (графическое, табличное, текстовое, символьное) описание, отражающее реально существующую или предполагаемую деятельность предприятия.

Модель, как правило, содержит следующие сведения о бизнес-процессе:

- набор составляющих процесс шагов – бизнес-функций;
- порядок выполнения бизнес-функций;
- механизмы контроля и управления в рамках бизнес-процесса;

- исполнителей каждой бизнес-функции;
- входящие документы и информацию, исходящие документы и информацию;

- ресурсы, необходимые для выполнения каждой бизнес-функции;

- документацию и условия, регламентирующие выполнение каждой бизнес-функции;

- параметры, характеризующие выполнение бизнес-функций и процесса в целом.

Каждый бизнес-процесс предметной области характеризуется множеством объектов и процессов, использующих объекты, а также множеством пользователей, характеризующихся различными взглядами на предметную область в процессе её информатизации.

Понятие информатизации является следствием развития компьютерных информационных технологий и в своем развитии

восходит к понятиям механизации и автоматизации как основным этапам этого развития (рис. 1.1).

Информатизация – это направленный процесс системной интеграции компьютерных средств, информационных и коммуникационных технологий с целью выработки релевантной информации в области ее применения и получения новых общесистемных свойств этой области.

В самом начале указанного процесса информатизации в центре внимания системного аналитика находится некий бизнес-процесс – фрагмент предметной области, который подвергается информатизации. Такой процесс также коротко называют *процессом информатизации*.



Рис. 1.1. Этапы развития применения ИТ

Процесс информатизации – продуктивная деятельность человека или группы, целенаправленно осуществляемая в предметной области и имеющая целью достижение конкретного результата, представляющего ценность для потребителя.

Данная деятельность, как правило, осуществляется на базе некоторого объекта: технического, экономического, организационного, который называют объектом информатизации.

Объект информатизации – область деятельности человека или группы, характеризующаяся определенной целью и обладающая свойствами системы, элементами которой являются в том числе люди как субъекты деятельности.

Примеры:

- объект информатизации: торговое предприятие; процесс информатизации: складской учет товаров;
- объект информатизации: трубопрокатный стан; процесс информатизации: сварка трубы;
- объект информатизации: автомобиль; процесс информатизации: интеллектуальное управление тормозной системой.

Пример системного описания объекта информатизации представлен в прил. А.

Программная система (ПС) принципиально отличается от ИС лишь функциональной спецификой и превалированием программного обеспечения над информационным, тогда как в ИС они примерно равнозначны. Так, в составе ИС, как правило, присутствует база данных, тогда как в составе ПС её вполне может не быть. Кроме того, спектр возможных предметных областей ИС уже ограничен такими фрагментами, которые предполагают оборот большого количества информации.

В ИС и ПС в качестве основного технического средства переработки информации используется ЭВМ. В крупных организациях наряду с ЭВМ в состав технической базы ПС может входить сервер или даже вычислительный кластер.

Особая роль в ИС (ПС) отводится человеку, так как техническое воплощение ИС само по себе ничего не будет значить, если не учтена роль человека, для которого предназначена производимая информация и без которого невозможно ее получение и представление.

Необходимо понимать разницу между ЭВМ и программными системами. ЭВМ, оснащенные специализированными программными средствами, являются технической базой и инструментом для информационных систем.

ПС немислима без персонала, взаимодействующего с ЭВМ и телекоммуникациями.

1.1.3. Этапы и стадии проектирования ПС

В процессе создания ПС можно выделить 8 стадий проектирования.

1. Предпроектное исследование.
2. Анализ подобных ПС.
3. Постановка задачи (ТЗ).
4. Эскизное проектирование.
5. Техническое проектирование.
6. Рабочее проектирование.
7. Отладка, тестирование, внедрение.
8. Ввод в постоянную эксплуатацию.

Более укрупненно процесс проектирования состоит из двух основных этапов.

1. Обоснование исходных данных (постановка задачи) для проектирования – внешнее проектирование. С внешним проектированием связаны стадии 1–4 проектирования ПС. Результатом данного этапа является постановка задачи и эскизный проект ПС.

2. Проектирование системы для сформулированных исходных данных – внутреннее проектирование. С внутренним проектированием связаны стадии 5–8 проектирования ПС. Результатом этапа является ПС, действующая в установившемся режиме.

На рис. 1.2 показана спиральная модель ЖЦ ПС, первые 3 сектора которой соответствуют внешнему проектированию (анализ предметной области включает в себя стадии 1 и 2), а остальные 5 – внутреннему.

Процесс обоснования исходных данных (внешнее проектирование) существенно зависит от того, является ли проектируемая система частью более сложной системы, т.е. подсистемой (или устройством), или она задумана автономной, т.е. может использоваться заказчиком самостоятельно.



Рис. 1.2. Спиральная модель ЖЦ ПС

Если ПС будет являться подсистемой более сложной системы, последнюю необходимо предварительно разбить на части (сегменты). Для сложных объектов выполнить одновременно оптимальное проектирование для всех частей не удастся. Особенно это относится к тем случаям, когда требуется не только выбрать параметры системы, но и синтезировать ее структуру. Поэтому при проектировании систем большой сложности их обычно разбивают на подсистемы или сегменты согласно принципу декомпозиции – базовому принципу системного анализа, используемому при проектировании ПС.

Подсистема – относительно независимая часть системы, обладающая свойствами системы и, в частности, имеющая подцель, на достижение которой ориентирована подсистема, а также другие свойства – целостности, коммуникативности и т.п., определяемые закономерностями системы.

Система может быть разделена на элементы (задачи) не сразу, а последовательным расчленением на подсистемы –

совокупности элементов. Такое расчленение, как правило, производится на основе определения независимой функции, выполняемой данной совокупностью элементов совместно для достижения некой частной цели, которая обеспечивает достижение общей цели системы, и называется декомпозицией. Подсистема отличается от простой группы элементов, для которой не выполняется условие целостности.

Последовательное разбиение системы в глубину приводит к получению иерархии подсистем, нижним уровнем которых является элемент – задача. С этой концепцией связано понятие структуры системы (рис. 1.3).

Задача – проблемная ситуация с явно заданной целью, которую необходимо достичь; в более узком смысле задачей также называют саму эту цель, данную в рамках проблемной ситуации, т.е. то, что требуется сделать.

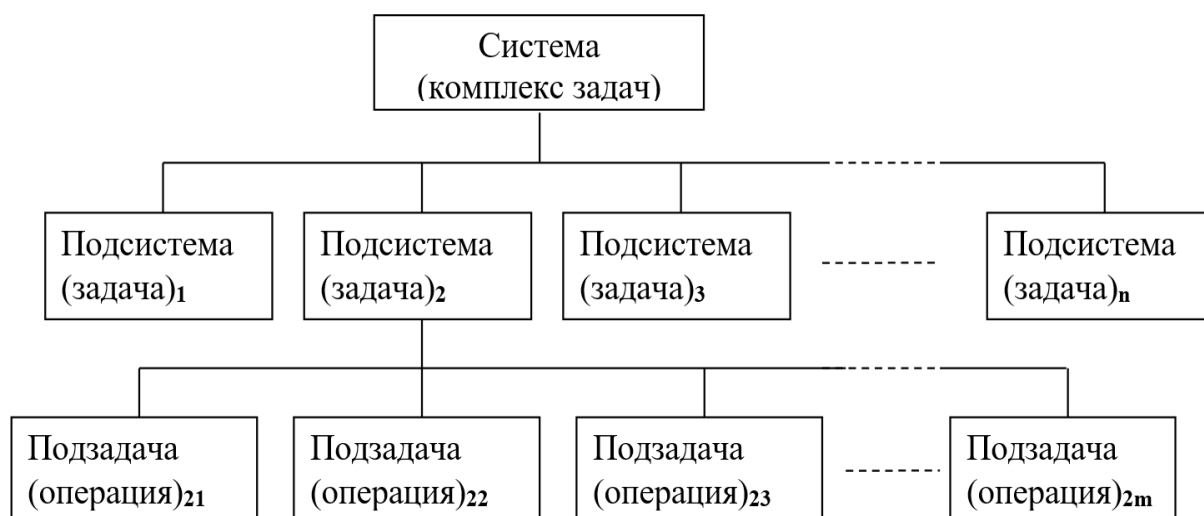


Рис. 1.3. Принцип декомпозиции в системном анализе

1.1.4. Автоматизация управления и ее цели

Появление ЭВМ положило начало кибернетическому направлению применения технических средств для повышения эффективности труда и управления производством, породив таким образом явление, называемое автоматизацией.

Строго говоря, процесс управления – это базовый процесс, который в том или ином виде реализует ПС.

Управление – это информационный процесс, связанный со сбором, передачей, накоплением информации о состоянии объекта управления, переработкой этой информации, выработкой управляющей информации и передачей этой информации на объект с целью принятия решений об изменении состояния объекта и приведения объекта управления в заданное (нормальное) состояние.

Сущность автоматизации заключается в использовании ЭВМ для усиления интеллектуальных возможностей человека. Все рассмотренные ранее пути ведут так или иначе к повышению производительности управляющих систем, но не повышают производительность умственного труда. В этом заключается их ограниченность.

Общей целью автоматизации и управления является повышение эффективности использования возможностей объекта управления по трем основным направлениям.

1. Повышение оперативности управления. Сокращение времени происходит в основном за счет таких процессов, как сбор, поиск, предварительная обработка и передача информации, засекречивание и рассекречивание информации, производство расчетов, решение логических задач, а также оформление и размножение документов.

2. Снижение трудозатрат человека на выполнение вспомогательных процессов. К ним относятся информационные и расчетные процессы, которые, имея вспомогательный характер, являются весьма трудоемкими. В типичной системе управления относительное распределение трудозатрат между процессами примерно следующее:

- информационные процессы – 65–70%;
- расчеты – 20–25%;
- творческие процессы – 5–15%.

В результате высвобождения от технической работы должностные лица могут сосредоточить основное внимание на творческих процессах управления.

3. Повышение степени научной обоснованности принимаемых решений. Процесс принятия решений строится на основе анализа и прогноза развития ситуации с применением математического аппарата. При этом сохраняют свое значение традиционные методы обоснования решений, опирающиеся на опыт и интуицию. Следует отметить, что оптимальных решений не всегда удастся достигнуть в условиях автоматизированного управления, поэтому говорят о рациональных решениях.

Приводя к повышению эффективности, автоматизация далеко не всегда сопровождается уменьшением численности людей, задействованных в управлении. Чаще всего происходит перераспределение личного состава внутри систем: сокращается численность должностных лиц, занятых непосредственно управлением, но увеличивается инженерный и технический персонал, обслуживающий программные и технические средства.

Основной эффект автоматизации достигается за счет своевременности и оптимальности (рациональности) принимаемых решений.

Таким образом, необходимость в автоматизированном управлении обусловлена резким усложнением процессов управления (усилением инфопотоков) и носит объективный характер. Создание информационных и управляющих систем на основе автоматизации позволяет повысить эффективность управленческой деятельности, а следовательно, и эффективность использования ресурсов (времени, сил и средств) в современных условиях. Будучи наиболее эффективным, этот путь совершенствования управления является вместе с тем и наиболее сложным в техническом и философском смысле.

Научную базу управления формирует кибернетика. Возникла как научное направление в 1943 г. Основателем этого направления является крупный американский ученый математик Норберт Винер.

Кибернетика – это наука об управлении, которая изучает системы и процессы их развития, основываясь на принципах информационных и математических моделей.

Значительный вклад в развитие кибернетики внесли советские академики М. К. Колмогоров, Н. М. Лебедев и В. М. Глушков. В настоящее время существует и развивается несколько направлений кибернетики, такие как техническая кибернетика, экономическая кибернетика, медицинская кибернетика, социальная кибернетика, военная кибернетика и др.

Базируется кибернетика на следующих направлениях:

- 1) математика (в широком понимании);
- 2) вычислительная техника;
- 3) информатика.

При автоматизации управления в центре внимания оказывается объект управления. Чаще всего он отождествляется с объектом информатизации. В общем случае объектом управления может быть что угодно, так как ни один объект в мире не является полностью независимым и автономным.

Объект управления – некий объект, выделенный в реальном мире, который рассматривается как база для решения задач в той или иной сфере деятельности человека и состояние которого подлежит нормализации с использованием компьютерных технологий.

Обобщенная структура процесса управления посредством ПС показана на рис. 1.4. Для управления и получения конкретного результата создается система управления, состоящая из двух основных частей (подсистем):

- 1) управляемой подсистемы (объект управления);
- 2) управляющей подсистемы (субъект управления).

Обязательным условием функционирования системы управления является постоянная информационная связь между управляемой подсистемой и управляющей подсистемой, т.е. должна быть постоянная информационная связь между объектом управления и субъектом управления. Такая информационная связь в системе управления называется **обратной связью**. Если обратная связь между объектом и управляющей подсистемой отсутствует, то такая система называется разомкнутой, или системой контроля.

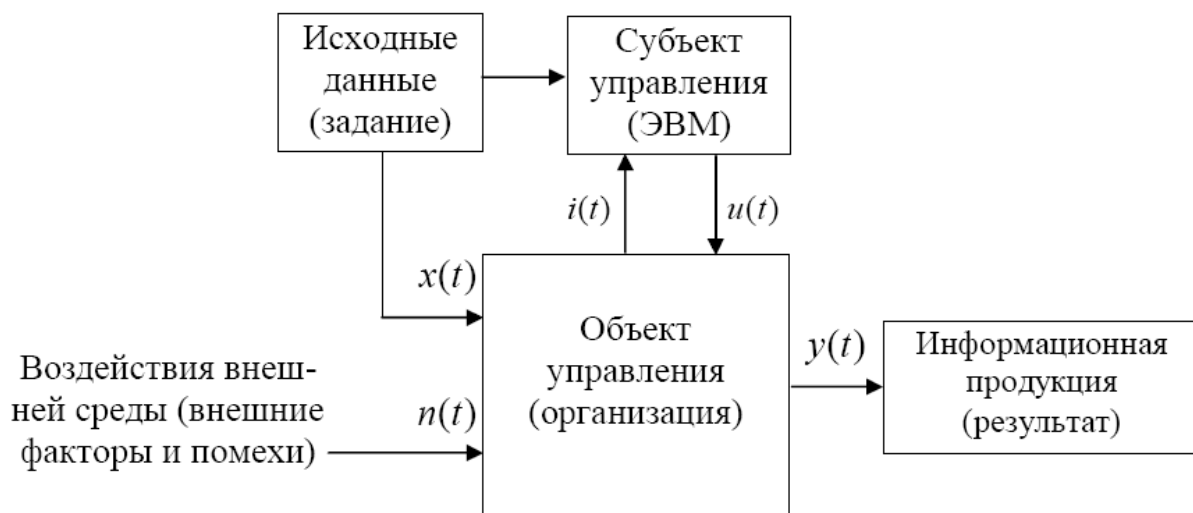


Рис. 1.4. Обобщенная структура процесса управления посредством ПС

В зависимости от природы объекта управления, его структуры и динамики автоматизированное управление бывает двух типов (рис. 1.5).



Рис. 1.5. Варианты применения ЭВМ в управлении:

а – человек – посредник между ЭВМ и внешним миром; *б* – ЭВМ – посредник между человеком и внешним миром

Случай (*а*) встречается чаще, он характерен для систем обработки информации. Согласно этой схеме, человек непосредственно взаимодействует с объектами внешнего мира (воспринимает от них информацию и оказывает на них воздействие), а ЭВМ играет вспомогательную роль: выполняет вычисления и преобразования информации, которые нужны человеку. Большинство систем, построенных по схеме (*а*), представляют собой автоматизированные ИС, АРМ (автоматизированные рабочие места), САПР (системы автоматизированного проектирования) и т.п. Управление,

реализуемое по схеме (а), является управлением верхнего уровня и относится к типу «Management».

Схема (б) соответствует тому случаю, когда человек (в силу собственного несовершенства) не способен самостоятельно взаимодействовать с объектами внешнего мира. В этом случае ЭВМ и системы на их основе играют роль посредника между человеком и объектами внешнего мира. Большинство систем, построенных по схеме (б), представляют собой АСУ – автоматизированные системы управления. Управление, реализуемое по схеме (б), является управлением нижнего уровня и относится к типу «Control».

Необходимо различать понятия «автоматический» и «автоматизированный».

Автоматические системы (или просто автоматы) – это устройства, выполняющие жестко заданный набор действий без участия человека.

Автоматизированные системы обычно работают при прямом или косвенном участии человека-оператора в качестве звена общей цепи управления объектом.

Также необходимо различать понятия «управление» и «регулирование». Под управлением обычно понимают сложную управляющую функцию автоматизированной системы. Регулирование – частный случай управления, представляющий собой принудительную стабилизацию небольшого количества (иногда всего одного) физических параметров, от которых зависит поведение объекта.

1.1.5. Принципы кибернетики, используемые при проектировании ПС

При создании ПС широко используются пять основных принципов кибернетики.

1. Принцип обратной связи. Заключается в том, что между объектом информатизации и управляющей подсистемой имеется постоянная информационная связь. На рис. 1.4 это связь $i(t)$. Она

противоположна прямой связи управления $u(t)$ и в совокупности с ней образует замкнутый контур управления ПС.

2. Принцип внешнего дополнения. При создании ПС необходимо рассматривать не только информацию, циркулирующую внутри системы между объектом и субъектом управления. Любая система при функционировании имеет кроме внутренних связей внешние связи как по вертикали, так и по горизонтали. Поэтому при создании ПС следует учитывать внешние и внутренние информационные связи. Учет внешних информационных связей при создании автоматизированных ПС и анализе их работы образует принцип внешнего дополнения. На рис. 1.4 это входящие инфопотоки $x(t)$ и исходящие инфопотоки $y(t)$, а также неконтролируемые внешние воздействия $n(t)$.

3. Принцип черного ящика. При изучении и исследовании сложных систем широко используется принцип черного ящика. Сущность этого принципа заключается в том, что изучаются только изменения входных и выходных показателей, исследуемых системой. Внутреннее же состояние системы при этом не учитывается. Т.е. рассматривается выданное задание на работу системы (исходные данные) – $x(t)$, и результат выполнения этого задания – $y(t)$. Также могут рассматриваться прочие внешние воздействия $n(t)$, если выяснится, что их влияние на результат существенно.

4. Принцип оптимальности. Предполагает получение оптимального (наилучшего) результата при функционировании системы. При этом важно понимать, когда следует прервать поиск наилучшего решения и не расходовать на него напрасно временной ресурс. («Лучшее – враг хорошего» – С. П. Королев). Поэтому для определения оптимального результата следует установить критерий (измеритель) оптимальности. Критерий оптимальности выбирается исходя из целевой функции оценки, т.е. что следует проанализировать и какой получить результат. В качестве критериев могут выступать прибыль, себестоимость, цена, качество продукции, время загрузки оборудования, расход энергии, скорость и объем памяти ЭВМ, точность (погрешность) вычисления и т.д. В зависимости от специфики предметной

области и решаемой задачи это может быть специфический набор метрик.

5. Принцип моделирования. Моделирование – процесс замены реального объекта или процесса моделью. Различают три основных вида моделирования.

1. Физическое моделирование. При физическом моделировании параметры реального объекта или процесса изменяются в модели в пространстве (размеры – высота, длина, объём) и во времени (ускоряется или замедляется процесс). Физическое моделирование широко используется в градостроительстве, создании электростанций, машиностроении, космонавтике, спорте и т.д.

2. Математическое моделирование. При математическом моделировании функционирование реального объекта или процесса заменяется математическими моделями. Такое моделирование представляет собой довольно мощный исследовательский инструмент, который может быть использован при создании ПС весьма широко.

3. Комбинированное моделирование. При исследовании и анализе функционирования сложных объектов и процессов используется комбинированное моделирование, т.е. физическое и математическое моделирование одновременно.

При создании ПС наиболее востребованным видом моделирования является математическое моделирование. Выделяют следующие его функции:

- дескриптивная (познавательная), позволяющая объяснить наблюдаемые на практике явления и процессы (как есть?);
- прогностическая, отражающая возможность предсказывать будущие свойства и состояния объекта, а также состояние внешней среды (как будет?);
- нормативная, которая заключается в построении нормативного образа объекта управления, желательного с точки зрения субъекта (рис. 1.4), интересы и предпочтения которого отражены в исходных данных заданными условиями, критериями, требованиями, нормами и принципами (как должно быть?).

При создании ПС математическое моделирование используют при исследовании объекта и процесса информатизации (дескриптивная функция) на этапе внешнего проектирования, при обеспечении соответствия требованиям ТЗ (нормативная функция) на стадии технического проектирования, а также на стадии разработки математического и программного обеспечения ПС, например, при решении задач оптимизации и т.д.

1.1.6. Проведение предпроектного обследования предприятий

Обследование предприятия является важным и определяющим этапом проектирования ПС. Длительность обследования обычно составляет 1–2 недели. В течение этого времени системный аналитик должен обследовать не более 2–3 видов деятельности (учет кадров, технология управления, перевозки, маркетинг и др.).

Сбор информации для построения полной структурной модели организации часто сводится к изучению документированных информационных потоков и функций подразделений, а также проводится путем интервьюирования и анкетирования персонала.

К началу работ по обследованию организация предоставляет комплект документов, в состав которого обычно входят следующие документы.

- 1) сводная информация о деятельности предприятия:
 - организационная структура предприятия;
 - информация об управленческой, финансово-экономической, производственной деятельности предприятия;
 - сведения об учетной политике и отчетности;
- 2) сведения об информационно-вычислительной инфраструктуре предприятия;
- 3) сведения об ответственных лицах;
- 4) регулярный документооборот предприятия. Вся информация циркулирует в рамках процесса информатизации в форме информационных потоков (инфопотоков) и требует

описания и систематизации. Такое описание уместно представить форме реестров входящей информации (табл. 1.2), внутренней информации (табл. 1.3), исходящей информации (табл. 1.4).

Таблица 1.2

Реестр входных информационных потоков

№	Наименование и назначение потока	Форма представления	Обработчик (кто обрабатывает)	Корреспондент (откуда поступает)	Характеристики обработки	
					Трудовые затраты, чел·ч	Периодичность, регламент

Таблица 1.3

Реестр внутренних информационных потоков

№	Наименование и назначение потока	Форма представления	Обработчик (кто обрабатывает)	Корреспондент (кому передает)	Характеристики обработки	
					Трудовые затраты, чел·ч	Периодичность, регламент

Таблица 1.4

Реестр выходных информационных потоков

№	Наименование и назначение потока	Форма представления	Обработчик (кто обрабатывает)	Корреспондент (куда отправляется)	Характеристики обработки	
					Трудовые затраты, чел·ч	Периодичность, регламент

Возможные формы представления информации: сигнал, устное сообщение, устное распоряжение, документ, датасет и др. Наименование потока должно адекватно отражать его сущность и соотноситься с его формой представления.

Трудовые затраты представляют собой вещественную неотрицательную оценку, выражающую количество часов, которые должен работать 1 человек, чтобы воспринять и обработать входной поток или сформировать и вывести

выходной поток. В редких случаях, когда человек не причастен к обработке инфопотока, выставляется оценка 0.

Периодичность отражает частоту следования инфопотока. Как правило, даются средние оценки. Наилучший способ определения этой характеристики – получить экспертное мнение специалиста из предметной области. Если такого специалиста нет, то периодичность следует оценить исходя из назначения инфопотока и логики его обработки. К примеру, для оперативной информации периодичность может составлять от 10 раз в сутки (например, заявка на ремонт оборудования) до 100 раз в секунду (например, сигнал от датчика системы реального времени). Для справочной информации периодичность гораздо ниже и может составлять 1 раз в год (обновление штатного расписания) или 10 раз в месяц (сведения о трудоустраиваемом лице). В крайнем случае, если никак невозможно установить приблизительную частоту следования инфопотока, допускается указать «По требованию».

Если форма представления инфопотока – документ, целесообразно представить его макет либо копию. При этом необходимо кратко описать такие реквизиты документа, смысл которых не очевиден.

Списки вопросов для интервьюирования и анкетирования составляются по каждому обследуемому подразделению и утверждаются руководителем кадровой службы. Это делается с целью:

- предотвращения доступа к конфиденциальной информации;
- усиления целевой направленности обследования;
- минимизации отвлечения сотрудников предприятия от выполнения должностных обязанностей.

Общий перечень вопросов (с их последующей детализацией) включает следующие разделы:

- основные задачи подразделений;
- собираемая и регистрируемая информация;
- результатная информация и отчетность;
- взаимодействие с другими подразделениями.

Анкеты для руководителей и специалистов могут содержать вопросы:

- об организационной структуре подразделения;
- задачах подразделения;
- последовательности действий при выполнении задач;
- целях (с позиций вашего подразделения) создания информационной системы управления предприятием;
- типах внешних организаций (банк, заказчик, поставщик и т.п.), с которыми взаимодействует подразделение;
- используемых справочных материалах;
- времени (в минутах), которое тратится на исполнение основных операций;
- датах, на которые приходится «пиковые нагрузки» (периодичность в месяц, квартал, год и т.д.);
- техническом оснащении подразделения (компьютеры, сеть, модем и т.п.);
- используемых программных продуктах для автоматизации бизнес-процессов в подразделениях;
- отчетах и частоте их выполнения для руководства предприятия;
- ключевых специалистах подразделения, способных ответить на любые вопросы по бизнес-процессам в подразделении;
- наличии удаленных объектов управления и их характеристиках;
- документообороте на вашем рабочем месте.

Собранные таким образом данные, как правило, не охватывают всех существенных сторон организационной деятельности и обладают высокой степенью субъективности. Кроме того, анализ опросов руководителей обследуемых организаций и предприятий показывает, что их представления о структуре организации, общих и локальных целях функционирования, задачах и функциях подразделений, а также подчиненности работников иногда имеют противоречивый характер. Часто эти представления расходятся с официально

декларируемыми целями и правилами или противоречат фактической деятельности.

Структуру информационных потоков можно выявить по образцам документов, конфигурациям компьютерных сетей и баз данных. Структура реальных микропроцессов, осуществляемых персоналом в информационных контактах (в значительной мере недокументированных), остается неизвестной. Ответы на эти вопросы может дать структурно-функциональная диагностика рабочего времени персонала. Цель диагностики – получение достоверного знания об организации и организационных отношениях ее функциональных элементов. В связи с этим к важнейшим задачам функциональной диагностики организационных структур относятся:

- классификация субъектов функционирования (категорий и групп работников);
- классификация элементов процесса функционирования (действий, процедур);
- классификация направлений (решаемых проблем), целей функционирования;
- классификация элементов информационных потоков;
- исследование распределения (по времени и частоте) организационных характеристик: процедур, контактов персонала, направлений деятельности, элементов информационных потоков – по отдельности и в комбинациях друг с другом по категориям работников, видам процедур и их направлениям (согласно результатам и логике исследований);
- выявление реальной структуры функциональных, информационных, иерархических, временных, проблемных отношений между руководителями, сотрудниками и подразделениями;
- установление структуры распределения рабочего времени руководителей и персонала относительно функций, проблем и целей организации;
- выявление основных технологий функционирования организации (информационных процессов, включая и

недокументированные), их целеполагания в сравнении с декларируемыми целями организации;

– выявление однородных по специфике деятельности, целевой ориентации и реальной подчиненности групп работников, формирование реальной модели организационной структуры и сравнение ее с декларируемой;

– определение причин рассогласования декларируемой и реальной структуры организационных отношений.

1.1.7. Результаты предпроектного обследования

Результатом предпроектного обследования является «Отчет о предпроектном обследовании предприятия» (характеристика объекта информатизации), в состав которого входят некоторые сведения.

1. Организационная структура объекта управления (предприятия).

2. Основные требования и приоритеты информатизации.

3. Краткое схематичное описание бизнес-процессов, в том числе в формате табл. 1.5. Описание документов бизнес-процесса представляется в формате табл. 1.6.

Таблица 1.5

Операции бизнес-процесса

Операция	Исполнитель	Частота	Входящие документы (документы-основания)	Исходящий документ (составляемый документ)

Таблица 1.6

Описание документов бизнес-процесса

Составляемый документ (исходящий документ)	Операция	Кто составляет (исполнитель)	Частота	Документы-основания (входящие документы)

4. Оценивание необходимых для обеспечения проекта ресурсов заказчика.

5. Оценивание возможности и целесообразности информатизации.

6. Предложения по созданию ПС с оценкой примерных сроков и стоимости.

Проведение предпроектного обследования позволяет решить следующие задачи:

- предварительное выявление требований к будущей системе;
- определение структуры организации;
- определение перечня целевых функций организации;
- анализ распределения функций по подразделениям и сотрудникам;
- выявление функциональных взаимодействий между подразделениями;
- выявление информационных потоков внутри подразделений и между ними, внешних информационных воздействий;
- анализ существующих средств информатизации на предприятии.

1.1.8. Постановка задачи на создание ПС

Постановка задачи на создание ПС, как правило, сводится к разработке технического задания (ТЗ).

Техническое задание на создание ПС – основной документ, определяющий требования и порядок создания (развития или модернизации) ПС, в соответствии с которым проводится разработка ПС и ее приемка при вводе в действие.

ТЗ разрабатывают на систему в целом, предназначенную для работы самостоятельно или в составе другой системы. Этот документ, которым руководствуются разработчики и проектировщики, приступая к разработке нового продукта, определяет основные направления разработки: технологии и принципы работы будущего продукта.

Техническое задание преследует множество целей. Среди них:

- постановка задачи исполнителям;
- подробное описание конечного программного продукта;
- определение порядка работ и его согласование с заказчиком;
- определение процедуры оценки приема конечного программного продукта.

Пример технического задания на разработку мобильного приложения туристических услуг приведен в прил. Б.

Требования к ПС во многом аналогичны требованиям к ПО. Вместе с тем ТЗ на создание системы включает в себя также требования к её обеспечивающим подсистемам (декомпозиция ПС на функциональные и обеспечивающие средства будет рассмотрена в параграфе 1.2). Эти требования также относятся к категории нефункциональных требований. Выделяют четыре основных обеспечивающих подсистемы: информационное, математическое, программное и техническое обеспечение ПС.

Требования к информационному обеспечению могут включать следующие пункты:

- требования к организации данных, которые должны сохраняться в ПС;
- отсутствие дублирования информации и сокращение чрезмерности данных, поддержка целостности, достоверности и актуальности данных;
- сетевой режим доступа к общим данным, распределение информационных ресурсов в подсистеме;
- низкая стоимость хранения и использования данных;
- возможность получения данных с помощью языка запросов высокого уровня без использования прикладных программ;
- защита от несанкционированного доступа к данным, их порчи и уничтожения;
- обеспечение конфиденциальности секретной информации;

- возможность ведения архивов и восстановление данных в случае разрушения баз данных после сбоев.

Для математического обеспечения приводят требования к составу, области применения (ограничения) и средств использования в системе математических методов и моделей, типичных алгоритмов и алгоритмов, которые подлежат разработке.

В требованиях к программному обеспечению ПС указываются:

- требования к структуре ПО подсистемы;
- требования к операционной среде;
- требования к инструментальным средствам разработки ПО;
- требования к использованию готовых программных пакетов;
- требования к составу и функциям специального ПО, подлежащему разработке;
- требования к вспомогательным программным средствам (сервисные программы и утилиты).

В требованиях к техническому обеспечению указываются требования к функциональным, конструктивным и эксплуатационным характеристикам технических средств ПС, а именно требования относительно выбора не менее двух конфигураций ЭВМ: для сервера (серверов) и клиентских рабочих станций. При этом необходимо учитывать, что эти требования обусловлены спецификой задач, которые решаются с помощью ПС.

Задача ПС – функция ПС или часть функции ПС, являющаяся формализованной совокупностью действий, выполнение которых приводит к результату заданного вида (для которых можно четко сформулировать результат). Например, задача ввода входных данных, задача формирования статистической отчетности, задача диагностики состояния объекта, задача анализа данных и принятия решений и др.

При описании требований к аппаратуре необходимо указывать не конкретные значения характеристик аппаратных

компонентов или конкретные типы, а лишь определенные ограничения на эти характеристики. Например, не следует писать «...основная память должна поддерживать частоту шины N ГГц, ... должна быть построена на модулях типа ...». Правильней здесь могла быть фраза «...частота, поддерживаемая основной памятью компьютера, должна быть не ниже N ГГц...» с последующим обоснованием.

1.2. ПРОЕКТНЫЕ РЕШЕНИЯ ПО СОЗДАНИЮ ПРОГРАММНОЙ СИСТЕМЫ

1.2.1. Структурные подсистемы ПС

Структурно ПС включает в себя набор взаимодействующих между собой подсистем.

Подсистема ПС – часть ПС, выделенная по функциональному или структурному признаку, отвечающему конкретным целям и задачам управления.

Принято при проектировании ПС различать *функциональную* и *обеспечивающую* части системы (рис. 1.6).

Функциональная часть ПС состоит из комплекса административных, организационных и математических методов, обеспечивающих решение задач учета, планирования, прогнозирования и анализа показателей производства для принятия управленческих решений в подсистемах и системе в целом. Подсистемы, входящие в функциональную часть ПС, называются *функциональными подсистемами ПС*.

Обеспечивающая часть ПС состоит из информационного, лингвистического, технического, математического и программного обеспечения. Подсистемы, входящие в обеспечивающую часть ПС, называются *обеспечивающими подсистемами ПС*.

Информационное обеспечение ПС – совокупность единой системы классификаций и кодирования технико-экономической информации, унифицированных систем документации и массивов информации, используемых в ПС.

Лингвистическое обеспечение ПС – совокупность научно-технических терминов и других языковых средств, используемых в ПС, а также правил формализации естественного языка, в целях повышения эффективности машинной обработки информации и облегчения общения человека с машиной.

Техническое обеспечение ПС – комплекс технических средств (КТС), предназначенных для обеспечения работы ПС.

Математическое обеспечение ПС – совокупность математических методов, моделей и алгоритмов для решения задач и обработки информации с применением вычислительной техники в ПС.

Программное обеспечение ПС – совокупность программ для различных целей и задач ПС и обеспечения функционирования комплекса технических средств ПС.

Экономическое обеспечение ПС – совокупность экономических показателей и методик их расчета, используемая для обоснования целесообразности работ по обработке информации.

Правовое обеспечение ПС – это совокупность правовых норм, которые регламентируют правоотношения, возникающие при работе ПС, а также юридический статус результатов, получаемых в ходе её функционирования.

Обычно лингвистическое и правовое обеспечение входит в состав информационного, а программное и экономическое обеспечение – в математическое, поэтому принято при проектировании ПС создавать информационное, математическое и техническое обеспечения.

1.2.2. Функциональная часть ПС

Функциональная часть ПС непосредственно связана с объектом информатизации. Предполагает реализацию функциональных требований, сформулированных заказчиком и представленных в ТЗ на создание ПС. Рассмотрим частный случай функциональной части ПС управления предприятием. На рис. 1.6 она дана в левой части схемы.

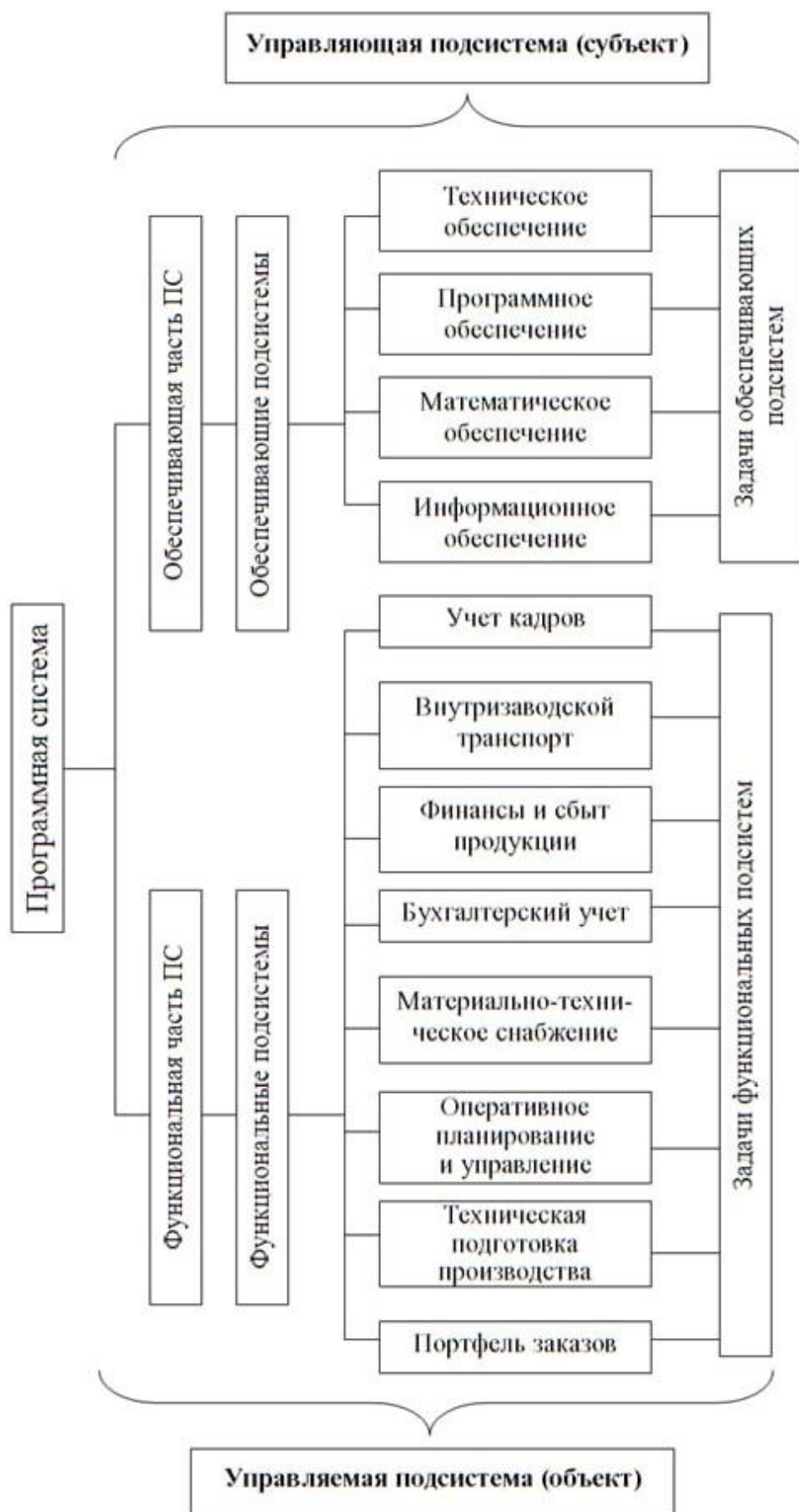


Рис. 1.6. Функциональная и обеспечивающая части ПС управления машиностроительным предприятием

Например, функциональная часть предприятия машиностроительного типа включает 8 типовых функциональных подсистем.

1. Портфель заказов.
2. Техническая подготовка производства.
3. Оперативное планирование и управление производством.
4. Логистическое управление запасами (материально-техническое снабжение).
5. Бухгалтерский учет.
6. Финансовое обеспечение и сбыт продукции.
7. Внутривозвездской транспорт (организационное управление).
8. Учет кадров.

В данных подсистемах решаются задачи, каждая из которых может стать прообразом для автоматизированной функции соответствующей функциональной подсистемы ПС. Примерный перечень задач (автоматизированных функций), реализуемых ПС машиностроительного предприятия, представлен в табл. Г.1 прил. Г. Для каждой функции указана принадлежность к одному из трех классов: информационная (И), вычислительная (В), управляющая (У).

Информационная функция – это наиболее распространенная функция в ИС, которая предполагает, например, получение информации извне, выполнение несложных операций по преобразованию, чтению / записи и хранению информации, а также формирование и вывод документов и другой результатной информации.

Вычислительная функция связана с реализацией математических вычислений при внутримашинной переработке информации. Как правило, предваряется информационной функцией. Часто предваряется информационной функцией ПС. Результаты ее реализации всегда используются при выполнении информационной или управляющей функции ПС.

Управляющая функция отвечает за выработку управленческих решений (управление типа «Management», рис. 1.5, а), за формирование и выбор управляющих воздействий

(управление типа «Control», рис. 1.5, б), а также их реализацию. Также к этому классу относят функции, связанные с конфигурированием технических устройств ПС. Управляющая функция всегда направлена на сохранение определенной структуры управляемого объекта, поддержание конкретного режима его деятельности, реализацию намеченной программы поведения объекта, обеспечение достижения им определенных целей.

Еще один пример – функциональная часть автоматизированной системы управления дорожным движением (АСУ ДД), которая решает следующие задачи:

- 1) мониторинг параметров транспортных потоков;
- 2) автоматическое управление локальным светофорным объектом;
- 3) передача данных между центральным управляющим пунктом и локальными светофорными объектами;
- 4) распределенное управление дорожным движением;
- 5) диспетчерское управление дорожным движением;
- 6) диспетчеризация событий в системе;
- 7) формирование выходной отчетной документации.

Примерный перечень задач (автоматизированных функций), реализуемых данной АСУ ДД представлен в табл. Г.2.

1.2.3. Порядок решения задач по созданию функциональных подсистем: организационный аспект

Организация работ по созданию функциональных подсистем ПС предполагает решение определенных задач.

1. Заключение договора на выполнение работ по информатизации.

2. Изучение объекта информатизации (выделение функциональных подсистем и автоматизированных функций в этих подсистемах).

3. Разработка технического задания на выполнение одной задачи (крупная), несколько задач или на функциональную подсистему, систему (согласно договору).

4. Постановка задачи:

- выполняется словесная формулировка задачи;
- устанавливается входная (текущая) информация;
- представляются формы выходной информации;
- определяется нормативно-справочная (условно-постоянная) информация, которая затем образует основу информационного банка данных;
- определяется частота решения задачи.

5. Формализованное описание задач: выбирается математический метод и разрабатывается математическая модель решения задачи.

6. Разработка макро- и микро-алгоритмов решения задачи.

7. Выбор языка программирования и разработка программ решения задачи.

8. Отладка программы и тестирование.

9. Создание (модификация) технической платформы для автоматизированного решения задачи.

10. Интеграция аппаратной и программной частей и решение контрольного примера.

11. Внедрение подсистемы ПС решения задачи в эксплуатацию.

12. Разработка инструкции пользователю (заказчику) по решению задачи с использованием ПС.

13. Разработка функциональной спецификации ПС.

14. Сдача-приемка подсистемы ПС пользователю исполнителем согласно договору (см. п. 1).

15. Постоянная эксплуатация ПС – решение задач при помощи разработанной подсистемы на постоянной основе.

16. Обслуживание и поддержка ПС согласно договору (см. п. 1).

В случае корректировки или расширения решенной задачи заказчик заключает с исполнителем дополнительный договор и выплачивает определенную сумму исполнителю по договоренности.

1.2.4. Информационное обеспечение ПС

Информационное обеспечение ПС в своем определении (см. п. 1.2.1) опирается на базовое понятие информации. Причем классическое определение информации как совокупности данных, зафиксированных на материальном носителе, а также устных данных, сохранённых и распространённых во времени и пространстве, здесь не применимо, поскольку в широком смысле это весьма абстрактное понятие, имеющее множество значений, в зависимости от контекста.

В хозяйственно-экономической деятельности под информацией в широком смысле понимают сведения об окружающем мире, которые могут быть получены от взаимодействия с ним, приспособления к нему и изменения его в процессе взаимодействия.

В административно-хозяйственной сфере *информация* всегда отражает соотношение между корреспондентом и адресатом, между ее создателем и потребителем. Информационный процесс всегда направлен.

Единицей количества информации считают такое количество, которое содержится в некотором стандартном сообщении. Моделью такого сообщения является элементарная система, имеющая определенное число возможных состояний, не менее двух, так как одно возможное состояние не несет никакой информации. Если элементарная система имеет два возможных состояния, то единицей информации считают бит. Количество информации также измеряют в байтах, килобайтах, мегабайтах. На предприятиях может быть использована модель документа, предполагающая измерение количества обрабатываемой информации в документах и документо-строках. При этом объем информации в отдельных реквизитах документов определяется исходя из их области допустимых значений.

При создании ПС необходимо знать не только суммарное количество информации, обращающейся в системе, но и скорость ее обращения и обработки. В связи с этим вводится понятие интенсивности потока информации I , что представляет собой

объем информации, передаваемой (обрабатываемой) за определенное время:

$$I = \frac{Q}{T},$$

где Q – количество информации в потоке; T – интервал времени, на котором проводится наблюдение.

Также важной характеристикой инфопотока является трудоемкость его переработки, выражаемая в человеко-часах. Данная характеристика определяется эмпирически и может быть представлена в виде среднего значения. Указанные характеристики инфопотока вносятся в соответствующие реестры (табл. 1.1–1.3).

Основными источниками информации на предприятии являются различного рода документы. Документация, необходимая для управления предприятием, классифицируется по следующим признакам:

- по способу получения;
- стабильности реквизитов документов;
- содержанию;
- назначению.

Указанная классификация позволяет более полно изучить инфопотоки на предприятии и выработать меры по их сокращению и упорядочиванию.

Документооборот определяет последовательность прохождения документов с момента их составления или получения до момента их обработки и использования. В основу правильной организации документооборота должны быть положены следующие принципы:

- рациональное и своевременное составление документов;
- последовательность охвата документами всех процессов хозяйственной деятельности предприятия;
- взаимосвязанность документов;
- сокращение (оптимизация) путей прохождения документов;

- систематическое изучение и совершенствование документооборота.

Рациональное составление документов предусматривает:

- сокращение числа заполняемых документов;
- расположение показателей в логической последовательности, удобной для дальнейшей компьютерной обработки;

- уменьшение количества документов.

Такой рациональный подход в конечном итоге упрощает документацию, сокращает документооборот за счет:

- уменьшения трудоемкости ее заполнения;
- сокращения цикла обработки;
- упрощения компьютерной обработки.

В условиях информатизации производства прохождение основных документов разбивается на три части:

- 1) до обработки (входная информация);
- 2) в процессе обработки (внутрисистемная информация);
- 3) после обработки (выходная информация – результат).

Анализ инфопотоков – часть комплекса работ при проектировании ПС. При изучении инфопотоков принимают во внимание только те из них, которые связывают различные производственные и управленческие подразделения. Сообщения, циркулирующие внутри подразделений, не считаются в данном случае основным предметом рассмотрения. Выдаются только сообщения, являющиеся входящими или исходящими из подразделений.

Совместно все инфопотоки образуют сеть потоков информации предприятия, которая формализуется в виде схемы инфопотоков. Задача аналитика – определить эту сеть и ее ветви.

После анализа документооборота предприятия составляется информационная модель, которая наиболее полно отражает места возникновения документов, их характеристики (количественные и качественные), направления движения, процесс обработки и передачи потребителю. Такая модель необходима для упорядочения структуры документооборота и определения

оптимального пути следования, а также для устранения избыточности информации.

При проектировании ПС информационная модель должна представлять системное информационное обеспечение по комплексу выбранных задач. В связи с этим к документам разрабатываются специальные требования, в соответствии с которыми они должны быть пригодными для компьютерной обработки. Для задач оперативно-диспетчерского управления (управления нижнего уровня) периодичность формирования информации определяется интервалом времени, за который может быть восстановлен запланированный режим работы после получения информации о фактическом состоянии объекта. Для задач текущего и перспективного экономического управления (управления верхнего уровня) эта периодичность определяется с учетом влияния результатов решения задач на ритмичность работы организационного объекта.

При формировании информационного обеспечения для ПС предусматривается:

- установление совокупности показателей, используемых в системе;
- разработка классификаторов и словарей наименований отдельных показателей;
- однократность записи информации, используемой при решении задач (в исключительных случаях разрешается дублирование в различных документах при условии обеспечения контроля их соответствия);
- межотраслевой обмен информацией на определенных уровнях управления;
- регламентация информационных связей между задачами;
- разработка единых форм документов, используемых в системе;
- выбор СУБД;
- разработка способов хранения, поиска и внесения изменений информации (создание информационной базы данных);

- разработка способов контроля формирования и обработки информации.

Вновь разрабатываемые для ПС формы документов должны обеспечивать:

- использование существующих форм документов, пригодных для компьютерной обработки;
- унификацию документации;
- соответствие документации требованиям государственных органов управления, которые обязательны для предприятий и организаций экономики страны.

1.2.5. Техническое обеспечение ПС

Разработка технического обеспечения ПС сводится к формированию комплекса технических средств (КТС).

Комплекс технических средств ПС – совокупность взаимосвязанных технических устройств регистрации, накопления, обработки, отображения, передачи и приема информации, предназначенных для переработки производственно-экономических сведений о функционировании объекта информатизации с целью управления им.

Взаимная связь классов устройств комплекса технических средств представлена на рис. 1.7.

В состав КТС развитой ПС входят:

- 1) устройства регистрации и сбора данных на объекте управления;
- 2) устройства накопления, частичной обработки и передачи информации;
- 3) оборудование информационно-вычислительного центра (ИВЦ) (как правило, реализуется на базе одного или нескольких серверов);
- 4) устройства отображения информации у сотрудников предприятия (подразделений) и других получателей информации;
- 5) устройства приема и передачи информации.

Рассмотрим взаимодействие устройств комплекса технических средств ПС.



Рис. 1.7. Обобщенная структура КТС программной системы

Устройства регистрации информации устанавливаются в местах возникновения информации (производственные участки, цеха, службы завода) и предназначены для механизированной фиксации информации о результатах и изменениях в производственном процессе (движение деталей по технологическим маршрутам, наличие страховых запасов, запуск деталей в производство, учет рабочего времени и т.д.). В ИС это, как правило, консольные устройства ввода. Результатом функционирования средств регистрации и сбора данных является их согласование и преобразование к виду, удобному для поступления по каналам связи или получения машинного носителя для передачи в персональный компьютер (сервер) информационно-вычислительного центра.

Средства накопления, частичной обработки и передачи информации выполняют функции решения задач оперативного управления производством на уровне подразделения, а также связи по каналам с устройствами сбора данных и с ИВЦ.

ИВЦ предназначен для централизованной обработки производственно-экономической информации в больших объемах. Комплектуется оборудованием различных классов:

- средств вычислительной техники;
- комплекта устройств приема-передачи информации;
- архивной техники для хранения информационных массивов;
- сервисной аппаратуры;
- средств питания;
- устройств для обеспечения комфортных условий работы техники, персонала;
- другого оборудования.

После обработки информации на *ИВЦ* данные поступают на *устройства отображения*, установленные у сотрудников предприятия (директор, главный инженер, диспетчер, оператор и т.д.) или в местах массового считывания. Пользователи на основании полученной информации принимают решения об изменении хода производственного процесса и выдают при необходимости соответствующие управляющие воздействия на объект. Указанные инфопотоки обеспечиваются устройствами класса 5.

С точки зрения функционирования КТС система управления должна быть информационно-замкнутой, поэтому отдельные устройства комплекса объединены между собой линиями прямых и обратных связей, что позволяет повысить оперативность и надежность управления.

Кроме того, КТС ПС имеет следующие внешние связи:

- горизонтальные – с другой техникой на предприятии (со стороны объекта информатизации) посредством устройств классов 1 и 5;
- вертикальные – с управляющей надсистемой и организацией межотраслевого обмена (со стороны *ИВЦ*).

Эти внешние связи характеризуют открытость ПС и определяют ее границы в абстракции от других производственных задач.

1.2.6. Требования к комплексу технических средств ПС

При выборе состава технических средств ПС следует руководствоваться некоторыми требованиями.

1. КТС должен представлять собой не произвольный, а взаимосвязанный набор устройств, т.е. все устройства должны представлять техническую систему.

2. Устройства должны располагаться последовательно, по этапам технологии обработки информации.

3. КТС должен быть агрегатируемым, что позволяет гибко перестраивать и наращивать технические средства для получения заданной производительности.

4. Вся информация, поступающая в КТС, должна быть формализована.

5. Устройства в технологической цепи преобразования информации должны быть согласованы по производительности.

6. Надежность как самих технических средств, так и всей структуры КТС должна отвечать требованиям функционирования системы управления.

7. Рекомендуемые проектом технические средства должны выпускаться серийно.

При выборе КТС необходимо учитывать, что ПС является сложной человеко-машинной системой и экономический эффект применения технических средств достигается в основном не за счет сокращения управленческого персонала, а непосредственно в производстве. Поэтому затраты на совершенствование техники управления должны соизмеряться с экономическими результатами в производстве.

При построении КТС необходимо стремиться к выполнению основного принципа информационного обеспечения ПС: однократности ввода информации в систему и многократности ее использования.

Выбор состава КТС производится на предпроектной стадии (определение общей стратегии построения КТС и предварительный выбор технических средств), а также на стадии

технического проектирования (определение структуры КТС, номенклатуры и количества технических средств). При техническом проектировании формируются конкретные проектные решения относительно выбора КТС и строится его структурная схема. В рабочем проекте осуществляется, как правило, некоторая корректировка решений технического проекта.

При разработке технического обеспечения ПС необходимо на основании анализа объекта управления, функциональной структуры ПС и комплекса задач, решаемых в системе, выполнить следующие работы:

- 1) определить организационно-технические требования к КТС и разработать ТЗ на выбор КТС;
- 2) определить критерий выбора и ограничения на КТС;
- 3) провести анализ технологии обработки информации и номенклатуры серийно выпускаемых технических средств в соответствии с допустимыми вариантами структур отдельных подсистем КТС;
- 4) на этапе технического проектирования определить номенклатуру и количество устройств, реализующих допустимые структурные варианты отдельных подсистем КТС и с учетом критериев и ограничений;
- 5) на этапе рабочего проектирования определить возможные варианты КТС для данного типа ПС и на основе выбранных критериев эффективности и ограничений выделить оптимальный вариант КТС. В случае, если номенклатура серийно выпускаемых технических средств не отвечает требованиям ПС, разрабатывается техническое задание на проектирование новых устройств.

При проектировании КТС для ПС в области, не связанной с управлением технологическими процессами, для управления бизнес-процессами, ограничиваются решениями по выбору конфигурации ЭВМ и телекоммуникационного оборудования.

Прежде всего, производится подбор конфигурации задействованных в системе ЭВМ. Поскольку современные ПС чаще всего строятся на основе технологии клиент–сервер, то

следует отображать решения относительно выбора двух конфигураций ЭВМ: сервера и компьютера для клиентских рабочих станций, например, автоматизированных рабочих мест пользователей ПС.

Следует помнить, что компьютерный рынок предлагает довольно большую номенклатуру компонентов для ЭВМ разных характеристик и конфигураций, поэтому необходимо обоснованно выбирать на нем самое необходимое для подсистемы по условиям и требованиям, выдвинутым в техническом задании. Выбор должен выполняться из современных аппаратных средств, т.е. сугубо новой техники, которая производится ныне и представлена в прайс-листах компьютерных фирм в пределах последнего полугодия. При этом следует учитывать не только текущее состояние, но и возможное развитие ИС в перспективе.

При распределении затрат на КТС, включая затраты на обслуживание и эксплуатацию, необходимо также учитывать возможные изменения числа управленческих работников, размеры их оплаты и прочие дополнительные затраты, связанные с автоматизацией управления.

На основании значений обобщенных параметров КТС и в соответствии с критерием эффективности автоматизации управления определяются количественные характеристики эффективности для каждого варианта КТС по формуле

$$П = С + E_n \cdot K,$$

где $П$ – приведенные затраты на КТС; $С$ – годовые эксплуатационные расходы на КТС; E_n – отраслевой нормативный коэффициент эффективности капитальных вложений; K – капитальные затраты на создание КТС. По наименьшему значению приведенных затрат выбирается наилучший вариант КТС.

Кроме минимизации затрат на КТС при выборе оборудования системы следует учитывать специфику производственного процесса предприятия и выполнение ПС целей и задач управления. В качестве примера рассмотрим сеть персональных компьютеров (ПК) для управления предприятием.

В данном случае вычислительная сеть имеет три уровня управления. На первом уровне располагаются АРМы пользователей системы. На втором уровне находится ИВЦ, где устанавливаются сервер или несколько серверов с базами данных (показателей), характеризующих работу предприятия. Третий уровень – сетевой, который реализован на базе глобальной сети передачи данных (Wide Area Networks, WAN). Общая схема такой структуры КТС показана на рис. 1.8.

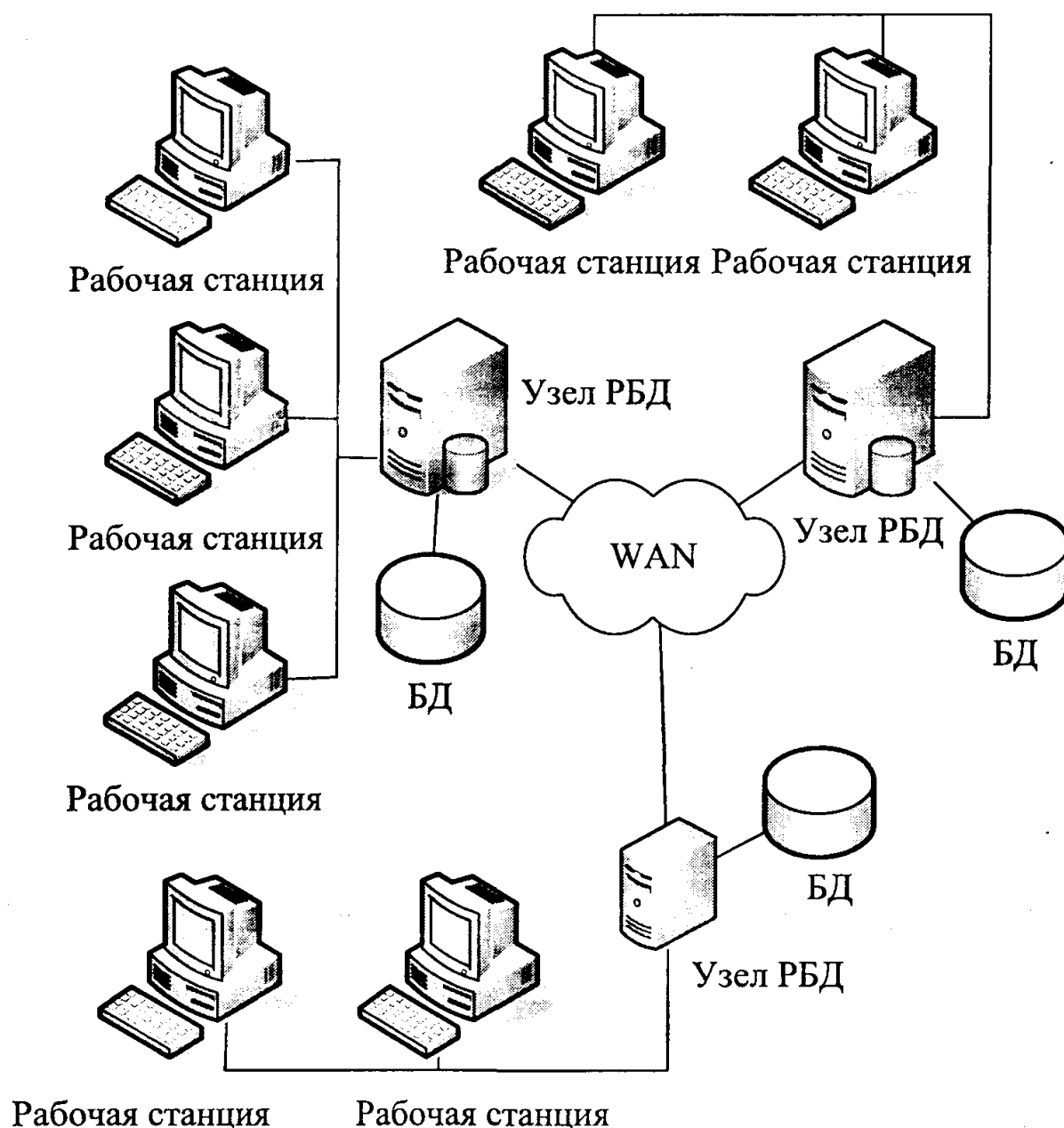


Рис. 1.8. Структурная схема трехуровневого КТС в общем виде

В памяти других ПК всех уровней создаются базы данных, образуя сеть распределенных баз данных. Возможна установка серверов (или нескольких серверов) на каждом уровне управления. Организация распределенных баз данных осуществляется с использованием системного программного обеспечения, а решение задач функциональной части управления производится с помощью программ специального (прикладного) программного обеспечения.

Конфигурация вычислительной сети в конкретном случае определяется функциональной (организационной) структурой предприятия, топологией расположения отделов, служб, производственных подразделений, номенклатурой выпускаемой продукции, численностью работающих и другими требованиями.

1.2.7. Математическое обеспечение ПС

Под математическим обеспечением ПС понимают совокупность математических методов, математических моделей, алгоритмов и программ регулярного применения, предназначенных для выполнения определенных функций.

1. Эффективное составление программ, включенных в систему математического обеспечения.

2. Организация вычислительного процесса в ЭВМ.

3. Эксплуатация комплекса технических средств.

4. Эффективное программирование задач, решаемых в ПС.

5. Функционирование ПС во времени, обусловленное использованием ее в управлении.

6. Решение задач, для выполнения которых предназначена система.

7. Обеспечение работы локальных и глобальных вычислительных сетей, включая выход в Интернет и др.

Математическое обеспечение делится на две части: общесистемное математическое обеспечение (внутреннее) и специальное (внешнее или прикладное).

Общесистемное математическое обеспечение – совокупность средств математизации для обеспечения разработки и эксплуатации программ регулярного применения.

Общесистемное МО предназначено для решения проблем, возникающих в системе, включая локальные, глобальные вычислительные системы и Интернет. Иногда алгоритмы сетевого взаимодействия выделяют в самостоятельную часть математического обеспечения. В комплекс средств общесистемного математического обеспечения входят:

- алгоритмы испытания, контроля, наладки ЭВМ и выявления неисправностей работы;
- методы и модели для организации вычислительного процесса;
- алгоритмы распределения машинного ресурса времени и других ресурсов;
- средства обработки заданий при подготовке данных;
- средства обслуживания коммуникаций и защиты информации;
- методы и алгоритмы контроля документированной информации;
- различные стандартные методы и алгоритмы, обеспечивающие управление и обслуживание вычислительных средств ПС и т.д.

Специальное математическое обеспечение – совокупность математических средств, предназначенных для решения по типовым схемам задач функциональных подсистем.

Специальное МО создается для решения задач процесса информатизации: учета, оперативного, текущего и календарного планирования развития производства, распределения ресурсов, материально-технического снабжения, кадрового учета, бухгалтерской и финансовой деятельности, отчетности и т.д. Специальное математическое обеспечение включает:

- алгоритмы для решения задач инфообработки для последующего ввода в машину;

– методы и модели, реализованные в библиотеках стандартных программ для решения типовых задач, встречающихся у различных потребителей;

– специальные математические средства, создаваемые каждым потребителем для решения отдельных задач.

Алгоритмы специального математического обеспечения определяются производственно-экономическими факторами, однако их программная реализация преимущественно зависит от особенностей обработки информации на ЭВМ. Во многих системах до 90% программы специального экономического обеспечения могут быть скомпонованы из программ общесистемного математического обеспечения.

1.2.8. Программное обеспечение ПС

Программное обеспечение (ПО), как правило, включается в математическое обеспечение как базовая его часть и является идеологическим содержанием ПС. К математическому обеспечению относятся математические методы, математические модели и алгоритмы, позволяющие провести математическую формализацию функций ПС и последующую их программную реализацию.

Математическая модель объекта может уточняться во время эксплуатации исходя из фактического состояния объекта. Алгоритм функционирования ИС представляет собой определенную последовательность элементарных операций и логических операций, доступных ЭВМ и позволяющих обрабатывать входную информацию о процессе информатизации и вычислять различную результатную информацию, обеспечивающую формирование выходных инфопотоков. В последнем случае вычисления производятся в темпе с технологией подготовки документов или даже в опережающем темпе – системы реального времени. Программное обеспечение разрабатывается при проектировании ПС на базе математического. Программное обеспечение необходимо для разработки и эксплуатации программ на ЭВМ. Программы как

последовательность действий над данными вместе с этими данными записываются в цифровом виде в память ЭВМ.

Программное обеспечение ПС – это комплекс компьютерных программ, обеспечивающих обработку и передачу данных, а также документация по их конфигурированию и применению. ПО ПС включает в себя системное, служебное, инструментальное, прикладное и специальное ПО.

К программному обеспечению, подлежащему разработке в процессе рабочего проектирования, относится категория специального ПО.

Специальное ПО – это совокупность программ, непосредственно реализующих алгоритмы решения функциональных задач.

Специальное ПО разрабатывается при проектировании ПС на базе математического обеспечения и представляет собой программную реализацию компонентов специального МО. Зачастую включает в себя интерфейсные средства: web-страницы, формы, отчеты, модули кода для реализации серверной части специального ПО, а также запросы, представления, курсоры, хранимые процедуры, макросы и другие модули программного кода. По результатам разработки специального ПО строится его структурная схема в виде диаграммы компонентов UML либо в произвольной нотации.

Системное ПО – это операционные системы, управляющие функционированием средств вычислительной техники, сетевого оборудования и прикладного ПО.

Служебное ПО – программные средства, выполняющие функции по обслуживанию компьютерной техники и программ других классов: различные утилиты, средства диагностики, антивирусные программы и т. п.

Инструментальное ПО – это совокупность программных средств, необходимых для проектирования и разработки ПС: CASE-средства, СУБД, средства разработки ПО (IDE), интерпретаторы и трансляторы программ и т. п.

Прикладное ПО – совокупность программ, необходимых для обеспечения функционирования программ решения задач

управления объектом: внешние библиотеки для прикладных программ, средства защиты данных, офисное ПО, графические пакеты, браузеры, а также иные программы, необходимые для полноценного использования специального ПО.

На стадии технического проектирования ПС проводится анализ и обоснованный выбор инструментального ПО (СУБД, IDE, CASE-средства и др.). Для анализа выбирается не менее двух современных систем (языков) программирования или инструментальных средств разработки (IDE), которые пригодны для разработки соответствующих программ.

Совокупность рассмотренных видов ПО ПС принято объединять в понятие общесистемного ПО.

Общесистемное программное обеспечение – совокупность средств автоматизации программирования и стандартных программ регулярного применения, используемых как программная база при составлении специальных программ решения производственно-экономических задач и для обеспечения функционирования системы в целом.

По результатам разработки общесистемного ПО строится его структурная схема в виде древовидной блок-схемы либо организационной диаграммы. Пример такой схемы показан на рис. 1.9.

Проектные решения по созданию ПС, как правило, не обходятся без моделей ПС на языке UML. Данное средство проектирования будет рассмотрено в главе 2 в рамках реализации объектно-ориентированного подхода к проектированию ПС.

1.2.9. Технико-экономическое обоснование проектных решений

Экономически целесообразность внедрения на предприятии информационных систем определяется на основе критериев, учитывающих формы собственности на этих предприятиях и рыночные отношения. Эта задача должна решаться в бизнес-плане, разрабатываемом на предприятии, на котором будет использоваться создаваемая ИС.

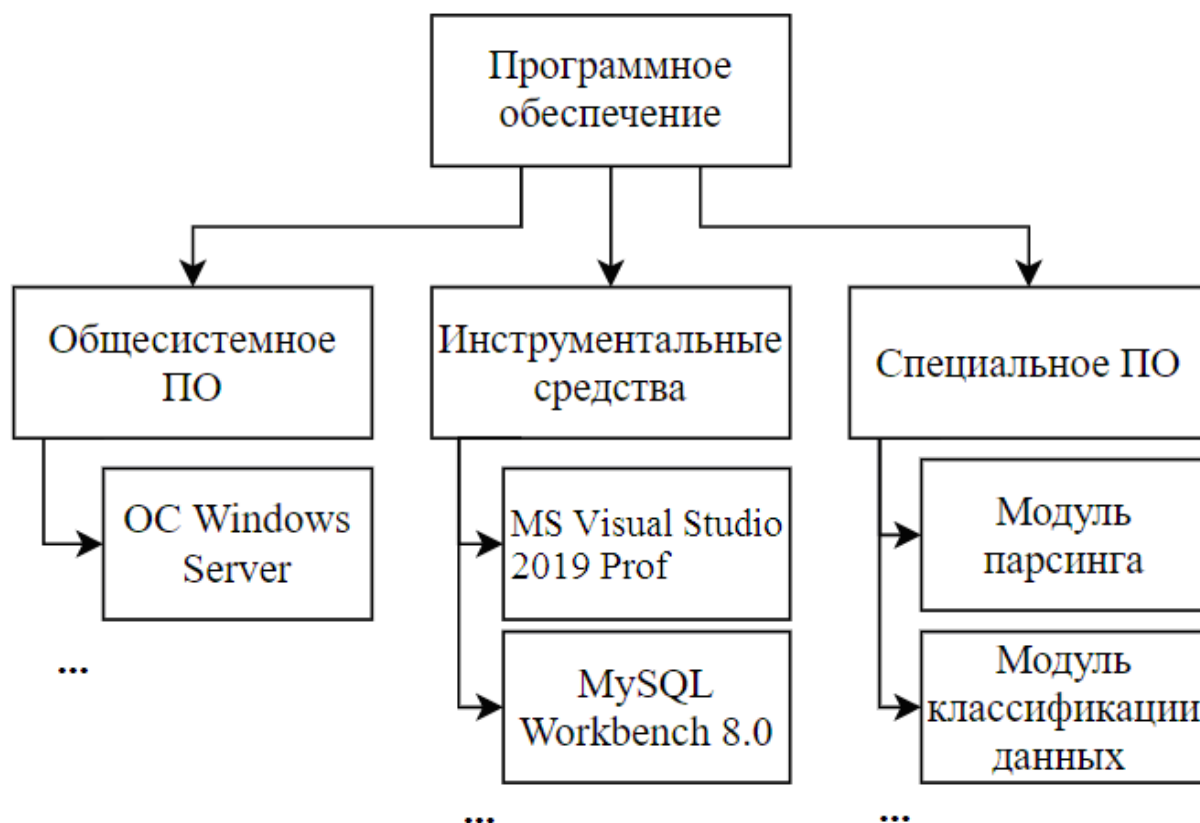


Рис. 1.9. Структурная схема общесистемного программного обеспечения (пример)

Международная практика обоснования проектов использует несколько обобщающих показателей, позволяющих подготовить решение целесообразности (либо нецелесообразности) вложения средств. Рекомендуется использовать при выборе решений проекта ПС следующие показатели: чистая текущая стоимость, внутренний коэффициент эффективности, период возврата капитальных вложений.

Показатель чистой текущей стоимости, или интегральный экономический эффект, представляет собой разность совокупного дохода от реализации продукции, рассчитанного за период реализации проекта, и всех видов расходов, суммированных за этот период, с учетом фактора времени. Максимум чистой текущей стоимости выступает как один из важнейших критериев при обосновании проекта ПС.

1. Для случаев, где внедрение проекта ПС обеспечивает увеличение дохода, определяется по формуле

$$E_{\Pi} = \sum_{t=1}^T \frac{D_t}{(1 + E_{BK})^t} - \sum_{t=1}^T \frac{S_t}{(1 + E_{BK})^t} - K_C \Rightarrow \max,$$

где E_{Π} – чистая текущая стоимость (интегральный экономический эффект), р.; D_t – валовой доход от реализации проекта (продукции) в t -ом г., р.; S_t – эксплуатационные расходы, связанные с использованием проекта в t -ом г., р.; E_{BK} – внутренний коэффициент эффективности, который можно принимать в зависимости от класса инвестиций (табл. 1.7) в долях единицы. В расходы включаются затраты по себестоимости, налоги на доход, на добавленную стоимость, платежи, отчисления.

2. Для случаев, где внедрение проекта ПС обеспечивает только экономию по некоторым факторам, определяется по формуле

$$E_{\Pi} = \sum_{t=1}^T \frac{\sum_{n=1}^N E_{\Pi}}{(1 + E_{BK})^t} - K_C \Rightarrow \max,$$

где E_{Π} – экономия затрат по n -му фактору, р./год; N – количество факторов экономии; n – один из факторов экономии; T – срок эксплуатации системы, лет; t – текущее время, $t = 0, 1, 2, \dots$; K_C – капиталовложения в создание и внедрение ПС, руб.; E_{BK} – внутренний коэффициент эффективности, определяется аналогично значениям из табл. 1.7.

Таблица 1.7

Классы инвестиций в создание ИС

Класс инвестиции	Внутренний коэффициент эффективности
Вложения с целью сохранения позиций на рынке	6
Обновление основных производственных фондов	12
Вложения с целью экономии текущих затрат	15
Вложения с целью увеличения доходов	20
Рисковые капитальные вложения	25

Если E_{Π} имеет положительное значение, то следует вкладывать средства в данный проект. Если рассматриваются два или более вариантов проекта ПС, то принимается тот, у которого E_{Π} больше.

$$\sum_{n=1}^N E_n = E_1 + E_2 + E_3 + E_4 + E_5,$$

где E_1 – экономия за счет увеличения выпуска продукции, р./год;
 E_2 – от увеличения продукции более высокого качества, р./год;
 E_3 – за счет снижения выпуска бракованной продукции, р./год;
 E_4 – за счет снижения расхода материалов, топлива, энергии, р./год;
 E_5 – за счет снижения трудовых затрат по автоматизированной обработке информации, р./год.

Внутренний коэффициент эффективности $E_{В.К}$ может определяться аналитически как такое пороговое значение рентабельности, которое обеспечивает равенство нулю интегрального эффекта, рассчитанного за экономический срок жизни инвестиций. Проект считается рентабельным, если внутренний коэффициент эффективности не ниже исходного порогового значения.

Кроме того, могут быть рассчитаны следующие показатели технико-экономической эффективности проекта по созданию ПС.

Период возврата капиталовложений представляет собой количество лет, в течение которых доход от продаж (реализации) за вычетом издержек возмещает основные капиталовложения. Этот показатель может быть установлен на графике, изображающем финансовый профиль проекта.

Предпринимательский риск – возможное отклонение от цели инвестиционного проекта (недополучение доходов / возникновение убытков) из-за влияния случайных (непредвиденных) факторов.

Порог рентабельности (бесприбыльный оборот) – величина выручки от реализации, при которой предприятие уже не имеет убытков, но ещё не имеет и прибыли.

Пороговое количество продукции (норма безубыточности) – это минимальный размер партии выпускаемой продукции, объём работ (услуг), при котором

обеспечивается нулевая прибыль (доход от деятельности равен издержкам производства).

1.3. ПОДХОДЫ К ПРОЕКТИРОВАНИЮ ПС

1.3.1. Методологии и технологии проектирования ПС

Методология и технология проектирования составляют основу проекта любой ПС. Методология реализуется через конкретные технологии и поддерживающие их стандарты, методики и инструментальные средства (CASE-средства), которые обеспечивают выполнение процессов ЖЦ.

Технология проектирования определяется как совокупность четырех составляющих.

1. Пошаговая процедура, определяющая последовательность технологических операций проектирования. Можно представить себе технологию проектирования в виде цепочки технологических операций, связанных между собой. Представление технологических операций проектирования, как звена этой цепочки, показано на рис. 1.10.

2. Критерии и правила для оценивания результатов выполнения технологических операций.



Рис. 1.10. Представление технологической операции проектирования

3. Нотации (графические и текстовые средства), используемые для описания проектируемой системы.

4. Технологические инструкции, составляющие основное содержание технологии, включают описания последовательности технологических операций, условий их выполнения, а также описания самих операций.

Технология проектирования, разработки и сопровождения ПС должна обеспечивать определенные возможности.

1. Поддержка полного цикла (ЖЦ) создания ПС.

2. Гарантированное достижение целей создания ПС с заданным качеством, в установленное время и определённый бюджет проекта.

3. Возможность выполнения крупных проектов в виде подпроектов (возможность декомпозиции проекта на составные части, разрабатываемые группами исполнителей ограниченной численности с последующей интеграцией составных частей). Опыт разработки крупных ПС показывает, что для повышения эффективности работ необходимо разбить крупный проект на отдельные слабо связанные по данным и функциям подпроекты. Реализация подпроектов должна выполняться отдельными группами специалистов. При этом необходимо обеспечить координацию ведения общего проекта и исключить дублирование результатов работ каждой проектной группы, которое может возникнуть в силу наличия общих данных и функций.

4. Возможность ведения работ по проектированию отдельных подпроектов небольшими группами (3–7 человек). Это обусловлено принципами управляемости коллектива и повышения производительности за счет минимизации числа внешних связей.

5. Минимальное время получения работоспособной ПС. Речь идет не о сроках готовности всей ПС, а о сроках реализации отдельных подсистем. Реализация ПС в целом в короткие сроки может потребовать привлечения большого числа разработчиков. При этом эффект может оказаться ниже, чем при реализации в более короткие сроки отдельных подсистем ПС меньшим числом

разработчиков. Практика показывает, что даже при наличии полностью завершеного проекта внедрение идет последовательно по отдельным подсистемам.

6. Возможность управления конфигурацией проекта, ведения версий проекта и его составляющих.

7. Возможность автоматического выпуска проектной документации и синхронизации ее версий с версиями проекта.

8. Независимость выполняемых проектных решений от средств реализации ПС (СУБД, операционных систем, языков и систем программирования).

9. Поддержка комплекса согласованных CASE-средств, обеспечивающих автоматизацию процессов на всех стадиях ЖЦ.

1.3.2. Стандарты проектирования ПС

Реальное применение любой технологии проектирования, разработки и сопровождения ПС в конкретной организации и конкретном проекте требует выработки ряда стандартов (правил, соглашений), которые должны соблюдаться всеми участниками проекта. Рассмотрим эти стандарты.

1. Стандарт проектирования должен устанавливать:

- набор необходимых моделей (диаграмм) на каждой стадии проектирования и степень их детализации;

- правила фиксации проектных решений на диаграммах, в том числе правила именования объектов (включая соглашения по терминологии), набор атрибутов для всех объектов и правила их заполнения на каждой стадии, а также правила оформления диаграмм, включая требования к форме и размерам объектов и т.д.;

- требования к конфигурации рабочих мест проектировщиков, включая настройки операционной системы, настройки CASE-средств, общие настройки проекта и т.д.;

- механизм обеспечения совместной работы над проектом, в том числе правила интеграции подсистем проекта, правила поддержания проекта в одинаковом для всех разработчиков состоянии (регламент обмена проектной информацией, механизм

фиксации общих объектов), правила проверки проектных решений на непротиворечивость и т.д.

2. Стандарт оформления проектной документации должен устанавливать:

- комплектность, состав и структуру документации на каждой стадии проектирования;
- требования к оформлению документации;
- требования к содержанию разделов, подразделов, пунктов, таблиц документации;
- правила подготовки, рассмотрения, согласования и утверждения документации с указанием предельных сроков для каждой стадии;
- требования к настройке издательской системы, используемой в качестве средства подготовки документации;
- требования к настройке CASE-средств для обеспечения подготовки документации в соответствии с установленными требованиями.

3. Стандарт пользовательского интерфейса должен устанавливать:

- правила оформления экранных форм (шрифты и цветовая палитра), состав и расположение окон и элементов управления;
- правила использования клавиатуры и мыши;
- правила оформления и содержания текстов помощи;
- перечень стандартных сообщений;
- правила обработки реакций пользователя.

1.3.3. Классификация подходов к проектированию ПС

Существуют различные подходы к представлению, анализу и проектированию систем.

1. Традиционный подход, применяющийся в математических исследованиях: определить элементы-переменные и связать их соответствующим соотношением (формулой, уравнением, системой уравнений), отображающим принцип взаимодействия элементов. Трудно реализуем для сложных систем.

2. Подход обследования («перечисление» системы), который сводится к формированию *пространства состояний* элементов и введению *меры близости* между элементами этого пространства. При обследовании предполагается выявить все элементы и связи между ними, при этом применяются разные способы:

- архивный (изучение документов и архивов предприятия);
- опросный, или анкетный (опрос сотрудников и экспертов).

Также трудно реализуем для сложных систем.

Отметим, что обследование является стартовым подходом при реализации более сложных подходов и может применяться, например, для «перечисления» отдельных автоматизированных функций для облегчения их дальнейшего проектирования структурными методами.

3. Структурные подходы. В настоящее время на основе обобщения предшествующего опыта сформировались два структурных подхода к отображению систем, первоначально предложенные для формирования структур целей:

- формирование системы сверху вниз – методы структуризации, декомпозиции, целевой, или целенаправленный подход;

- формирование системы снизу вверх – подход, который называют морфологическим (в широком смысле) методом «языка» системы. С помощью этого подхода реализуют поиск взаимосвязей (мер близости) между элементами.

Подходы «сверху вниз» и «снизу вверх» называют также *аксиологическим* и *каузальным* соответственно. На практике обычно эти подходы сочетают. Одним из вариантов их комплексного применения является блочно-иерархический подход.

1.3.4. Блочно-иерархический подход

При использовании блочно-иерархического подхода (БИП) представления о проектируемой системе расчленяют на

иерархические уровни. На верхнем уровне используют наименее детализированное представление, отражающее только самые общие черты и особенности проектируемой системы. На следующих уровнях степень подробности описания возрастает, при этом рассматривают отдельные блоки системы с учетом воздействий на каждый из них его соседей. Такой подход позволяет на каждом иерархическом уровне формулировать задачи приемлемой сложности, поддающиеся решению с помощью имеющихся средств проектирования. Разбиение на уровни должно быть таким, чтобы документация на блок любого уровня была обозрима и воспринимается одним человеком.

Список иерархических уровней в каждом случае может быть специфичным. Преимущественно выделяют три уровня.

1. Системный уровень, на котором решают наиболее общие задачи проектирования систем, программно-технических комплексов и бизнес-процессов. Результаты проектирования представляют в виде структурных схем, укрупненных блок-схем, схем размещения оборудования, диаграмм потоков данных, вариантов использования (требований), классов, логической модели данных, генеральных планов и т.п.

2. Макроуровень, на котором проектируют отдельные устройства, подсистемы, подпроцессы. Результаты представляют в виде функциональных и принципиальных схем, детализированных блок-схем, физической модели данных, сборочных чертежей и т.п.

3. Микроуровень, на котором проектируют отдельные детали и элементы подсистем, устройств. Рассматриваются отдельные задачи, подзадачи, строятся модели поведения UML, детализированные блок-схемы алгоритмов и т.д.

1.3.5. Функционально-ориентированный подход

Функционально-ориентированный подход (ФОП) рассматривает объект исследования как набор функций, преобразующий поступающий поток информации в выходной поток. Процесс преобразования информации потребляет

определенные ресурсы. Принципиальное отличие ФОП заключается в четком отделении функций (методов обработки данных) от самих данных.

Распространенной методикой реализации ФОП считается IDEF0 (Integration Definition for Function Modeling), являющаяся следующим этапом развития графического языка описания функциональных систем SADT (Structured Analysis and Design Technique, технология структурного анализа и проектирования). Разработка SADT началась в 1969 г. и была опробована на практике в компаниях различных отраслей (аэрокосмическая отрасль, телефония и т.д.). Публично появилась на рынке в 1975 г. и получила широкое распространение в мире. IDEF0 – результат программы компьютеризации промышленности, которая была предложена ВВС США. Автоматизация деятельности предприятий потребовала соответствующих методик и инструментов.

Цель методики – построение функциональной схемы исследуемой системы, описывающей все необходимые процессы с точностью, достаточной для однозначного моделирования деятельности системы. В основе методологии лежат четыре основных понятия: функциональный блок, интерфейсная дуга, декомпозиция, глоссарий.

Функциональный блок (Activity Box) представляет собой некоторую конкретную функцию в рамках рассматриваемой системы. По требованиям стандарта название каждого функционального блока должно быть сформулировано в глагольном наклонении (например, «производить услуги»). На диаграмме функциональный блок изображается прямоугольником с текстом названия внутри.

Интерфейсная дуга (Arrow) отображает элемент системы, который либо обрабатывается функциональным блоком, либо является результатом этой обработки, либо оказывает иное влияние на функцию, представленную данным функциональным блоком. С помощью дуг отображают объекты, в той или иной степени определяющие процессы, происходящие в системе. Это могут быть элементы реального мира (детали, вагоны,

сотрудники и т.д.) или инфопотоки (документы, данные, инструкции и т.д.). В зависимости от того, к какой из сторон функционального блока подходит дуга, она носит название «входящей» (слева), «исходящей» (справа) или «управляющей» (сверху и снизу).

Любой функциональный блок по требованиям стандарта должен иметь по крайней мере одну управляющую интерфейсную дугу и одну исходящую. В отношении построения дуг действуют следующие правила:

- любая дуга может иметь ответвление;
- слияние дуг не допускается;
- все дуги должны быть именованы;
- во всей иерархии диаграмм не должно быть дуг с одинаковыми именами (за исключением периферийных дуг диаграммы декомпозиции, дублирующих внешнее дополнение родительского блока).

Декомпозиция (Decomposition) является основным понятием стандарта IDEF0. Принцип декомпозиции применяется при разбиении сложного процесса на составляющие его функции (подробнее см. п. 1.2.2). При этом уровень детализации процесса определяется непосредственно разработчиком. Декомпозиция позволяет постепенно и структурированно представлять модель системы в виде иерархической структуры отдельных диаграмм, что делает ее менее перегруженной и легко усваиваемой.

Модель IDEF0 всегда начинается с представления системы как единого целого – одного функционального блока с интерфейсными дугами, простирающимися за пределы рассматриваемой области. Такая диаграмма с одним функциональным блоком называется *контекстной диаграммой* и представляет собой реализацию принципа черного ящика. При этом интерфейсные дуги отражают внешнее дополнение системы согласно принципу внешнего дополнения. Принципы кибернетики, используемые при проектировании ПС, рассмотрены в п. 1.1.5.

Дальнейшее проектирование требует уяснения функциональной структуры ПС, представленной в виде черного

ящика. Это лучше делать посредством перечисления её автоматизированных функций. Перечисление функций выполняется по пунктам простыми повествовательными предложениями. Объем текста описания для несложной информационной функции ПС обычно составляет 30–50 слов. В случае описания сложной вычислительной или управляющей функции ПС слов может быть больше в разы. При этом в описание при необходимости могут быть включены формулы, таблицы и дополнительные иллюстрации. Так, в описание функции операции обмена валюты может быть представлена формула для расчета суммы к выдаче. Пример контекстной диаграммы для ПС учета обмена валют показан на рис. 1.11.

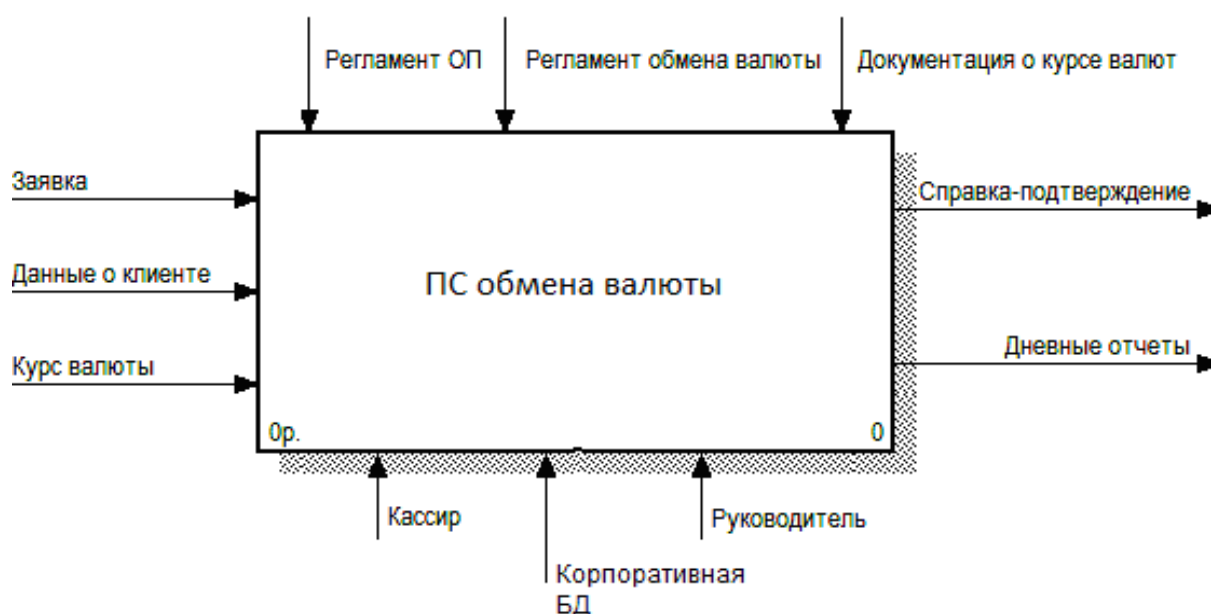


Рис. 1.11. Контекстная диаграмма ПС выполнения операции обмена валюты в BPWin

В процессе декомпозиции базовый функциональный блок подвергается детализации на другой диаграмме. Получившаяся диаграмма второго уровня содержит функциональные блоки, отображающие главные подфункции функционального блока контекстной диаграммы, и называется дочерней (Child Diagram) по отношению к нему, а каждый функциональный блок дочерней диаграммы – дочерним блоком (Child Box). В свою очередь

функциональный блок-предок называется родительским блоком по отношению к дочерней диаграмме (Parent Box), а диаграмма, к которой он принадлежит, – родительской диаграммой (Parent Diagram). Каждая из подфункций дочерней диаграммы может быть далее детализирована путем аналогичной декомпозиции соответствующего ей функционального блока. В каждом случае декомпозиции функционального блока все интерфейсные дуги, входящие в данный блок или исходящие из него, фиксируются на дочерней диаграмме. Этим достигается структурная целостность IDEF0-модели.

Диаграмма декомпозиции контекстной диаграммы ПС обмена валют (рис. 1.11) показана на рис. 1.12. Управляющая дуга «Корпоративная БД» отражает механизм автоматического запроса курса валюты при выполнении функции «Операция обмена валюты».

Дальнейшая декомпозиция до уровня операций производится в отдельности для каждого из четырех подпроцессов – блоков диаграммы, показанной на рис. 1.12.

Обычно IDEF0-модели несут в себе сложную и концентрированную информацию. Для того чтобы ограничить их перегруженность и сделать удобочитаемыми, в стандарте приняты соответствующие ограничения сложности:

- количество блоков на диаграмме от 3 до 6;
- количество интерфейсных дуг на один функциональный блок не более 5;
- любой функциональный блок должен иметь не менее одной управляющей (Control, Mechanism) и одной исходящей (Output) дуги;
- компоновка блоков и интерфейсных дуг с подписями должна обеспечивать максимальную читабельность диаграммы.

Преимущества ФОП выражаются в наглядности графического языка, структурной целостности модели и преемственности при организации новых проектов. Недостатком его является сложность проектирования, возрастающая вместе со сложностью проектируемой системы.

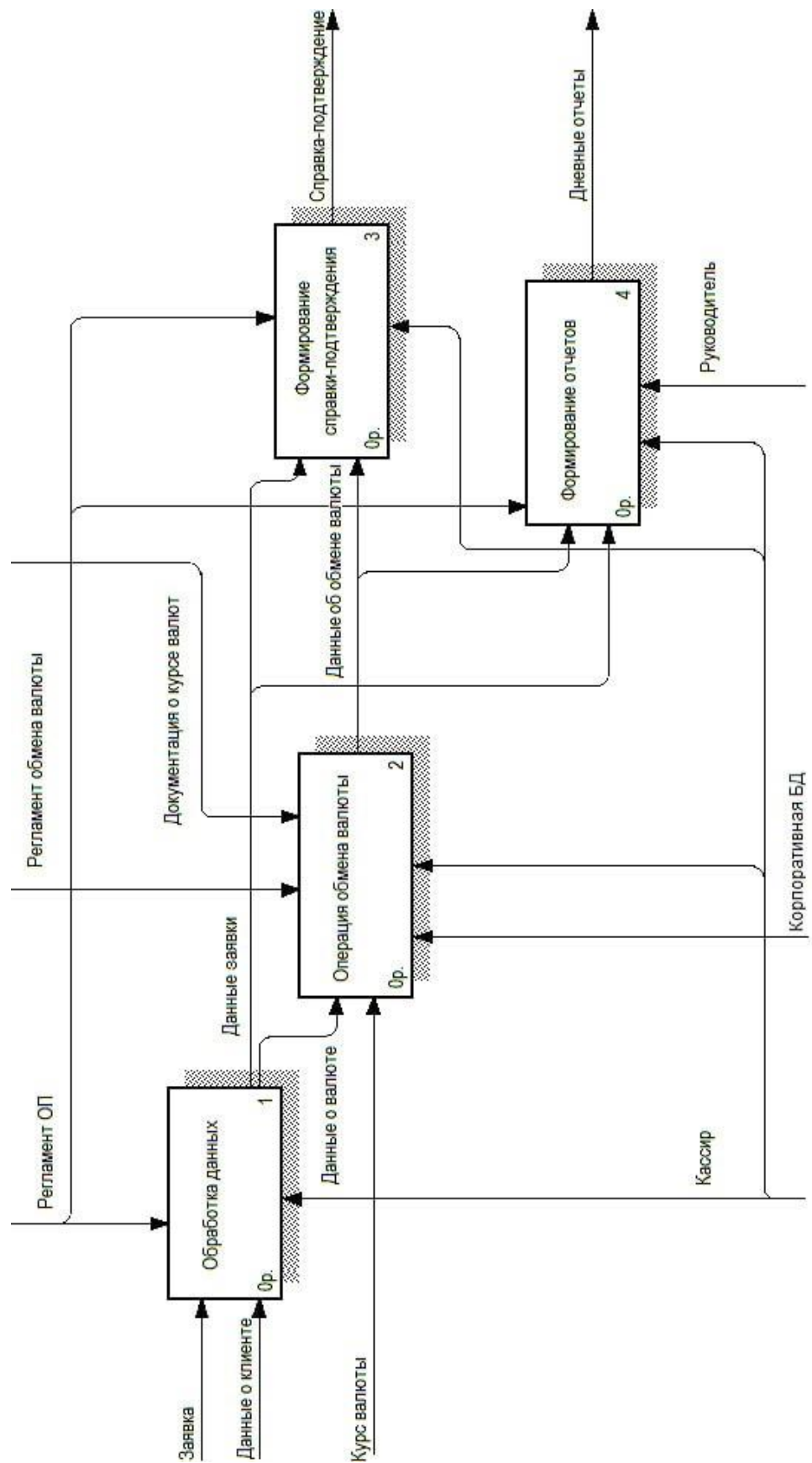


Рис. 1.12. Диаграмма декомпозиции ПС обмена валюты, выполненная в BPWin

Основной недостаток функционального моделирования связан с отсутствием явных средств для объектно-ориентированного представления моделей сложных систем. Некоторые аналитики отмечают важность знания и применения нотации IDEF0, однако отсутствие возможности реализации соответствующих графических моделей в объектно-ориентированном программном коде существенно сужают диапазон задач, решаемых с ее помощью.

1.3.6. Объектно-ориентированный подход

Как БИП, так и ФОП имеют существенный недостаток: сложность проектирования, возрастающая вместе со сложностью проектируемой системы. Указанного недостатка лишено объектно-ориентированное проектирование. Здесь подразумевается, что существует набор автономно действующих подсистем, взаимодействующих с целью обеспечения поведения системы, соответствующего более высокому уровню. Объектно-ориентированный подход (ООП) позволяет создавать более открытые и модифицируемые системы, поскольку конструкция таких систем базируется на устойчивых промежуточных формах – объектах системы. Объекты в свою очередь служат экземплярами классов. Понятие о классе является основополагающим понятием ООП.

Класс (class) – абстрактное описание множества однородных объектов, имеющих одинаковые атрибуты, операции и отношения с объектами других классов.

Для модификации проекта бывает достаточно замены одного объекта из иерархической цепочки наследования классов. Те из них, которые находятся на вершине цепочки, могут использоваться для реализации простейших функций. Чем ниже спускаемся, тем с более сложным поведением получаем объекты.

Преимуществами использования ООП являются:

- естественность представления в виде объектов модели реального мира;

- управляемость разработкой проекта и восприимчивость его к модернизации;
- логичность и наглядность в манипуляциях с данными;
- применение объектной базы данных для хранения неоднородной информации.

К недостаткам ООП относятся:

- высокие начальные затраты;
- длительная окупаемость.

Эффект от применения ООП виден после разработки двух-трех проектов и накопления повторно используемых компонентов.

Для реализации ООП на практике широко используются CASE-средства, поддерживающие язык UML. Подробно рассмотрен ООП посредством UML в параграфе 2.1.

1.4. ДОКУМЕНТИРОВАНИЕ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

Создание документации является важным этапом в разработке программного обеспечения.

Документирование – вспомогательный процесс жизненного цикла ПС, обеспечивающий выполнение его основных процессов посредством всеобъемлющей четкой и понятной фиксации проектных решений по созданию ПС на всех этапах её жизненного цикла в форме документов машиночитаемого формата.

Документирование ПС помогает разработчикам и отделу сопровождения понимать структуру системы и назначение ее элементов. Часть документации предназначена для конечных пользователей и администраторов, она помогает понимать, как устанавливать и использовать систему, а также как устранить ошибки. Документация может быть представлена в виде письменного текста или иллюстраций, которые прилагаются к ПС или встроены в её исходный код. Назначение документации – объяснить, как работает ПС или как её использовать, и это может быть важно для людей, выполняющих разные роли.

Современная документация доступна в нескольких форматах, включая наиболее старые – печатные документы, а также электронную документацию и документацию, размещенную в сети Интернет. Печатные документы предоставляют преимущества легкости восприятия и упрощения поиска по неформальным запросам, в то время как электронные документы могут быть более детальными и легче для поиска по ключевым словам. Web 2.0-документация включает в себя wiki, записи в блогах и ответы на сайтах, созданные как разработчиками, так и пользователями системы. Документацию можно писать вручную с использованием систем редактирования и верстки (например, Microsoft Word или LaTeX). Более продуктивный способ – использование автоматических генераторов документации из кода, таких как Javadoc, PHPDoc, Doxygen. В этом случае документирование осуществляется в исходных файлах программной системы, чаще всего в виде специальных комментариев (хотя возможны другие варианты, например, строки документации в Python). Такую документацию можно читать при просмотре исходного кода программы. Она же используется для контекстной помощи в средах разработки. Наконец, при помощи генератора документацию можно выделить в отдельные документы (например, HTML или страницы man). Развитие идеи комментирования исходного кода программы – грамотное программирование (literate programming), разработанное Дональдом Кнутом для системы TeX. В этом подходе роли комментариев и кода меняются: текст программы представляет собой эссе на естественном языке с отдельными фрагментами кода. С помощью специальных инструментов код выделяется из эссе и формирует исходный код на определенном языке программирования, после чего можно проводить компиляцию и т.д. Грамотное программирование позволяет проследить ход мысли разработчика программы, лучше понять ее структуру и решения, принятые при разработке. При всех своих преимуществах грамотное программирование применяется редко – не все программисты любят писать эссе.

При осуществлении рабочего проектирования ПС создаются следующие виды документации:

- функциональные спецификации – описывают функционирование отдельных компонентов ПС и системы в целом в отношении реализации функциональных требований к системе;

- проектные спецификации – раскрывают проектные решения, которые необходимы заказчику для выполнения функций сопровождения ПС;

- руководство пользователя – документ, регламентирующий действия пользователя при работе с ПС.

Документирование часто сопровождается диаграммами UML, которые позволяют визуально описывать приложение, наглядно представлять архитектуру и требования к приложению.

На этапе тестирования ПС готовятся и документируются тесты для проверки и верификации ПС, а также результаты этих мероприятий:

- тестовые случаи – документ, определяющий последовательность тестовых действий, или листинг программы автоматического теста;

- ошибки (несоответствия) – документированные сведения о несоответствиях, выявленных в процессе тестирования.

Основное требование к документации – она должна быть понятна и соответствовать функциональной и обеспечивающей структуре ПС. Основные ошибки при документировании ИС – это нечеткие формулировки, игнорирование аудитории, для которой предназначена данная документация, а также пропуск важных аспектов, связанных с нефункциональными требованиями.

1.4.1. Классификация документации ПО

В документации ПС можно выделить два типа документов: те, которые описывают процессы разработки (например, спецификации требований и планы проектирования), и те, которые описывают результаты этих процессов (например,

руководства пользователя и UML-схемы модулей). Согласно гибкой методологии разработки, объем документации должен быть минимальным, чтобы не тратить много времени и средств на ее составление, учитывать быструю эволюцию структуры программной системы, из-за которой документация быстро устаревает (более подробно данное предписание рассмотрено в п. 3.5.1). Разумеется, полностью отказаться от документации нельзя: в любом случае сохраняется потребность в пользовательской документации, такой как руководства пользователя. Существуют различные типы документации ПС.

1. Спецификации требований – документ, который описывает функциональные и нефункциональные требования к программному продукту. В этом документе указываются требования к функциональности, производительности, безопасности и другим аспектам системы.

2. Планы проектирования – документ, который описывает план работы над проектом, включая расписание работ, бюджет, ресурсы и другие аспекты проекта. В этом документе также могут быть описаны методы и инструменты, используемые для разработки ПО.

3. Руководства пользователя – документ, который описывает функциональность программного продукта и инструкции по его использованию. В этом документе могут быть описаны основные функции системы, интерфейс пользователя и другие аспекты, которые помогут пользователю использовать программу.

4. Техническая документация – документ, который описывает технические аспекты программного продукта, такие как архитектура, базы данных, сетевые протоколы и т.д. Этот документ может быть полезен для разработчиков, которые работают над различными аспектами системы.

5. UML-схемы модулей – документ, который описывает структуру и взаимодействие различных модулей программного продукта. Этот документ может быть полезен для разработчиков, которые работают над различными модулями системы и нуждаются в понимании их взаимодействия.

6. Тестовая документация – документ, который описывает тестовые случаи и результаты тестирования программного продукта. Этот документ может быть полезен для тестировщиков, которые работают над проверкой функциональности и качества системы.

Пользовательская документация, или руководство пользователя, объясняет пользователям, как они должны действовать, чтобы применить ПС по её назначению. Она необходима, если ПС предполагает какое-либо взаимодействие с пользователями. К такой документации относятся документы, которыми должен руководствоваться пользователь при установке ПО, при применении ПС для решения своих задач и при управлении ПС (например, при взаимодействии ПС с другими системами). В связи с этим следует различать две категории пользователей ПС: ординарных пользователей и администраторов.

Ординарный пользователь ПС использует систему для решения своих задач (в рамках своей предметной области). Он может не знать многих деталей работы компьютера или принципов программирования. Администратор же управляет использованием ПС обычными пользователями и поддерживает ПС в части, не связанной с модификацией её компонентов. Например, он может регулировать права доступа к ПС между обычными пользователями, связываться с поставщиками типовых компонентов ПС (например, ПК, сетевого оборудования, общесистемного или инструментального ПО) или предпринимать определенные действия для поддержания работы ПС, когда она является частью другой системы.

Состав пользовательской документации зависит от того, для какой аудитории она предназначена. Каждый документ должен содержать информацию, необходимую для конкретной аудитории. Применение документации зависит от того, какой ее вид пользователь выбирает. Обычно для изучения больших программных комплексов пользователи применяют документацию в качестве руководства, а для уточнения информации – в качестве справочника.

Примерная структура руководства пользователя.

1. Назначение приложения. Здесь задаются общие понятия, необходимые навыки для работы с системой, такие как название приложения и номер версии, класс ПС, для которых предназначено приложение, характеристики пользовательского интерфейса и др.

2. Установка приложения. В этой части описывается процесс установки и базовой настройки системы.

3. Концепция приложения. Приводится общая структура системы, справочник терминов и понятий, которые используются в интерфейсах управления системой в данном руководстве, описывается процесс функционирования в общем виде.

4. Функциональный состав приложения. Рассматривается процесс решения задач на объекте в рамках функционального состава ПС. По ходу описания приводятся экранные формы всех интерфейсных окон или страниц, посредством которых пользователи взаимодействуют с приложением и устанавливаются ссылки на страницы.

5. Инструменты и настройки приложения. Описывается назначение и применение настроек приложения, работа с модулями, обновлениями и полезными инструментами системы.

6. Получение помощи. Приводятся способы решения возникших проблем и получения помощи (Help).

1.4.2. Проблемы стандартов единой системы программной документации

В основе отечественной нормативной базы в области документирования ПО лежит комплекс стандартов Единой системы программной документации (ЕСПД). Система межгосударственных стандартов стран СНГ (ГОСТ), действующих на территории Российской Федерации, представляет собой основную и бóльшую часть комплекса ЕСПД, который был разработан в 1970-е и 1980-е гг. Сейчас этот комплекс является системой межгосударственных стандартов, действующих на территории Российской Федерации на основе

межгосударственного соглашения по стандартизации. ЕСПД устанавливает правила разработки, оформления и обращения программ и программной документации, преимущественно ту часть, которая создается в процессе разработки ПС и связана с документированием функциональных характеристик ПС. Стандарты ЕСПД носят рекомендательный характер, однако они становятся обязательными на контрактной основе при ссылке на них в договоре на разработку или поставку ПС в соответствии с Законом РФ «О стандартизации». В состав ЕСПД входят:

- основополагающие и организационно-методические стандарты;
- стандарты, определяющие формы и содержание программных документов, применяемых при обработке данных;
- стандарты, обеспечивающие автоматизацию разработки программных документов.

1.4.3. Актуальные вопросы при разработке документации

Во все времена при разработке технической документации для ПС будут актуальны следующие основные вопросы.

1. Какова нормативная база и как её следует применять?
2. Какая именно документация нужна среди большого количества её видов?

В настоящее время действуют следующие стандарты документирования:

- ГОСТ 19.201 (Единая система программной документации – ЕСПД);
- ГОСТ 2.015-2013 (Единая система конструкторской документации – ЕСКД);
- ГОСТ 34.602 (Комплекс стандартов на автоматизированные системы – КСАС).

Указанные стандарты с 2006 г. применять не обязательно. В соответствии с нормами Федерального закона № 184-ФЗ «О техническом регулировании» вместо стандартов применяются специально разработанные технические регламенты. Но ГОСТ по-прежнему нужно применять при разработке документации для

государственных заказчиков, а также для крупных организаций (особенно с государственным участием). Последние, как правило, разрабатывают собственные стандарты на основе указанных ГОСТ.

Следует учитывать, что в соответствии с ФЗ «О техническом регулировании» национальные стандарты всегда имеют приоритет перед международными. Использование международных стандартов допустимо только в том случае, если они не противоречат национальным. Отечественные стандарты предоставляют больше свободы действий, чем зарубежные. Они пересматриваются каждые 5–7 лет и характеризуются большей конкретизацией, отображая весь актуальный опыт за указанный период времени. Отечественные стандарты также характеризуются глубокой разработкой концептуальных моментов, что позволяет создавать на их основе стандарты, соответствующие требованиям времени.

2. ИНСТРУМЕНТЫ АВТОМАТИЗИРОВАННОГО ПРОЕКТИРОВАНИЯ ПРОГРАММНЫХ СИСТЕМ

Методология автоматизированного проектирования ПС тесно связана с использованием CASE-средств (computer aided software engineering). Существует огромное количество инструментальных средств, применяемых для проектирования ПС от анализа до создания программного кода. Среди них отдельно выделяют CASE-средства верхнего уровня и CASE-средства нижнего уровня. При использовании CASE-средств возникают такие проблемы, как жесткие требования к процессу разработки, применение ресурсов, трудности в организации взаимодействия между командами, работающими над различными элементами проекта, и даже адаптация под конкретный проект. Подробный обзор CASE-средств дан в параграфе 2.2. Прежде всего рассмотрим языковые средства визуального моделирования ПС на стадиях её жизненного цикла.

В современной сфере разработки ПС, которая, как правило, осуществляется в объектно-ориентированном стиле, становится все труднее создавать и поддерживать приложения, обладающие высоким качеством и разработанные за приемлемое время. Унифицированный язык моделирования UML появился в ответ на эту потребность. UML – своего рода вариант чертежа, принятый в индустрии информационных технологий. Это метод детального описания архитектуры системы. С его помощью легче создавать и сопровождать систему, вносить в нее требуемые изменения, составлять план тестирования и документацию готовой системы или ее части.

2.1. UML И ЕГО ПРИМЕНЕНИЕ В ПРОГРАММНОМ ПРОЦЕССЕ

Первые попытки предложить подходящий синтаксис для визуального представления схем БД и ПО воспринимались разработчиками скептически и неоднозначно. На этом фоне разработка и стандартизация унифицированного языка

моделирования UML вызвала воодушевление у всего сообщества программистов.

Унифицированный язык моделирования (Unified Modeling Language, UML) – стандартизированное языковое средство, реализующее объектно-ориентированный подход к проектированию и направленное главным образом на создание и поддержку высококачественных проектов в сфере информационных технологий.

Язык UML был разработан компанией Rational Software Corporation для унификации лучших свойств, которыми обладали более ранние методы и способы нотации (рис. 2.1). В 1997 г. организация OMG признала его в качестве стандартного языка моделирования. С тех пор UML получил дальнейшее развитие и широкое признание в отрасли ИТ.

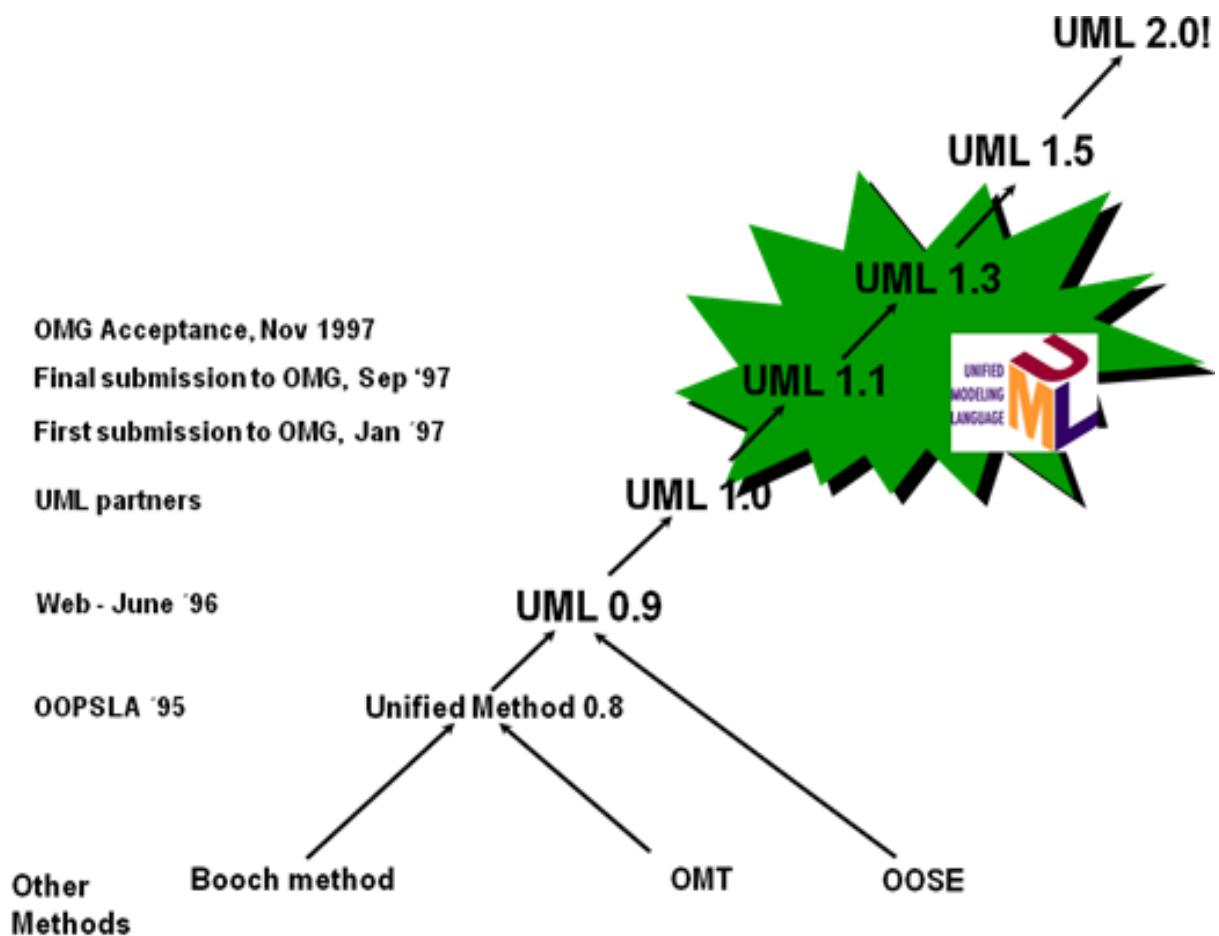


Рис. 2.1. История развития языка UML

Термин «унифицированный» не означает претензий на универсальность. UML был создан путем объединения трех языков: Booch'91, OMT (Object Modelling Technique), OOSE (Object Oriented Software Engineering), носит универсальный характер в плане его применения не только в программной инженерии, но и в других сферах.

Создатели UML характеризуют его как язык для представления, проектирования и документирования программных, организационно-экономических, технических и других систем различной природы.

UML – это язык графического описания, предназначенный для моделирования бизнес-процессов, системного проектирования и отображения организационных структур, моделирования, визуализации, конструирования и документирования программ, проектирования баз данных. UML относится к языкам промежуточного уровня – между формальными и неформальными языками. Это расширяемый (но не переписываемый) язык. Это язык моделирования (но не имитации процессов).

UML – это функциональный язык моделирования общего назначения, который используется для спецификации, визуализации, конструирования и документирования артефактов программной системы. Язык UML основан на некотором числе базовых понятий, которые могут быть изучены и применены большинством программистов и разработчиков, знакомых с методами объектно-ориентированного анализа и проектирования. При этом базовые понятия могут комбинироваться и расширяться таким образом, что специалисты объектного моделирования получают возможность самостоятельно разрабатывать модели больших и сложных систем в самых различных областях приложений.

Основные аспекты модели UML:

- прецеденты;
- сущности (в ООП абстрактные классы, классы и экземпляры);

- описания структур классов, основанные на отношениях наследования, агрегации, композиции, ассоциации, а также других связей;
- состояния объектов или классов, переходы между ними и условия переходов;
- взаимодействия объектов с внешним миром и между собой.

Данные аспекты находят отражение в отдельных диаграммах UML, комплекс которых составляет модель объекта / процесса моделирования.

Диаграмма UML (UML Diagram) – это графическое представление некоторой части графа модели UML.

Диаграммы и есть та основная накладывается на модель структура, которая облегчает создание и использование модели.

UML стандартизирован и встроен во многие CASE-средства и интегрированные среды разработки ПО.

Далее рассмотрим наиболее распространенные версии UML: 1.5 и 2.0.

2.1.1. OMG UML Specification 1.5

Версия UML 1.5 принята в качестве международного стандарта ISO/IEC 19501:2005. Основные документы: UML Summary, UML Semantics, UML Notation Guide, UML Extention, UML CORBA, UML XMI DTD, OCL. Всего в спецификации более 1000 страниц.

В UML-1 определено девять типов диаграмм.

1. Диаграмма вариантов использования (Use Case Diagram) – диаграмма, отражающая отношения между акторами и прецедентами и являющаяся составной частью модели прецедентов, позволяющей описать систему на концептуальном уровне.

2. Диаграмма классов (Class Diagram) – структурная диаграмма языка моделирования UML, демонстрирующая общую структуру иерархии классов системы, их коопераций, атрибутов, методов, интерфейсов и взаимосвязей между ними.

3. Диаграмма объектов (Object Diagram) – диаграмма объектов в языке моделирования UML предназначена для демонстрации совокупности моделируемых объектов и связей между ними в фиксированный момент времени.

4. Диаграмма состояний (Statechart Diagram) – ориентированный граф для конечного автомата, в котором вершины обозначают состояния дуги, показывают переходы между двумя состояниями.

5. Диаграмма деятельности (Activity Diagram) – UML-диаграмма, отображающая действия и последовательность их выполнения, состояние которых описано на диаграмме состояний.

6. Диаграмма последовательности (Sequence Diagram) – UML-диаграмма, на которой для некоторого набора объектов на единой временной оси показан жизненный цикл объекта и взаимодействие акторов информационной системы в рамках прецедента.

7. Диаграмма кооперации (Collaboration Diagram) – диаграмма, на которой изображаются взаимодействия между частями композитной структуры или ролями кооперации. В отличие от диаграммы последовательности, на диаграмме кооперации явно указываются отношения между объектами, а время как отдельное измерение не используется.

8. Диаграмма компонентов (Component Diagram) – статическая структурная диаграмма, которая показывает разбиение программной системы на структурные компоненты и связи между ними. В качестве физических компонентов могут выступать файлы, библиотеки, модули, исполняемые файлы, пакеты и т. п.

9. Диаграмма размещения (Deployment Diagram). Другое название – диаграмма развертывания. Показывает топологию системы и распределение компонентов системы по ее узлам, а также соединения – маршруты передачи информации между аппаратными узлами.

Component Diagram и Deployment Diagram относятся к подклассу диаграмм реализации, которые служат для

представления физических компонентов сложной системы и поэтому относятся к ее физической модели.

Deployment Diagram – единственная диаграмма, на которой применяются «трехмерные» обозначения: узлы системы обозначаются параллелепипедами. Все остальные обозначения в UML – плоские фигуры.

Диаграммы UML представляют собой комплекс модели, структура которого в нотации UML показана на рис. 2.2.

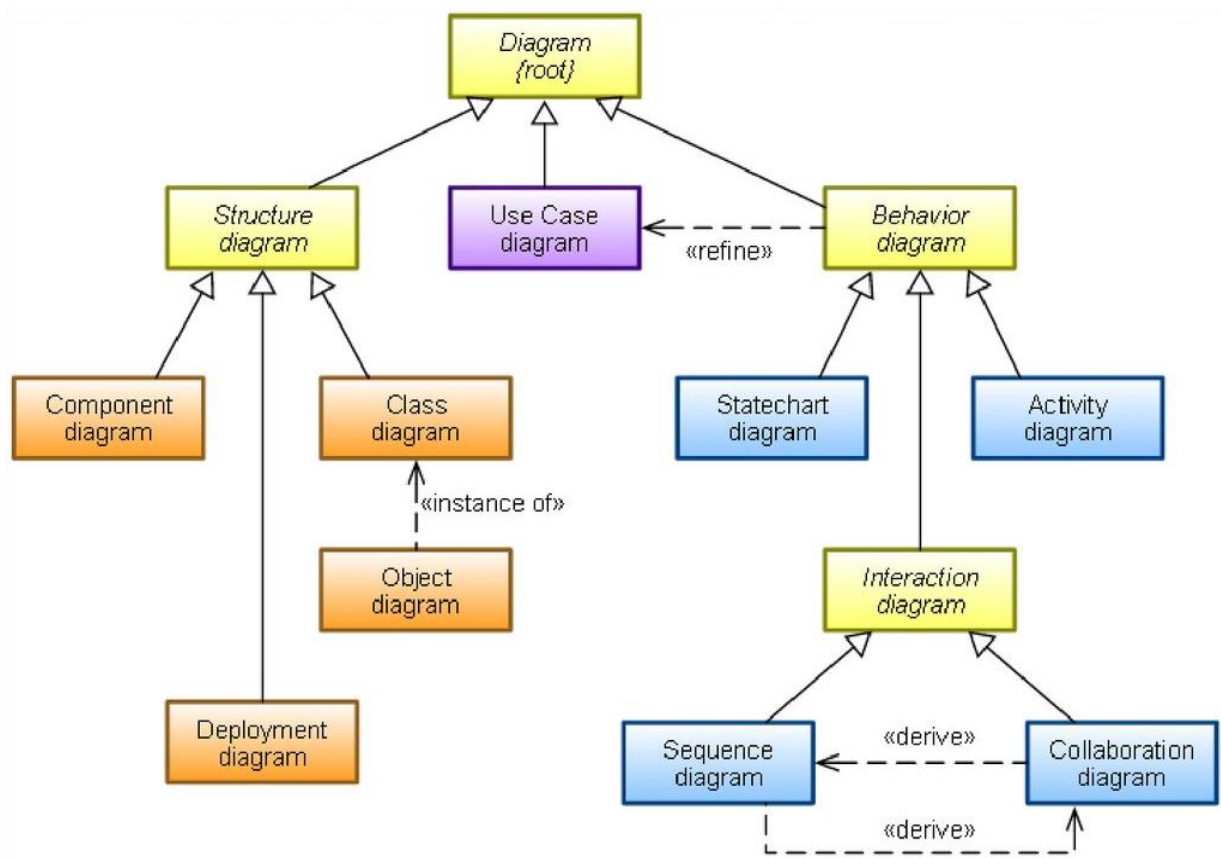


Рис. 2.2. Иерархия диаграмм для UML-1 согласно стандарту ISO/IEC 19501:2005

2.1.2. OMG UML Specification 2.0

Формальная спецификация версии UML 2.0 опубликована в августе 2005 г. Семантика языка была значительно уточнена и расширена. Последняя версия UML 2.5 опубликована в июне 2015 г. В качестве международного стандарта UML 2.4.1 был

принят стандарт ISO/IEC 19505-1, 19505-2. Основные документы: UML 2.0 Superstructure, UML 2.0 Infrastructure, UML 2.0 Diagram Interchange, UML Extensions, UML MOF Metamodel, UML CORBA, UML XMI DTD, OCL.

В комплекс канонических диаграмм были внесены значительные коррективы – их число увеличилось до 13. Также изменился список доступных конструкций языка, что расширило область его применения. Кроме этого, две диаграммы были переименованы: диаграмма кооперации – в диаграмму коммуникации, а диаграмма состояний – в диаграмму автомата.

В UML 2.0 различают общие и специальные диаграммы. Общие диаграммы практически не зависят от предмета моделирования и могут применяться в любом программном проекте. Специальные диаграммы служат для дополнения общей диаграммы, например, являются ее частным случаем или же просто играют вспомогательную роль, уточняя некоторые детали. Пример пары общей и специальной диаграммы: Class Diagram и Object Diagram (рис. 2.2).

Список новых структурных диаграмм UML 2.0.

1. Диаграмма составной структуры (Composite Structure Diagram). Представляет внешние интерфейсы и внутреннюю композицию структурированных классов.

2. Диаграмма пакетов (Package Diagram). Определяет размещение элементов модели в пакетах. Относится к классу специальных диаграмм UML.

Часть комплекса структурных диаграмм UML 2.0 показана на рис. 2.3.

Список новых поведенческих диаграмм UML 2.0.

1. Диаграмма коммуникации (Communication Diagram). Описывает сообщения посылаемые и принимаемые структурированными объектами.

2. Диаграмма автомата (State Machine Diagram). Отражает детальное описание поведения на основе явного выделения состояний и описания переходов между состояниями.

3. Временная диаграмма (Timing Diagram). Другое название – диаграмма синхронизации. Отражает взаимодействие

с сообщениями, развертываемое во времени. Относится к классу специальных диаграмм UML.

4. Диаграмма обзора взаимодействий (Interaction Overview Diagram). Сочетает некоторые аспекты диаграмм деятельности и последовательности.

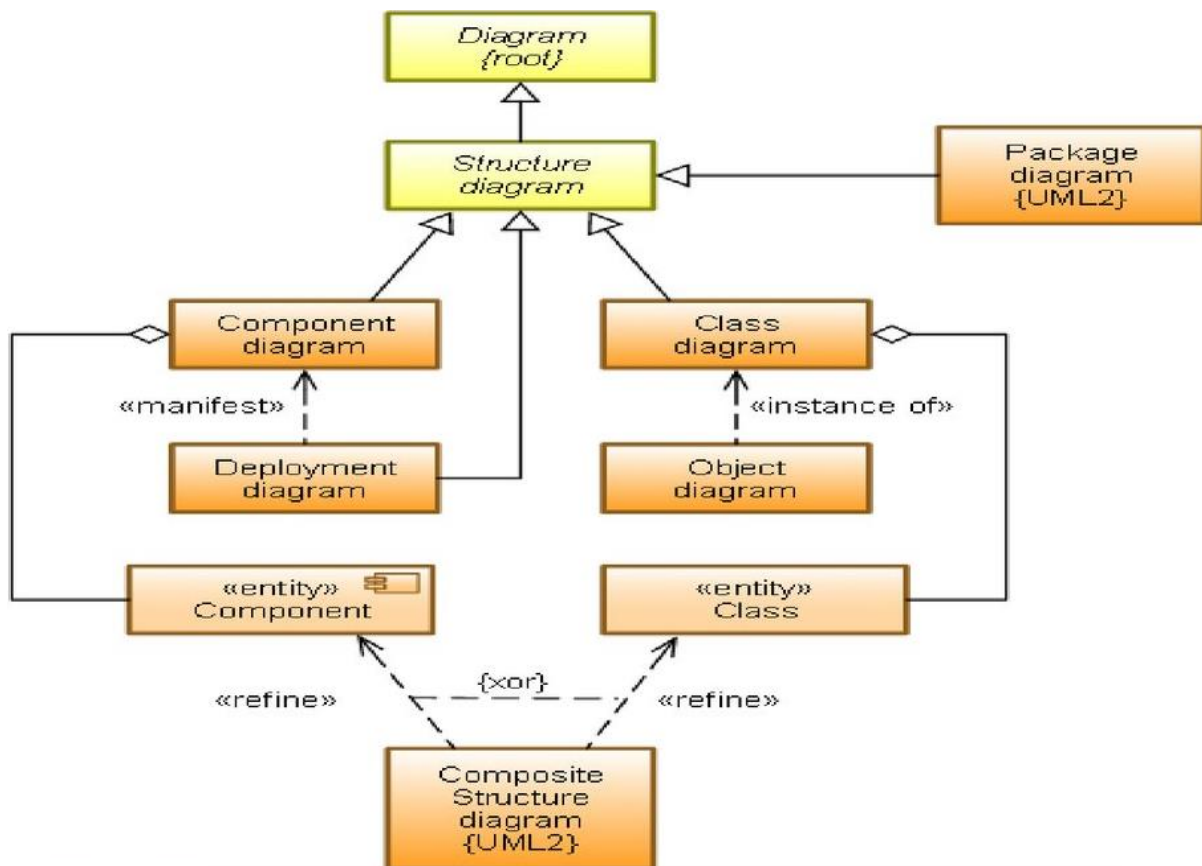


Рис. 2.3. Иерархия структурных диаграмм для UML-2 согласно стандарту ISO/IEC 19505

Часть комплекса поведенческих диаграмм UML 2.0 показана на рис. 2.4.

Перечень этих диаграмм и их названия являются каноническими в том смысле, что представляют собой неотъемлемую часть графической нотации языка UML. Более того, процесс ООП неразрывно связан с процессом построения этих диаграмм. Вместе с тем совокупность построенных диаграмм является самодостаточной в том смысле, что в них

содержится вся информация, которая необходима для реализации проекта сложной системы.

Каждая из этих диаграмм детализирует и конкретизирует различные представления о модели сложной системы в терминах языка UML. При этом диаграмма вариантов использования представляет собой наиболее общую концептуальную модель сложной системы, которая является исходной для построения всех остальных диаграмм.

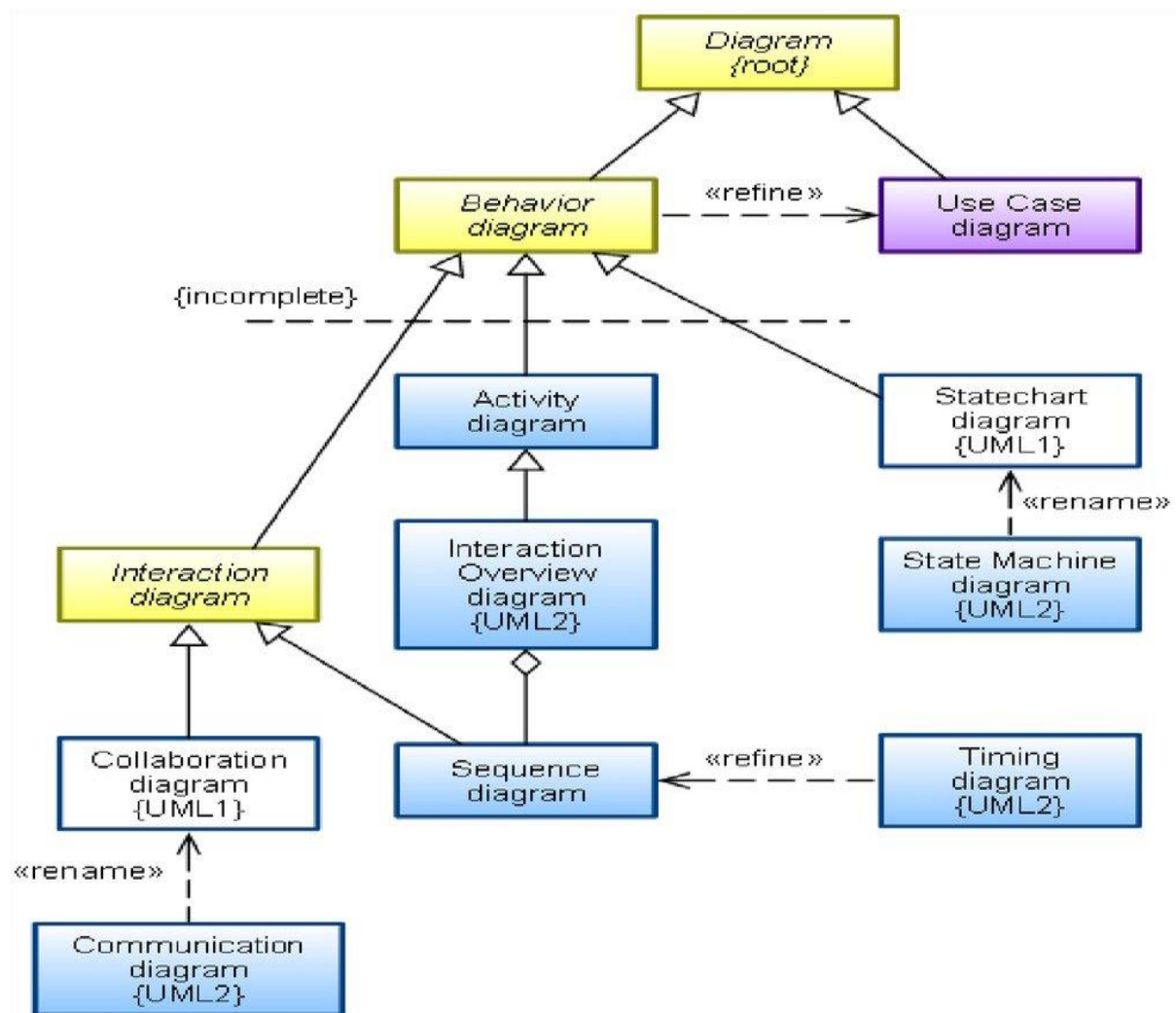


Рис. 2.4. Иерархия поведенческих диаграмм для UML-2 согласно стандарту ISO/IEC 19505

Диаграмма классов служит логической моделью, отражающей статические аспекты структурного построения сложной системы.

Диаграммы поведения также являются разновидностями логической модели, которые отражают динамические аспекты функционирования сложной системы.

2.1.3. Пакеты в UML

Все виды элементов UML можно группировать в пакеты, задавая некоторое пространство имен. Каждый элемент может принадлежать одному пакету. Пакет может вкладываться в другой пакет, образуя отношение «быть подмножеством». Ограничений на выбор группируемых элементов нет. Распространённые подходы к группированию:

- по стереотипу, например, классы-сущности, граничные классы;
- по функциональности, например «работа с клиентами».

Важно, что пакеты зависимы, если по крайней мере два элемента в этих пакетах зависимы. Т.е. изменения в одном элементе могут повлечь за собой необходимость изменения в другом.

Элементы пакета можно изображать в его поле (рис. 2.5, *а*) и вне его, указывая связи (рис. 2.5, *б*).

2.1.4. Особенности изображения диаграмм UML

Большинство диаграмм UML являются в своей основе графами специального вида, состоящими из вершин в форме геометрических фигур, которые связаны между собой ребрами или дугами. Поскольку информация, которую содержит в себе граф, имеет в основном топологический характер, ни геометрические размеры, ни расположение элементов диаграмм не имеют принципиального значения. Исключение составляют диаграмма последовательностей и диаграмма синхронизации с метрической осью времени.

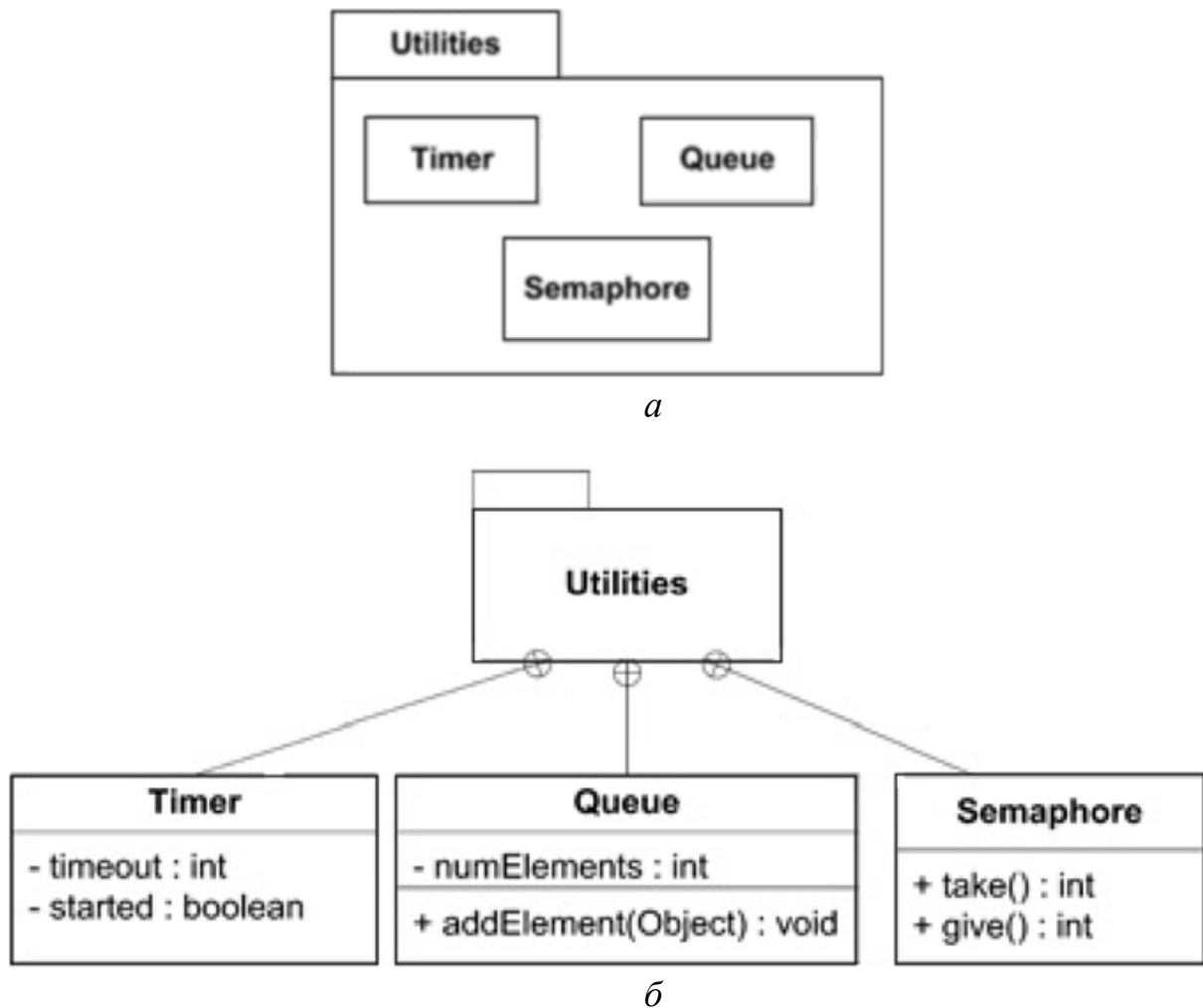


Рис. 2.5. Моделирование пакетов на UML: *a* – содержание пакета в его поле; *б* – содержание пакета вне его поля

Для диаграмм UML существуют три типа визуальных обозначений, которые важны с точки зрения заключенной в них информации.

1. Графические символы – как правило, элементы, несущие основную семантическую нагрузку.

2. Текст, который содержится внутри границ отдельных геометрических фигур на плоскости. При этом форма этих фигур (прямоугольник, эллипс) соответствует элементам языка UML (класс, вариант использования и т.д.) и имеет фиксированную семантику.

3. Связи, которые представляются различными линиями (со стрелками разных видов или без) на плоскости. Связи в языке

UML обобщают понятие дуг и ребер из теории графов, но имеют менее формальный характер.

Графические конструкции UML могут иметь различную высоту и ширину с целью размещения внутри этих фигур необходимого текста и других конструкций UML. Наиболее часто внутри таких символов помещаются строки текста, которые уточняют семантику или фиксируют отдельные свойства соответствующих элементов.

Информация, содержащаяся внутри фигур, имеет важное значение для конкретной модели проектируемой системы, поскольку регламентирует реализацию соответствующих элементов в программном коде системы. Предполагается, что каждое использование строки текста должно соответствовать синтаксису в нотации языка UML.

Связи на диаграммах UML соединяют отдельные графические символы, выражая таким образом системные свойства соответствующих моделей. По стандарту UML концевые точки отрезков линий должны обязательно соприкасаться с геометрическими фигурами, служащими для обозначения вершин диаграмм, как принято в теории графов. С концептуальной точки зрения путям в языке UML придается особое значение, поскольку они являются простыми топологическими сущностями. Кроме того, отдельные части пути или сегменты могут не существовать сами по себе вне содержащего их пути. Пути всегда соприкасаются с другими графическими символами или значками на обеих границах соответствующих отрезков линий.

2.1.5. Правила графического изображения диаграмм

При графическом изображении диаграмм следует придерживаться основных рекомендаций и правил.

1. Каждая диаграмма должна служить законченным представлением соответствующего фрагмента моделируемой предметной области. В процессе разработки диаграммы

необходимо учесть все сущности, важные с точки зрения контекста данной модели и диаграммы.

2. Все сущности на диаграмме модели должны быть одного концептуального уровня. Здесь имеется в виду не только согласованность имен одинаковых элементов, но и возможность вложения отдельных диаграмм друг в друга для достижения полноты представлений.

3. В случае достаточно сложных моделей с множеством вложений диаграмм желательно придерживаться стратегии последовательного уточнения или детализации отдельных диаграмм.

4. Вся информация о сущностях должна быть представлена на диаграммах.

5. Диаграммы не должны содержать противоречивой информации. Противоречивость модели может приводить к ошибкам в программном коде. К примеру, наличие элементов с одинаковыми именами и различными атрибутами свойств в одном пространстве имен приводит к неоднозначной интерпретации и может служить источником проблем при кодировании.

6. Диаграммы не следует перегружать текстовой информацией. Визуализация модели является наиболее эффективной, если она содержит минимум пояснительного текста. Важно уметь правильно отображать те или иные сущности и аспекты моделирования в соответствующие элементы канонических диаграмм.

7. Каждая диаграмма должна быть самодостаточной для правильной интерпретации всех ее элементов и понимания семантики всех используемых графических символов. Модель системы на языке UML представляет собой пакет иерархически вложенных диаграмм, детализация которых должна быть достаточной для последующей генерации программного кода проектируемой системы.

8. Количество типов диаграмм для конкретной модели системы не является строго фиксированным. Речь идет о том, что для простых приложений нет необходимости строить все без

исключения типы диаграмм. Некоторые из них могут просто отсутствовать в проекте системы, и этот факт не будет считаться ошибкой разработчика. Например, модель системы может не содержать диаграмму развертывания для приложения, выполняемого локально на компьютере пользователя. Важно понимать, что перечень диаграмм зависит от специфики конкретного проекта системы.

2.1.6. Диаграммы вариантов использования

Одной из первых диаграмм в модели ПС строится диаграмма вариантов использования. Это исходная концептуальная модель системы в процессе ее проектирования.

Диаграмма прецедентов (вариантов использования, Use Case) – это базовая диаграмма UML, на которой изображаются отношения между действующими лицами и действиями (прецедентами).

Назначение Use Case диаграмм.

1. Определить функциональность проектируемой системы в виде обозримой диаграммы или набора таких диаграмм.

2. Определить контекст / контексты моделируемой предметной области.

3. Специфицировать поведение проектируемой ПС в виде вариантов использования, возможно связанных между собой.

4. Разработать концептуальную модель системы, выделив наиболее общие понятия и предполагая её последующую детализацию описаниями потоков данных (в технологии DDD), логическими, физическими моделями и моделями структур данных.

5. Подготовить документацию для взаимодействия разработчиков системы с заказчиками и пользователями.

Система (процесс) представляется в форме вариантов её (его) использования, с которыми взаимодействуют внешние сущности или акторы (в английской транскрипции – экторы, акторы). При этом актором называется любой объект, субъект, система или устройство, взаимодействующие с моделируемой

системой извне. Концептуальный характер этой диаграммы проявляется в том, что подробная детализация диаграммы на начальном этапе проектирования имеет отрицательный характер, поскольку предопределяет способы реализации поведения системы – эти аспекты должны быть сознательно скрыты от разработчика на модели прецедентов.

Вариант использования (Use Case) – внешняя спецификация последовательности действий, которые система или другая сущность могут выполнять в процессе взаимодействия с акторами.

Отдельный вариант использования обозначается на диаграмме эллипсом, внутри которого содержится его краткое имя в форме глагольного существительного (рис. 2.6, *а*) или глагола (рис. 2.6, *б*) с пояснительным текстом. Сам текст имени прецедента должен начинаться с заглавной буквы и размещаться внутри эллипса или под ним.

Диаграмма вариантов использования содержит конечное множество вариантов, которые в целом должны определять все возможные аспекты ожидаемого поведения системы.

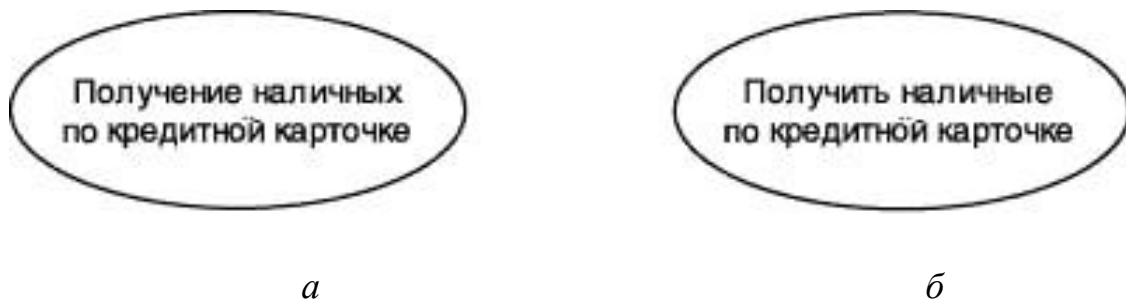


Рис. 2.6. Графическое обозначение варианта использования:
а – глагольное существительное; *б* – глагол

Варианты использования могут включать описания:

- основных вариантов реализации сервиса;
- альтернативных вариантов;
- исключительных ситуаций, таких как корректная обработка ошибок системы.

Примерами вариантов использования могут быть следующие действия:

- занесение в систему данных о клиенте;
- проверка состояния текущего счета клиента;
- оформление заказа на покупку товара;
- получение информации о кредитоспособности клиента;
- отображение графической формы на экране монитора;
- авторизация пользователя;
- и другие действия, процессы, задачи.

Акторы находятся вне проектируемой системы.

Актор (actor) – внешняя спецификация, выражающая определенную роль относительно конкретного варианта использования.

Стандартным графическим обозначением актора на диаграммах является фигурка человечка, под которой записывается имя актора (рис. 2.7). Имя актора должно быть достаточно информативным с точки зрения семантики. Это может быть:

- а) наименование должности (продавец, кассир, менеджер и т.д.);
- б) технические устройства (датчики, терминалы обслуживания и т.д.);
- в) другие системы.



Рис. 2.7. Графическое обозначение действующего лица:

а – роль человека; *б* – техническое устройство; *в* – техническая система (подсистема)

Не рекомендуется давать актерам имена собственные или названия моделей конкретных устройств, даже если это с очевидностью следует из контекста проекта, так как одно и то же лицо (устройство) может выступать в нескольких ролях и обращаться к различным сервисам проектируемой системы.

Между элементами модели прецедентов могут существовать различные отношения, которые описывают взаимодействие экземпляров одних акторов и вариантов использования с экземплярами других акторов и вариантов использования.

Отношение (relationship) – семантическая связь между отдельными элементами модели.

Один актер может взаимодействовать с несколькими вариантами использования. Это значит, что этот актер обращается к нескольким сервисам ПС. В свою очередь один прецедент может взаимодействовать с несколькими актерами, предоставляя свой сервис для них. В то же время два варианта использования, определенные в рамках одной системы, могут взаимодействовать друг с другом. В обоих случаях способы взаимодействия элементов модели предполагают обмен сигналами или сообщениями, которые инициируют реализацию функционального поведения моделируемой системы. В языке UML имеется несколько стандартных видов отношений между элементами модели прецедентов: ассоциация, включение, расширение и обобщение. Эти отношения показаны на рис. 2.8.

Отношение **ассоциации** – одно из фундаментальных понятий в языке UML и используется при построении всех канонических диаграмм. На диаграмме требований устанавливается только между актором и прецедентом и служит для обозначения специфической роли актора при взаимодействии с прецедентом. На диаграмме требований, так же как и на других диаграммах, ассоциация обозначается сплошной линией между актором и прецедентом, которая может иметь дополнительные обозначения, например имя и кратность (рис. 2.8, а).

Включение (include) в языке UML – это разновидность отношения зависимости между базовым вариантом использования и его специальным случаем (рис. 2.8, б). При этом

отношением зависимости (dependency) является такое отношение между двумя элементами модели, при котором изменение одного элемента (независимого) неизбежно приводит к изменению другого элемента (зависимого). Процесс выполнения базового варианта использования включает в качестве собственного подмножества последовательность действий, которая определена для включаемого варианта использования. Причем выполнение включаемой последовательности действий происходит всегда при инициировании базового прецедента.

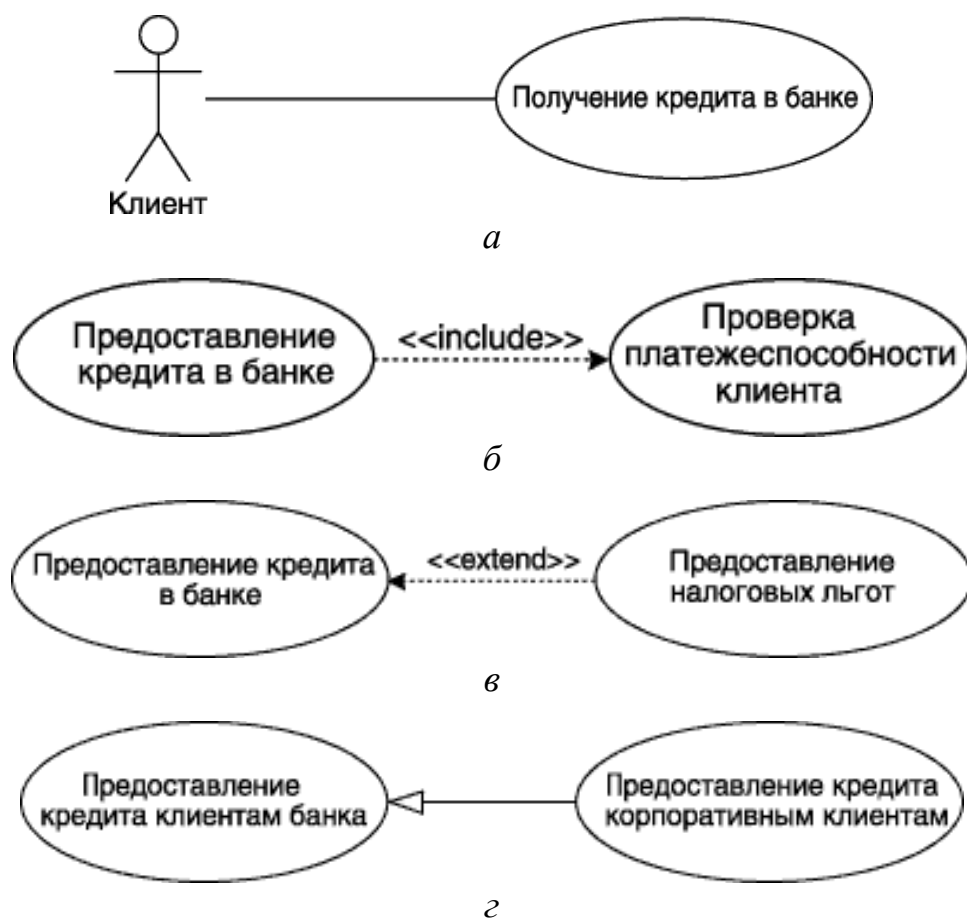


Рис. 2.8. Пример графического представления отношений на диаграмме прецедентов: *а* – отношение ассоциации; *б* – отношение включения; *в* – отношение расширения; *г* – отношение обобщения

Расширение (extend) определяет взаимосвязь базового прецедента с другим прецедентом, функциональное поведение которого задействуется базовым не всегда, а только при выполнении дополнительных условий. Расширение является

зависимостью, направленной к базовому прецеденту и соединенной с ним в так называемой точке расширения (рис. 2.8, в). В отличие от отношения включения, данное отношение всегда предполагает проверку условия и ссылку на точку расширения в базовом прецеденте. Один вариант использования может быть расширением для нескольких базовых прецедентов, а также иметь в качестве собственных расширений другие прецеденты. Любой базовый вариант использования не зависит от своих расширений.

Отношения включения и расширения обозначаются посредством стереотипов.

Стереотип – это механизм обобщения, позволяющий расширять словарь UML.

Стереотип изображается как имя, заключенное в кавычки «» или знаки << >> и обычно расположенное над именем элемента. Примеры стереотипов:

- *extend* – целевой вариант использования расширяет поведение исходного (базового) в данной точке расширения;
- *include* – исходный (базовый) прецедент явно включает поведение другого прецедента в точке, определяемой исходным.

Обобщение представляет собой реализацию принципа наследования в ООП. Помимо вариантов использования, может также связывать два и более актора, которые могут иметь общие свойства, т.е. взаимодействовать с одним и тем же множеством вариантов использования одинаковым образом. Обобщение может быть установлено с абстрактным прецедентом (актером), который моделирует соответствующую общность действий (ролей). Прецедент-потомок является специализацией прецедента-предка. Следует отметить, что потомок наследует все свойства поведения своего родителя, а также может обладать дополнительными особенностями поведения.

Графически отношение обобщения обозначается сплошной линией со стрелкой в форме треугольника без заливки, которая указывает на родительский прецедент или действующее лицо (рис. 2.8, г). Эта стрелка-обобщение используется также в других канонических диаграммах UML.

Построение диаграмм вариантов использования.

Хорошим источником для идентификации вариантов использования служат внешние события. Следует начать с перечисления всех событий, происходящих во внешнем мире, на которые система должна каким-то образом реагировать. Какое-либо конкретное событие может повлечь за собой реакцию системы, не требующую вмешательства пользователей, или, наоборот, вызвать пользовательскую реакцию. Идентификация событий, на которые необходимо реагировать, помогает идентифицировать варианты использования.

Рекомендации по разработке диаграмм вариантов использования.

Главное назначение диаграммы вариантов использования заключается в формализации функциональных требований к системе с помощью понятий соответствующего пакета и возможности согласования полученной модели с заказчиком на ранней стадии проектирования. Любой из вариантов использования может быть подвергнут дальнейшей декомпозиции на множество подвариантов использования отдельных элементов, которые образуют исходную сущность. Рекомендуемое общее количество актеров в модели – не более 20, а вариантов использования – не более 50. В противном случае модель теряет свою наглядность и, возможно, заменяет собой одну из некоторых других диаграмм.

С системно-аналитической точки зрения построение диаграммы вариантов использования не только специфицирует функциональные требования к проектируемой системе, но и выполняет исходную структуризацию предметной области.

Пример диаграммы вариантов использования в нотации UML для ПС выполнения операции обмена валюты показан на рис. 2.9. На ней даны все виды отношений между прецедентами: включение, расширение и обобщение. Также показаны отношения обобщения между актерами, где работники банка обобщаются родительским актором «Пользователь», реализующим вход в систему.

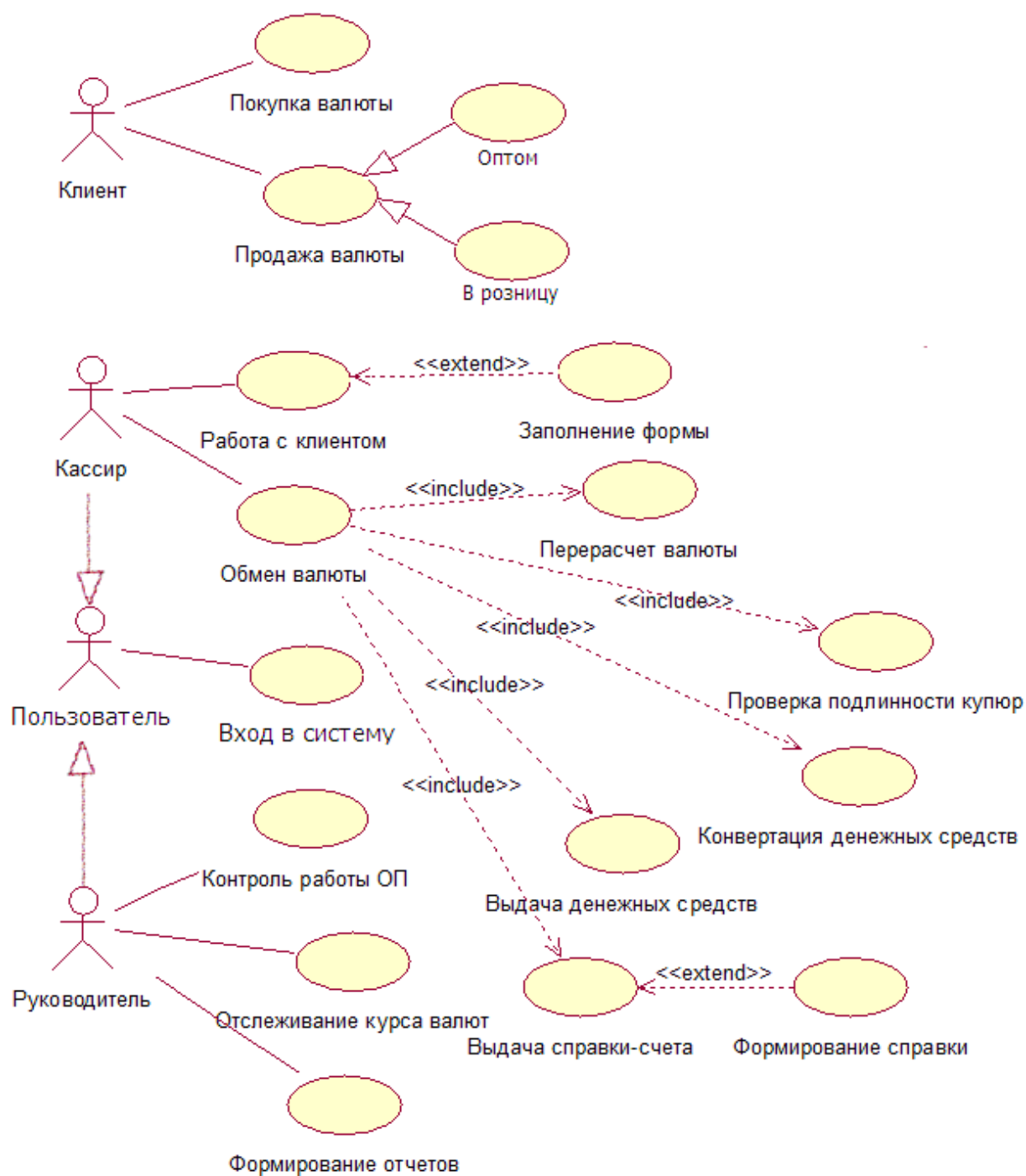


Рис. 2.9. Модель вариантов использования для ПС выполнения операции обмена валюты

2.1.7. Объектно-ориентированный анализ прецедентов

Объектно-ориентированный анализ – это методика, при которой требования к системе воспринимаются с точки зрения классов и объектов, выявленных в предметной области.

Варианты использования (прецеденты) моделируемой системы описывают, что должна будет делать система, на концептуальном уровне – обобщенно. Чтобы фактически разработать систему, потребуются более конкретные детали. Эти детали описываются в документе, называемом «поток событий» (flow of events). Целью потока событий является документирование процесса обработки данных, реализуемого в рамках варианта использования. Этот документ подробно описывает, что будут делать пользователи системы и что – сама система.

Хотя поток событий и описывается подробно, он также не должен зависеть от реализации. Цель – описать, что будет делать система, а не как она будет это делать. Обычно поток событий включает:

- краткое описание;
- предусловия (pre-conditions);
- основной поток событий;
- альтернативный поток событий (или несколько альтернативных потоков);
- постусловия (post-conditions).

Последовательно рассмотрим эти составные части.

Описание. Каждый вариант использования должен иметь связанное с ним короткое описание того, что он будет делать (например: вариант использования «Перевести деньги» позволяет клиенту или служащему банка переводить деньги с одного счета до востребования или сберегательного счета на другой).

Предусловия. Предусловия варианта использования – это такие условия, которые должны быть выполнены, прежде чем вариант использования начнет выполняться сам. Например, таким условием может быть выполнение другого варианта использования или наличие у пользователя прав доступа, требуемых для запуска этого. Не у всех вариантов использования бывают предварительные условия.

Основной и альтернативный потоки событий. Конкретные детали вариантов использования описываются в основном и альтернативных потоках событий. Поток событий

поэтапно описывает, что должно происходить во время выполнения заложенной в варианты использования функциональности. Поток событий уделяет внимание тому, что будет делать система, а не как она будет делать это, причем описывает все это с точки зрения пользователя. Основной и альтернативный потоки событий включают следующее описание:

- способ запуска варианта использования;
- различные пути выполнения варианта использования;
- нормальный, или основной, поток событий варианта использования;
- отклонения от основного потока событий (так называемые альтернативные потоки);
- потоки ошибок;
- способ завершения варианта использования.

Анализ потока событий рассмотрим на примере варианта использования «Снять деньги» системы управления банкоматом.

Основной поток.

1. Вариант использования начинается, когда клиент вставляет свою карточку в банкомат.
2. Банкомат выводит приветствие и предлагает клиенту ввести персональный идентификационный номер.
3. Клиент вводит номер.
4. Банкомат подтверждает введенный номер. Если номер не подтвержден, выполняется альтернативный поток событий А1.
5. Банкомат выводит список доступных действий:
 - положить деньги на счет;
 - снять деньги со счета;
 - перевести деньги.
6. Клиент выбирает пункт «Снять деньги».
7. Банкомат запрашивает, сколько денег надо снять.
8. Клиент вводит требуемую сумму.
9. Банкомат определяет, имеется ли на счету достаточно денег. Если денег недостаточно, выполняется альтернативный поток А2. Если во время подтверждения суммы возникают ошибки, выполняется поток ошибок Е1.

10. Банкомат вычитает требуемую сумму из счета клиента.
11. Банкомат выдает клиенту требуемую сумму наличными.
12. Банкомат возвращает клиенту его карточку.
13. Банкомат печатает чек для клиента.
14. Вариант использования завершается.

Альтернативный поток A1. Ввод неправильного идентификационного номера.

1. Банкомат информирует клиента, что идентификационный номер введен неправильно.
2. Банкомат возвращает клиенту его карточку.
3. Вариант использования завершается.

Альтернативный вариант использования A2. Недостаточно денег на счету.

1. Банкомат информирует клиента, что денег на его счету недостаточно.
2. Банкомат возвращает клиенту его карточку.
3. Вариант использования завершается.

Поток ошибок E1. Ошибка в подтверждении запрашиваемой суммы.

- 1) Банкомат сообщает пользователю, что при подтверждении запрашиваемой суммы произошла ошибка и дает ему номер телефона службы поддержки клиентов банка.
- 2) Банкомат заносит сведения об ошибке в журнал ошибок. Каждая запись содержит дату и время ошибки, имя клиента, номер его счета и код ошибки.
- 3) Банкомат возвращает клиенту его карточку.
- 4) Вариант использования завершается.

Постусловия. Постусловиями называются такие условия, которые всегда должны быть выполнены после завершения варианта использования. Например, в конце варианта использования можно пометить флажком какой-нибудь переключатель. Информация такого типа входит в состав постусловий. Как и для предусловий, с помощью постусловий можно вводить информацию о порядке выполнения вариантов использования системы. Если, например, после одного из

вариантов использования должен всегда выполняться другой, это можно описать как постусловие. Такие условия имеются не у каждого варианта использования.

2.1.8. Диаграммы классов

Одной из первых диаграмм в модели ПС строится диаграмма классов. Некоторые авторы считают, что эта диаграмма первична, поскольку представляет первичную концептуальную структуру ПС. Однако это утверждение спорное, поскольку диаграмма классов отражает также функционал ПС набором методов, инкапсулированных в классы.

Диаграмма классов (class diagram) – диаграмма языка UML, на которой представлена совокупность декларативных или статических элементов модели, таких как классы с атрибутами и операциями, а также связывающие их отношения.

Диаграмма классов предназначена для представления статической структуры модели системы в терминологии классов объектно-ориентированного программирования. Когда говорят о данной диаграмме, имеют в виду статическую структурную модель проектируемой системы, т.е. графическое представление таких структурных взаимосвязей логической модели системы, которые не зависят от времени.

Класс (class) – абстрактное описание множества однородных объектов, имеющих одинаковые атрибуты, операции и отношения с объектами других классов.

Для идентификации классов и объектов используют текстовые описания потоков событий проектируемой системы. Из этих описаний выделяют кандидатов в абстракции. Для этого используют синтаксический разбор предложений из текстов потоков событий и в качестве кандидатов в абстракции принимают подлежащие. Абстракции, которые обладают выраженным поведением, принимают в качестве классов. Остальные становятся атрибутами классов.

Все абстракции могут быть разделены на 3 типа:

- абстракция сущности – объект представляет собой полезную модель некой сущности в предметной области;
- абстракция поведения – объект состоит из обобщенного множества операций;
- абстракция-интерфейс – объект группирует операции, которые вместе используются более высоким уровнем управления как интерфейс системы.

Также абстракции могут быть разделены на виды в соответствии с классификатором, представленном в табл. 2.1. Результатом анализа являются характеристики абстракций, которые сводятся в табл. 2.2 и 2.3.

Таблица 2.1

Классификатор видов абстракций

Вид	Описание
Роли	Роли, которые исполняют пользователи, работающие с системой
Предметы	Осязаемый материальный объект или группа объектов
Места, локации	Области, связанные с людьми или предметами, существенные для работы системы
Инструменты	Средства деятельности, используемые определенными ролями
Организации	Формально организованная совокупность людей, ресурсов, оборудования, инструментов, которая имеет определенную цель и существование которой в целом не зависит от людей
Концепции	Принципы и идеи, сами по себе неосязаемые, но предназначенные для организации деятельности и/или общения, или же для наблюдения за ними
События	Нечто случающееся с чем-то / кем-то в заданное время или последовательно. Происшествия, которые должны быть зафиксированы
Показатели	Характеристики, определяющие деятельность, либо ее отдельные процессуальные компоненты
Устройства	Устройства, с которыми взаимодействует система
Другие системы	Внешние системы, с которыми взаимодействует проектируемая система

Таблица 2.2

Абстракции подсистемы

№	Абстракция	Тип	Вид	Описание
---	------------	-----	-----	----------

Таблица 2.3

Абстракции подсистемы и их поведение

№	Абстракция	Требование	Описание поведения
---	------------	------------	--------------------

Графически класс в нотации UML изображается в виде прямоугольника, который может быть разделен горизонтальными линиями на разделы или секции (рис. 2.10). В этих секциях указываются имя класса, атрибуты и операции класса.

На начальных этапах разработки диаграммы отдельные классы могут обозначаться простым прямоугольником, в котором должно быть указано его имя (рис. 2.10, а). По мере проработки отдельных компонентов диаграммы описания классов дополняются атрибутами (рис. 2.10, б) и операциями (рис. 2.10, в). Четвертая секция (рис. 2.10, г) не обязательна и служит для размещения дополнительной информации справочного характера, например, об исключениях или ограничениях класса, сведения о разработчике или языке реализации класса. Предполагается, что окончательный вариант диаграммы содержит наиболее полные описания классов, которые состоят из трех или четырех секций.

Имя класса должно быть уникальным в пределах пакета, который может содержать одну или несколько диаграмм классов (подробнее о пакетах см. п. 2.1.3). Имя указывается в самой верхней секции прямоугольника, поэтому она называется секцией имени класса. В дополнение к общему правилу именования элементов языка UML, имя класса записывается по центру секции полужирным шрифтом и должно начинаться с заглавной буквы. Рекомендуются в качестве имен классов использовать имена существительные, записанные по практическим соображениям без пробелов. Необходимо помнить, что имена классов образуют словарь предметной области при проектировании системы.



Рис. 2.10. Варианты графического изображения класса на диаграмме:
а – класс с одной секцией имени; *б* – класс с двумя секциями (имени и атрибутов); *в* – класс с тремя секциями (имени, атрибутов и операций);
г – полное обозначение класса

В секции имени класса могут находиться стереотипы или ссылки на стандартные шаблоны, от которых образован данный класс и от которых он наследует свои свойства: атрибуты и операции. В этой секции может также приводиться информация о разработчике класса и статус состояния разработки.

Атрибут (attribute) – содержательная характеристика класса, описывающая множество значений, которые могут принимать отдельные объекты этого класса, и определяющая состояние его экземпляра.

Атрибут класса служит для представления отдельного свойства или признака, который является общим для всех объектов данного класса. Атрибуты класса записываются в секции атрибутов (рис. 2.10, *б*).

Операция (operation) – это сервис, предоставляемый каждым экземпляром или объектом класса по требованию своих клиентов, в качестве которых могут выступать другие объекты, в том числе и экземпляры данного класса, и определяющий поведение его экземпляра.

Операции класса записываются в секции операций (рис. 2.10, *в*). Совокупность операций характеризует

функциональный аспект поведения всех объектов данного класса, который описывается другими каноническими диаграммами UML. Имя операции представляет собой строку текста, которая используется в качестве идентификатора соответствующей операции и должна быть уникальной в пределах данного класса. Имя операции должно начинаться со строчной буквы и, как правило, записываться без пробелов.

Идентификация классов анализа заключается в предварительном определении набора классов системы на основе описания предметной области. Прежде всего, выделяются основные абстракции предметной области. Способы идентификации основных абстракций аналогичны способам идентификации сущностей в модели «сущность—связь» – поиск абстракций, описывающих физические или материальные объекты, процессы и события, роли людей, организации и другие понятия предметной области.

Единственным формальным способом идентификации сущностей является анализ текстовых описаний предметной области, выделение из описаний имен существительных и выбор их в качестве кандидатов на роль абстракций. Каждая сущность должна иметь наименование, выраженное существительным в единственном числе. Далее абстракции идентифицируются по трем типам, которые используются для уточнения семантики отдельных классов.

1. Управляющий класс (control class) – класс, отвечающий за координацию действий других классов на диаграмме. На каждой диаграмме классов должен быть хотя бы один управляющий класс, который контролирует последовательность выполнения действий данного варианта использования. Данный класс является активным и инициирует рассылку множества сообщений другим классам модели.

Управляющий класс изображается со стереотипом <<control>> в секции имени. Примеры управляющих классов: менеджер транзакций, журнал операций, координатор ресурсов, обработчик событий, обработчик ошибок и т.п.

2. Класс-сущность (entity class) – пассивный класс, информация о котором должна храниться постоянно и не уничтожаться с выключением системы или завершением работы программы.

Как правило, этот класс соответствует отдельной сущности логической модели данных системы. В этом случае его атрибуты являются потенциальными полями таблицы, а операции – присоединенными или хранимыми процедурами. Этот класс пассивный и лишь принимает сообщения от других классов модели, реагируя на них посредством своих операций.

Источники выявления классов-сущностей:

- ключевые абстракции, созданные в процессе архитектурного анализа;
- глоссарий проекта;
- описание потоков событий вариантов использования.

Класс-сущность изображается со стереотипом <<entity>> в секции имени класса.

3. Граничный класс (boundary class) – класс, который располагается на границе системы с внешней средой и непосредственно взаимодействует с акторами, но тем не менее является составной частью системы.

Как правило, для каждой пары «действующее лицо – вариант использования» определяется один граничный класс.

Граничный класс изображается со стереотипом <<boundary>> в секции имени. Типы граничных классов:

- пользовательский интерфейс, выражающий обмен информацией с пользователем без деталей интерфейса: кнопок, списков, окон и т.п.;
- системный интерфейс с надсистемой или другой системой (используемые протоколы обмена без деталей их реализации);
- аппаратный интерфейс для связи с внешними техническими устройствами (аппаратные средства, протоколы, драйверы).

Все классы вступают друг с другом в отношения. На диаграмме классов выделяют такие виды отношений: ассоциация,

наследование, агрегирование и использование. При изображении конкретной связи ей можно сопоставить текстовую метку, документирующую имя этой связи и/или ее роль. Имя отношения должно быть уникально в своем контексте. Некоторые отношения могут иметь по ее концам обозначения кратности (множественности), некоторые из которых показаны на рис. 2.11.

*	любое число	—————	ассоциация
1..*	один или много	—————>	наследование (обобщение)
0..1	ноль или 1	◊—————	агрегирование (включение)
1..8	от 1 до 8	◆—————	композиция (состав)
12	точно 12		
{2,7,11}	фикс. множество	←—————	использование (зависимость)

Рис. 2.11. Обозначения множественности и значки отношений между классами

Ассоциация является универсальным типом отношения между классами и в принципе может заменить любое другое. Значок ассоциации соединяет два класса и означает наличие семантической связи между ними. Для уточнения семантики ассоциации часто отмечаются существительными (например, «выбор», «содержит», «уточняет», «инициирует» и т.д.), описывающими природу связи. Одна пара классов может иметь одну или несколько ассоциативных связей.

Наследование представляет отношение типа «общее – частное», выглядит как значок ассоциации с треугольной стрелкой, которая указывает от подкласса к суперклассу. При этом подкласс наследует структуру и поведение своего суперкласса. Класс может иметь один (одиночное наследование) или несколько (множественное наследование) суперклассов.

Агрегирование выражает отношение «целое – часть» и получается из значка ассоциации добавлением незакрашенного кружка (ромбика) на конце, обозначающего агрегат. Экземпляры класса на другом конце стрелки будут в определенном смысле частями экземпляров класса-агрегата. Разрешается рефлексивная и циклическая агрегация. Агрегация не требует обязательного физического включения части в целое.

Композиция – это разновидность агрегирования с четко выраженным отношением владения, причем время жизни частей и целого совпадают. Части с нефиксированной кратностью могут быть созданы уже после создания самого композита, живут и уничтожаются вместе с ним. Также части могут быть удалены явным образом еще до уничтожения композита. Кроме того, в композитном агрегировании целое отвечает за диспозицию своих частей, т.е. композит должен управлять их созданием и уничтожением. Кружок (ромбик) у значка такого отношения закрашен.

Использование представляет отношение типа «клиент – сервер» и изображается как ассоциация с открытой стрелкой на конце, обозначающей класс-клиент. Эта связь выражает зависимость клиента от сервера и означает, что клиент нуждается в услугах сервера. Т.е. операции класса-клиента вызывают операции класса-сервера или имеют сигнатуру, в которой возвращаемое значение и /или аргументы принадлежат классу сервера. Фактически отношение использования показывает, что один элемент использует другой.

Общие рекомендации по построению диаграмм классов:

- использовать зависимость только в случае, если моделируемое отношение не является структурным;
- использовать обобщение, только если имеет место отношение типа «является»;
- располагать элементы так, чтобы свести к минимуму число пересекающихся линий отношений;
- пространственно организовывать элементы так, чтобы семантически близкие сущности располагались рядом;

- при необходимости и в случае невозможности соблюдения предыдущей рекомендации допускается дублирование класса в другом месте диаграммы;

- чтобы привлечь внимание к важным особенностям диаграммы, использовать примечания (комментарии) и выделение цветом.

Диаграммы классов, как и диаграммы прецедентов, дают представление о моделируемой системе в статике. Доскональное описание динамики моделируемой системы представляется диаграммами поведения UML. Для моделирования поведения на логическом уровне в языке UML могут использоваться сразу несколько канонических диаграмм: состояний, деятельности, последовательности и кооперации, каждая из которых фиксирует внимание на отдельном аспекте функционирования системы.

2.1.9. Диаграммы состояний

Диаграмма состояний относится к классу диаграмм поведения (behaviour diagrams). Диаграмма описывает процесс изменения состояний одного экземпляра определенного класса или (и) моделирует все возможные изменения в состоянии конкретного объекта ПС. Эти изменения определяют все возможные состояния, в которых может находиться объект, а также процесс смены состояний объекта в результате некоторых событий, идентификация которых предшествует построению диаграмм поведения (см. п. 2.1.7).

Эти диаграммы обычно используются для описания поведения одного объекта в нескольких прецедентах. Вместе с тем изменение состояния объекта может быть вызвано внешними воздействиями со стороны других объектов или актором извне. Именно для описания реакции объекта на подобные внешние воздействия и используются диаграммы состояний.

Основными понятиями, входящими в формализм автомата диаграммы состояний, являются состояние и переход. Главное различие между ними заключается в том, что длительность нахождения системы в отдельном состоянии существенно

превышает время, которое затрачивается на переход из одного состояния в другое. Предполагается, если не оговорено другое, что в пределе логическое время перехода из одного состояния в другое равно нулю. Другими словами, логически переход объекта из состояния в состояние происходит мгновенно.

Состояние в языке UML – абстрактный метакласс, используемый для моделирования отдельной ситуации, в течение которой имеет место выполнение некоторого условия.

Состояние может быть задано в виде набора конкретных значений атрибутов объекта, при этом изменение их отдельных значений будет отражать изменение состояния моделируемого класса или объекта. Состояния на диаграмме представляются прямоугольниками со скругленными углами (рис. 2.12).

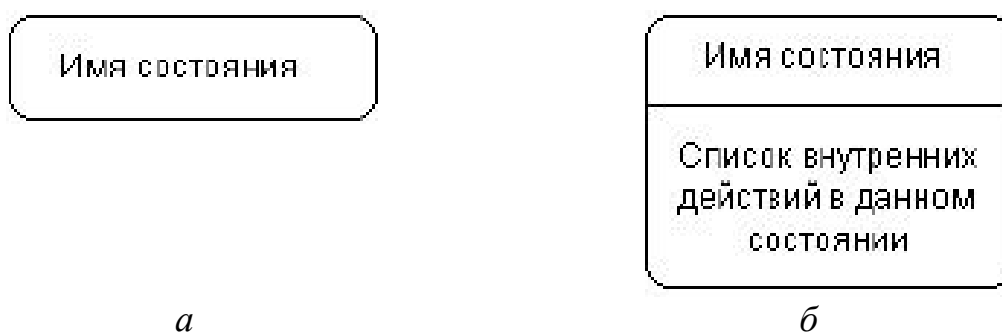


Рис. 2.12. Графическое изображение состояний на диаграмме состояний:
а – простое состояние; б – состояние с внутренними действиями

Действие в языке UML – некоторая атомарная операция, выполнение которой приводит к изменению состояния или возврату некоторого значения (например, «истина» или «ложь»).

Имя состояния представляет собой строку текста, которая раскрывает содержательный смысл данного состояния. Имя всегда записывается с заглавной буквы. Поскольку состояние системы является составной частью процесса ее функционирования, рекомендуется в качестве имени использовать глаголы в настоящем времени (запускает, печатает, ожидает) или соответствующие причастия (занят, свободен, передано, получено).

Имя у состояния может отсутствовать, т.е. оно необязательно для некоторых состояний. В этом случае состояние является анонимным, и если на диаграмме состояний их несколько, то все они должны различаться между собой.

Состояние может включать секцию внутренних действий (рис. 2.12 б), которая содержит перечень внутренних действий или активностей, которые выполняются в процессе нахождения моделируемого элемента в данном состоянии. Каждое из действий записывается в виде отдельной строки и имеет следующий формат:

<метка-действия '/' выражение-действия>

Метка действия указывает на обстоятельства или условия, при которых будет выполняться деятельность, определенная выражением действия. При этом выражение действия может использовать любые атрибуты и связи, которые принадлежат области имен или контексту моделируемого объекта. Если список выражений действия пустой, то разделитель в виде наклонной черты '/' может не указываться.

Перечень меток действия имеет фиксированные значения в языке UML, которые не могут быть использованы в качестве имен событий. Значения могут быть следующими:

entry – указывает на действие, специфицированное следующим за ней выражением действия, которое выполняется в момент входа в данное состояние (входное действие);

exit – указывает на действие, специфицированное следующим за ней выражением действия, которое выполняется в момент выхода из данного состояния (выходное действие);

do – специфицирует выполняющуюся деятельность (do activity), которая выполняется в течение всего времени, пока объект находится в данном состоянии, или до тех пор, пока не закончится вычисление, специфицированное следующим за ней выражением действия (при этом при завершении события генерируется соответствующий результат);

include – используется для обращения к подавтомату, при этом следующее за ней выражение действия содержит имя этого подавтомата.

Метки entry и exit, как правило, логически связываются с предусловиями и постусловиями потока событий моделируемого варианта использования (см. п. 2.1.7).

Во всех остальных случаях метка действия идентифицирует событие, которое запускает соответствующее выражение действия. Эти события называются внутренними переходами и семантически эквивалентны переходам в само это состояние, за исключением той особенности, что выход из этого состояния или повторный вход в него не происходит. Это означает, что действия входа и выхода не выполняются. В качестве примера рассмотрим ситуацию ввода пароля пользователя при аутентификации входа в некоторую программную систему (рис. 2.13).

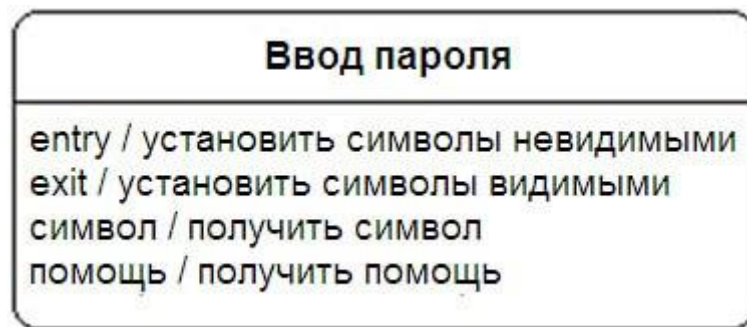


Рис. 2.13. Пример состояния с непустой секцией внутренних действий

В этом случае список внутренних действий в данном состоянии не пуст и включает 4 отдельных действия, первые два из которых стандартные и описаны ранее, а два последних определяются своей спецификацией.

Начальное состояние представляет собой частный случай состояния, которое не содержит никаких внутренних действий (псевдосостояния). В этом состоянии находится объект по умолчанию в начальный момент времени. Оно служит для указания на диаграмме состояний графической области, от которой начинается процесс изменения состояний. Графически начальное состояние в языке UML обозначается в виде закрашенного кружка (рис. 2.14, а), из которого может только выходить стрелка, соответствующая переходу.



Рис. 2.14. Графическое изображение начального и конечного состояний на диаграммах UML: *a* – начальное состояние; *б* – конечное состояние

На самом верхнем уровне представления объекта переход из начального состояния может быть помечен событием создания (инициализации) данного объекта. В противном случае переход никак не помечается. Если этот переход не помечен, то он является первым переходом в следующее за ним состояние.

Конечное (финальное) состояние представляет собой частный случай состояния, которое также не содержит никаких внутренних действий (псевдосостояния). В этом состоянии будет находиться объект по умолчанию после завершения работы автомата в конечный момент времени. Оно служит для указания на диаграмме состояний графической области, в которой завершается процесс изменения состояний или жизненный цикл данного объекта. Графически конечное состояние в языке UML обозначается в виде закрашенного кружка, помещенного в окружность (рис. 2.14, *б*), в которую может только входить стрелка, соответствующая переходу.

Переход между состояниями представляет собой разновидность отношения на диаграммах деятельности.

Простой переход (simple transition) – отношение между двумя последовательными состояниями, которое указывает на факт смены одного состояния другим.

Пребывание моделируемого объекта в первом состоянии может сопровождаться выполнением некоторых действий, а переход во второе состояние будет возможен после завершения этих действий, а также после удовлетворения некоторых дополнительных условий. В этом случае говорят, что «переход срабатывает» или «происходит срабатывание перехода».

До срабатывания перехода объект находится в предыдущем от него состоянии, называемом исходным состоянием, или в источнике (не путать с начальным состоянием – это разные

понятия), а после его срабатывания объект находится в последующем от него состоянии (целевом состоянии).

Переход осуществляется при наступлении некоторого события: окончания выполнения деятельности (do activity), получения объектом сообщения или приема сигнала. На переходе указывается имя события. Кроме того, на переходе могут указываться действия, производимые объектом в ответ на внешние события при переходе из одного состояния в другое. Срабатывание перехода может зависеть не только от наступления некоторого события, но и от выполнения определенного условия, называемого сторожевым условием. Объект перейдет из одного состояния в другое в том случае, если произошло указанное событие и сторожевое условие приняло значение «истина».

Переходы представляются стрелками от одного состояния к другому, которые вызываются выполнением некоторых функций объекта. Переходы имеют метки, которые синтаксически состоят из трех необязательных частей (рис. 2.15): <Событие> <[Условие]> </ Действие>.



Рис. 2.15. Диаграмма состояний объекта «заказ»

Переход, стрелка которого соединена с границей некоторого составного состояния, обозначает переход в составное состояние. Он эквивалентен переходу в начальное состояние каждого из подавтоматов (возможно, единственному), входящих в состав данного суперсостояния.

В отдельных случаях переход может иметь несколько состояний-источников и несколько целевых состояний, относящихся к соответствующим процессам. Такой переход называется параллельным.

Параллельный переход (parallel transition) – отношение между параллельными состояниями, которое указывает на факт перехода из одного процесса в другой.

Введение в рассмотрение параллельного перехода обусловлено необходимостью синхронизировать и / или разделить отдельные подпроцессы на параллельные нити без спецификации дополнительной синхронизации в параллельных подавтоматах.

Графически такой переход изображается вертикальной чертой. Если параллельный переход имеет две или более входящие дуги (рис. 2.16, а), то его называют соединением (join). Если же он имеет две или более исходящие из него дуги (рис. 2.16, б), то его называют ветвлением (fork).

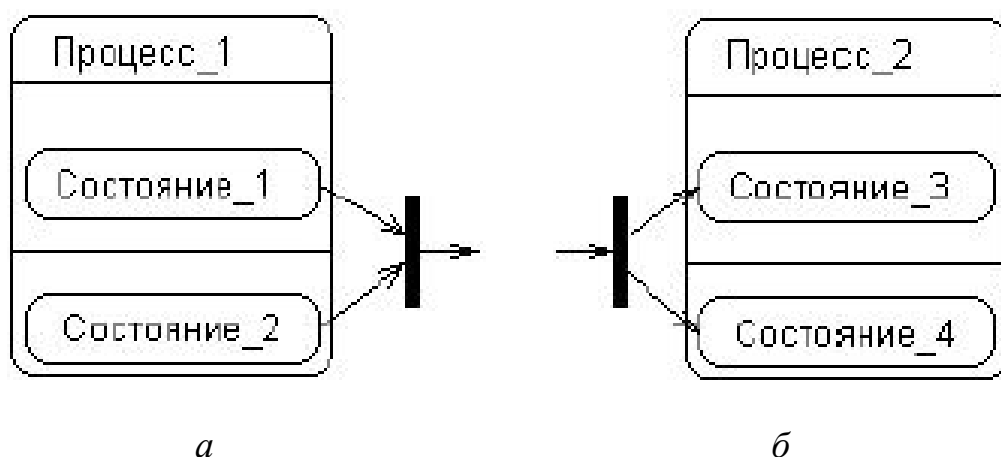


Рис. 2.16. Различные варианты переходов в (из) составное состояние:
а – соединение; б – ветвление

По своему назначению диаграмма состояний не является обязательным представлением в модели и присоединяется к тому элементу, который, по замыслу разработчиков, имеет нетривиальное поведение в течение своего жизненного цикла.

Наличие у системы нескольких состояний, отличающихся от простой дихотомии типа «исправен – неисправен», «активен – неактивен», «ожидание – реакция» и т.п., уже служит признаком необходимости построения диаграммы состояний. В качестве начального варианта диаграммы состояний, если нет очевидных соображений по поводу состояний объекта, можно воспользоваться этими суперсостояниями, рассматривая их как составные и уточняя их (детализируя их внутреннюю структуру) по мере рассмотрения логики поведения объекта.

При выделении состояний и переходов следует помнить, что длительность срабатывания отдельных переходов должна быть существенно меньшей, чем нахождение моделируемого объекта в соответствующих состояниях. Каждое из состояний должно характеризоваться определенной устойчивостью во времени. Другими словами, из каждого состояния на диаграмме не может быть самопроизвольного перехода в другое состояние. Все переходы должны быть явно специфицированы, в противном случае построенная диаграмма состояний является либо неполной, либо ошибочной.

При разработке диаграммы состояний нужно постоянно следить, чтобы объект в каждый момент мог находиться только в единственном состоянии. Если это не так, то данное обстоятельство может быть как следствием ошибки, так и неявным признаком наличия параллельности у поведения моделируемого объекта. В последнем случае следует явно специфицировать необходимое число подавтоматов, вложив их в то составное состояние (рис. 2.16), которое характеризуется нарушением условия одновременности.

Следует обязательно проверять, что никакие два перехода из одного состояния не могут сработать одновременно (требование отсутствия конфликтов у переходов). Наличие такого конфликта может служить признаком ошибки либо

неявной параллельности типа ветвления рассматриваемого процесса на два и более подавтомата. Если параллельность по замыслу разработчика отсутствует, то необходимо ввести дополнительные сторожевые условия либо изменить существующие, чтобы исключить конфликт переходов. При наличии параллельности следует заменить конфликтующие переходы одним параллельным переходом типа ветвления (рис. 2.16, б).

2.1.10. Диаграммы деятельности

Диаграмма деятельности – это частный случай диаграммы состояний. На диаграмме деятельности представлены переходы потока управления от одной активности к другой внутри системы. Этот вид диаграмм обычно используется для описания поведения, включающего множество параллельных процессов.

К основным элементам диаграмм деятельности относятся (рис. 2.17):

- овалы, изображающие действия объекта;
- линейки синхронизации, указывающие на необходимость завершить или начать несколько действий (модель логического условия «И»);
- ромбы, отражающие принятие решений по выбору одного из маршрутов выполнения процесса (модель логического условия «ИЛИ»);
- стрелки – отражают последовательность действий, могут иметь метки условий.

На диаграмме деятельности могут быть представлены действия, соответствующие нескольким вариантам использования. На таких диаграммах появляется множество начальных точек, поскольку они отражают реакцию системы на множество внешних событий. Таким образом, диаграммы деятельности позволяют получить полную картину поведения системы и легко оценивать влияние изменений в отдельных вариантах использования на конечное поведение системы.

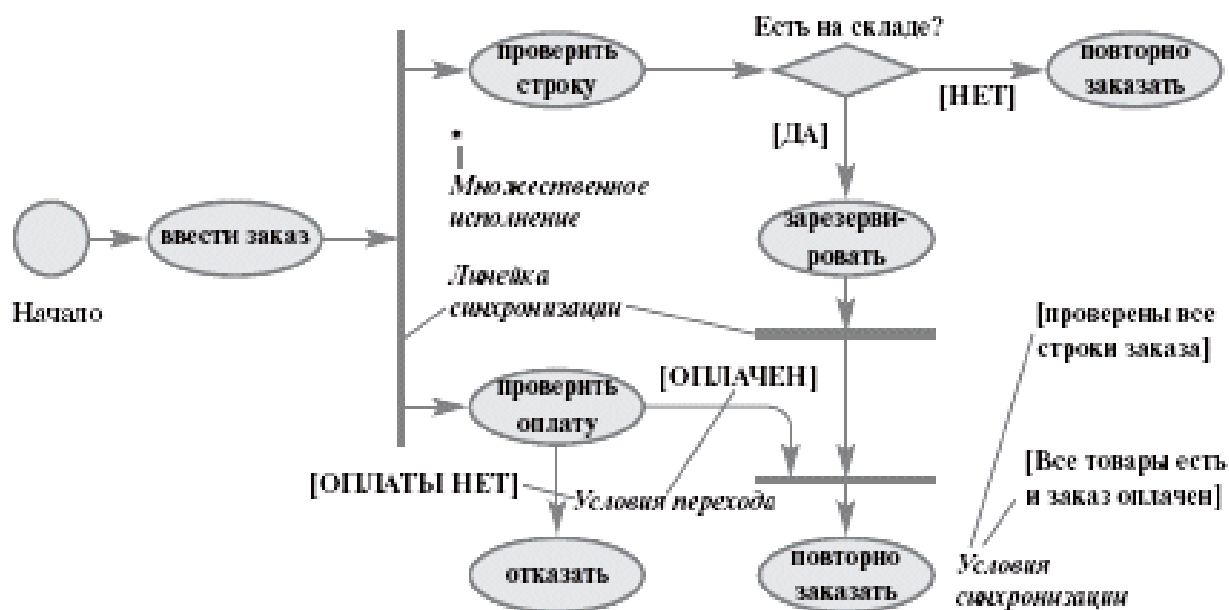


Рис. 2.17. Диаграмма деятельности – обработка заказа

Любая деятельность может быть подвергнута дальнейшей декомпозиции и представлена в виде отдельной диаграммы деятельности или спецификации (словесного описания).

Каждое состояние на диаграмме деятельности соответствует выполнению некоторой элементарной операции, а переход в следующее состояние срабатывает только при завершении этой операции в предыдущем состоянии. Графически диаграмма деятельности представляется в форме графа деятельности, вершинами которого являются состояния действия, а дугами – переходы от одного состояния действия к другому.

Достоинством диаграммы деятельности считается возможность развёртывания её в виде дорожек, т.е. с привязкой к исполнителям конкретных операций алгоритма. В качестве примера рассмотрим вариант использования «Обработка заказа». Цель процесса – принять заказ, подготовить его к выполнению и выполнить. В рамках данного процесса выполняется прием товара, фиксация его оплаты, отпуск товара со склада и отгрузка клиенту. В процессе участвуют три отдела торгового предприятия. Схема диаграммы деятельности для задачи «Обработка заказа» с дорожками показана на рис. 2.18.

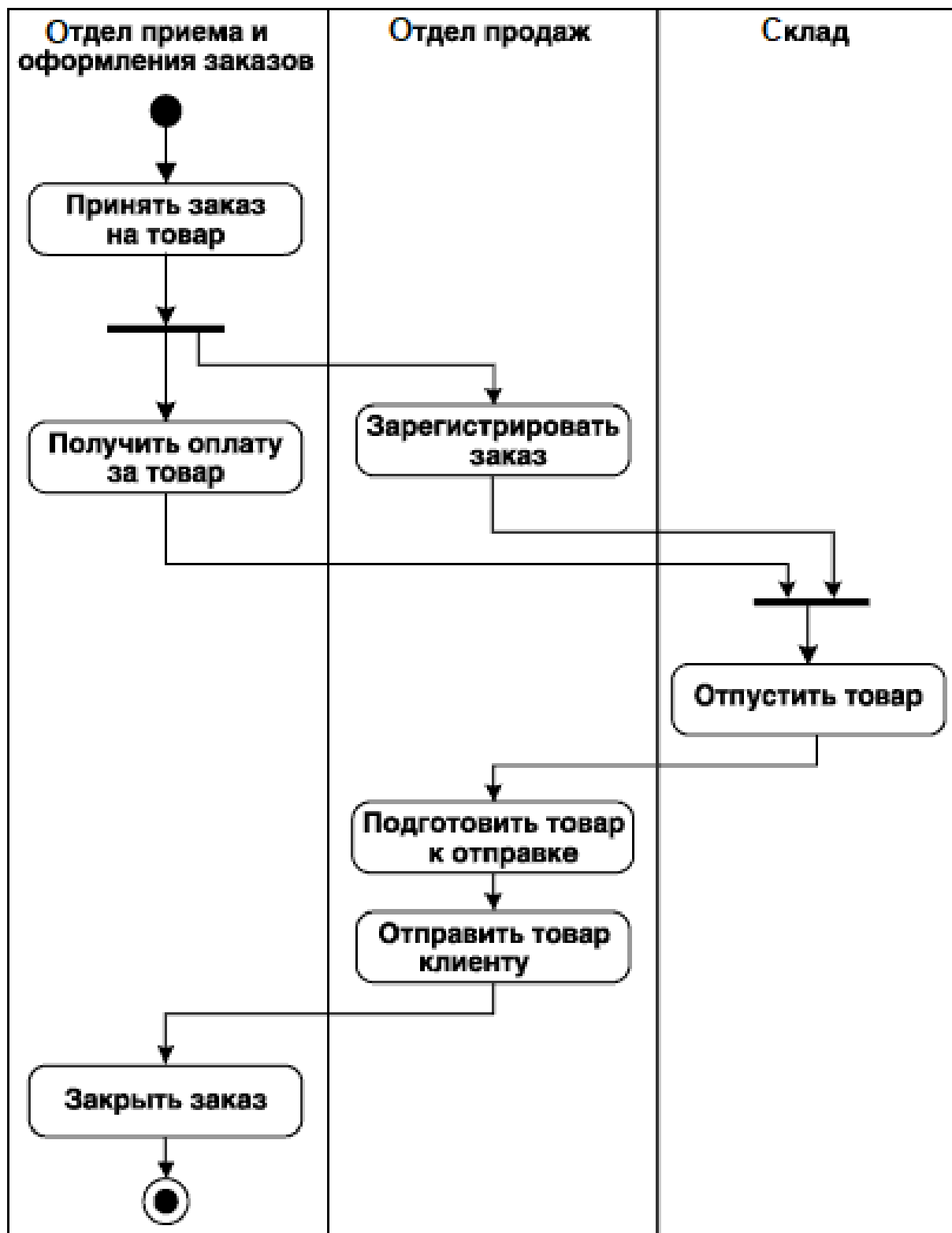


Рис. 2.18. Пример диаграммы деятельности с дорожками для процесса «Обработка заказа»

2.1.11. Диаграммы взаимодействия

Диаграммы взаимодействия предназначены для моделирования динамических аспектов системы. Они показывают взаимодействие объектов, включающее набор объектов и их отношений, а также пересылаемые между объектами сообщения. К диаграммам взаимодействия относятся диаграммы последовательности (Sequence Diagram) и диаграммы сотрудничества (Collaboration Diagram). Эти диаграммы позволяют с разных точек зрения рассмотреть взаимодействие объектов в создаваемой системе.

Диаграмма последовательности (sequence diagram) – диаграмма UML, на которой показаны взаимодействия объектов, упорядоченные по времени их проявления.

Элементами диаграммы последовательности являются объекты, линии жизни, фокус управления, сообщения (рис. 2.19).

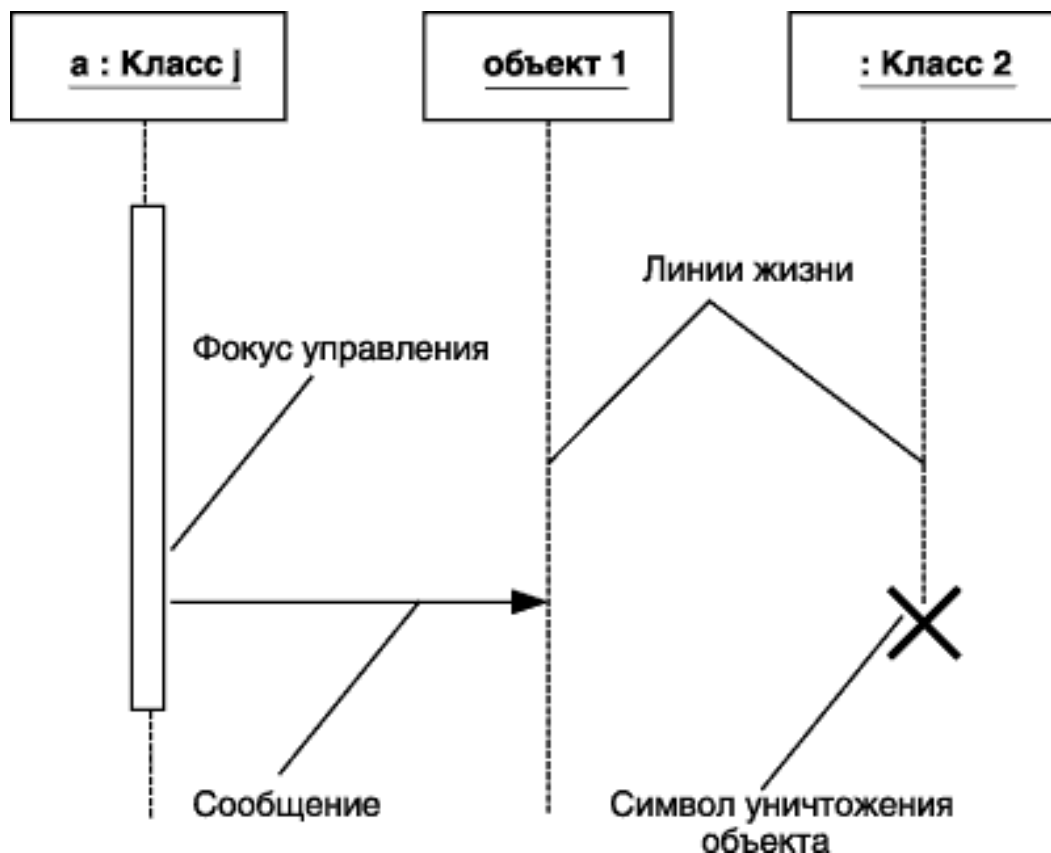


Рис. 2.19. Элементы диаграммы последовательности

Объекты на диаграмме последовательности представляются в виде прямоугольников, от которых проведена пунктирная линия, называемая линией жизни объекта. Данная линия представляет собой фрагмент жизненного цикла объекта. Фокус управления – специальный символ, указывающий период времени, в течение которого объект выполняет некоторое действие, находясь в активном состоянии. Сообщения отображены в виде стрелок между двумя линиями объектов и сигнатурой соответствующего метода.

Порядок построения диаграмм последовательности.

1. Построение диаграммы последовательности целесообразно начинать с выделения из всей совокупности только тех классов, объекты которых участвуют в моделируемом взаимодействии. После этого все объекты нанести на диаграмму с соблюдением некоторого порядка инициализации сообщений. Здесь необходимо установить, какие объекты будут существовать постоянно, а какие временно (только на период выполнения требуемых действий).

2. После визуализации на диаграмме объектов можно приступить к спецификации сообщений. При этом следует учитывать те роли, которые играют сообщения в проектируемой системе. При необходимости уточнения этих ролей надо использовать их разновидности и стереотипы. Для уничтожения объектов, которые создаются на время выполнения своих действий, нужно предусмотреть отдельное сообщение.

3. Наиболее простые случаи ветвления процесса взаимодействия можно изобразить на одной диаграмме с использованием соответствующих графических примитивов. Однако следует помнить, что каждый альтернативный поток управления может существенно затруднить понимание построенной модели. Поэтому общим правилом является визуализация каждого потока управления на отдельной диаграмме последовательности. В этой ситуации такие отдельные диаграммы должны рассматриваться совместно как одна модель взаимодействия.

4. Дальнейшая детализация диаграммы последовательности связана с введением временных ограничений на выполнение отдельных действий в системе. Для простых асинхронных сообщений временные ограничения могут отсутствовать. Однако необходимость синхронизировать сложные потоки управления, как правило, требует введения в модель таких ограничений.

На диаграмме последовательности изображаются объекты, которые непосредственно участвуют во взаимодействии, и не показываются возможные статические ассоциации с другими объектами. Для диаграммы последовательности ключевым моментом является динамика взаимодействия объектов во времени. Поэтому диаграмма последовательности имеет два измерения:

1) слева направо в виде вертикальных линий, каждая из которых изображает линию жизни отдельного объекта, участвующего во взаимодействии. Графически каждый объект изображается прямоугольником и располагается в верхней части своей линии жизни;

2) вертикальная временная ось, направленная сверху вниз. Начальному моменту времени соответствует самая верхняя часть диаграммы.

Взаимодействия объектов реализуются посредством сообщений, которые посылаются одними объектами другим. Сообщения изображаются в виде горизонтальных стрелок с именем сообщения и также образуют порядок по времени своего возникновения. Масштаб на оси времени не указывается, поскольку диаграмма последовательности моделирует лишь временную упорядоченность взаимодействий типа «раньше – позже».

Для уточнения динамических аспектов взаимодействия в модель UML вводятся диаграммы кооперации.

Диаграмма кооперации (Collaboration Diagram) – диаграмма UML, на которой показаны взаимодействия объектов сообщения для выполнения определенной функции или сценария.

Диаграммы кооперации полезны для более глубокого анализа взаимодействия в системе при ее проектировании. Они

позволяют визуализировать взаимодействие между компонентами и понять, как они работают вместе для достижения общей цели.

Основными целями применения диаграмм кооперации являются:

- моделирование взаимодействий объектов друг с другом, визуализирующее сложные процессы;
- упрощение коммуникации при обсуждении архитектуры системы между разработчиками и бизнес-аналитиками;
- выявление и уточнение требований к системе (важно показать, какие объекты необходимы для выполнения определённых функций);
- выявление потенциальных проблем во взаимодействии объектов до начала разработки.

Выделим основные компоненты диаграммы кооперации.

1. Объекты – обозначают классы или экземпляры объектов, участвующих во взаимодействии. Обозначаются прямоугольниками с текстом внутри.

2. Сообщения – стрелки между объектами, указывающие на направление и порядок вызова методов или передачи данных. Сообщения могут быть синхронными или асинхронными.

3. Порядок сообщений – нумерация сообщений указывает последовательность их отправки.

4. Связи – линии, соединяющие объекты, показывают, что они могут взаимодействовать друг с другом.

В качестве примера рассмотрим процесс оформления товара в книжном интернет-магазине. На диаграмме кооперации (рис. 2.20) можно выделить в качестве объектов клиента (пользователь, совершающий покупку), интернет-магазин (основная система, через которую выполняются продажи), корзину (объект, хранящий выбранные товары), книгу (объект, представляющий товар в магазине), заказ (объект, предоставляющий оформленный заказ), платежную систему (система, обрабатывающая платежи). Определим следующие сообщения: добавление «Клиентом» «Книги» в «Корзину», «Клиент» просматривает содержимое «Корзины», оформление

«Заказа» «Клиентом», «Интернет-магазин» создает новый «Заказ», «Интернет-магазин» запрашивает информацию о платеже у «Платежной системы», «Платежная система» обрабатывает платеж, «Интернет-магазин» подтверждает заказ клиенту.

Преимущества диаграмм кооперации:

- визуальное представление упрощает понимание сложных процессов и взаимодействий;
- в отличие от диаграмм последовательности легко модифицируемы в процессе разработки по мере появления новых требований или изменений в архитектуре;
- позволяют заранее увидеть возможные проблемы в логике взаимодействия, что снижает риски на этапе реализации;
- обеспечивают разделение системы на модули, что облегчает разработку и тестирование.

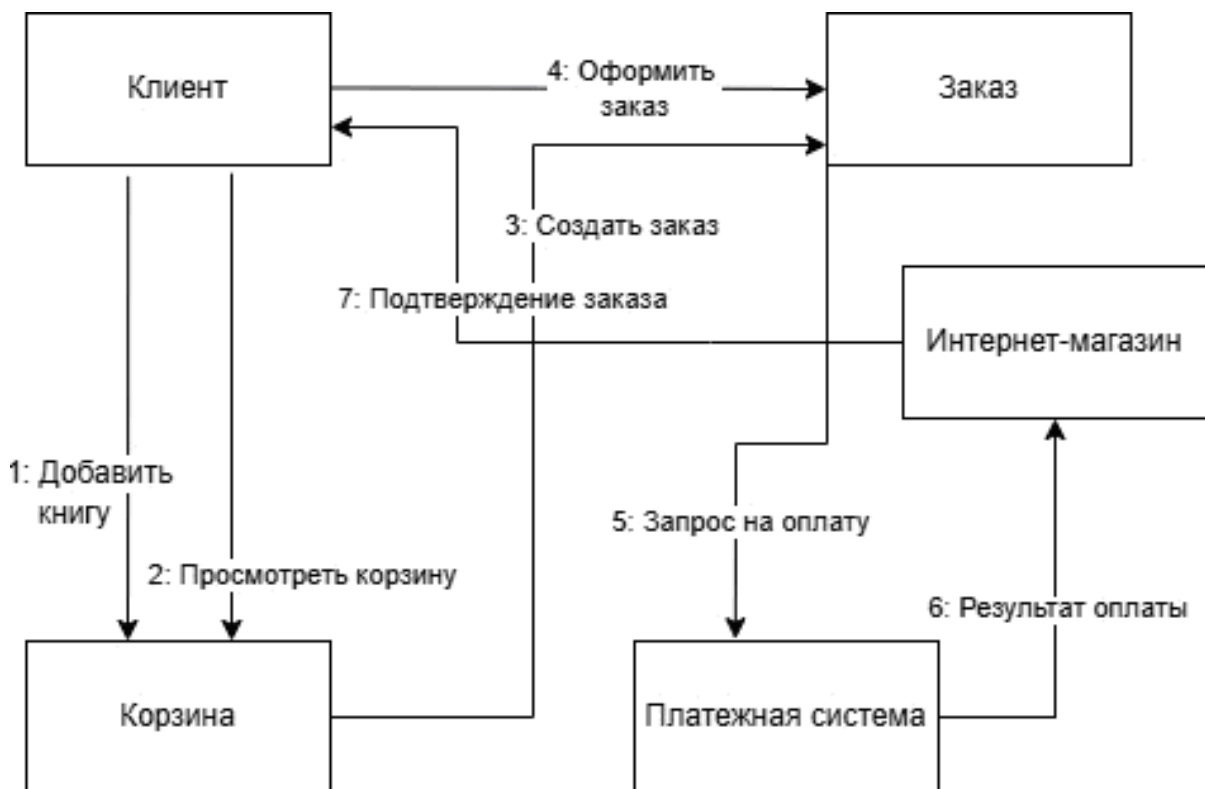


Рис. 2.20. Пример диаграммы кооперации для обработки заказа в интернет-магазине

2.1.12. Диаграммы компонентов

Диаграммы компонентов (component diagram) – диаграмма физической реализации ПО, используемая для визуализации статического аспекта его физических компонентов и их отношений, а также для специфицирования деталей этих компонентов в UML.

Диаграмма компонентов позволяет определить состав программных компонентов, в роли которых может выступать исходный, бинарный и исполняемый код, и др., а также установить зависимости между ними.

При разработке диаграмм компонентов ставятся следующие цели:

- спецификация общей структуры исходного кода системы;
- спецификация исполнимого варианта системы.

Для графического представления компонентов используются прямоугольники. Внутри прямоугольника записывается имя компонента и стереотип. Возможные типы компонентов показаны в табл. 2.4. Компоненты-файлы на схеме приводятся с указанием расширения.

Таблица 2.4

Типы компонентов программного приложения

Стереотип	Тип компонента
«source»	файл исходного текста программы
«form»	файл интерфейсной формы или web-страницы
«executable»	исполняемый файл
«library»	статическая или динамическая библиотека
«document»	файл документации
«data»	файл данных
«table»	таблица базы данных
«view»	представление как объект базы данных
«file»	любой другой файл, кроме указанных в этой таблице

При спецификации общей структуры исходного кода ПС необходимо учитывать специфику используемого языка программирования. В частности, в Java рекомендуется отдельный

класс описывать в отдельном файле несмотря на то, что язык позволяет описывать несколько классов в одном файле и использовать механизм внутренних классов. Диаграмма компонентов, применяемая для рассматриваемой цели, изображена на рис. 2.21.

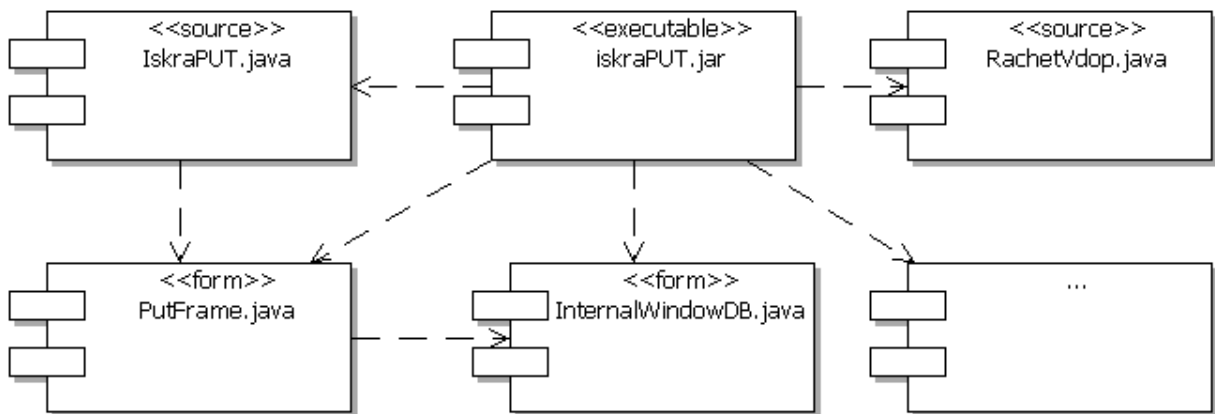


Рис. 2.21. Фрагмент диаграммы компонентов, специфицирующей структуру исходного кода

На диаграмме компонентов могут быть показаны детали структурной реализации. На рис. 2.22 представлены варианты структурной организации модуля исходного кода.

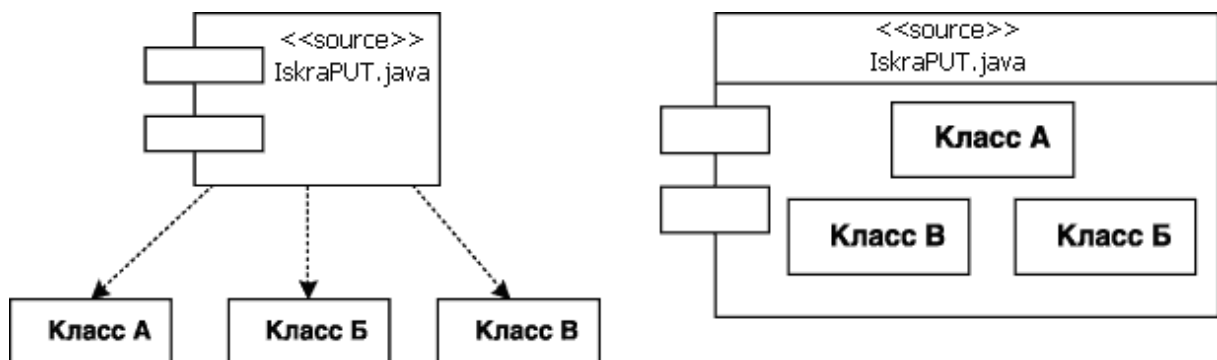


Рис. 2.22. Способы реализации классов компонентом

Зависимость между программными компонентами может отражать наличие в независимом компоненте описаний классов, которые используются в зависимом компоненте для создания соответствующих объектов. Зависимости могут связывать

компоненты и импортируемые этим компонентом интерфейсы, а также различные виды компонентов между собой. Так, изображенный на рис. 2.23 фрагмент диаграммы компонентов отражает отношение зависимости модуля `main` от импортируемого интерфейса `IDialog`, который, в свою очередь, реализуется компонентом с именем `Library`. При этом для второго компонента этот интерфейс является экспортируемым.

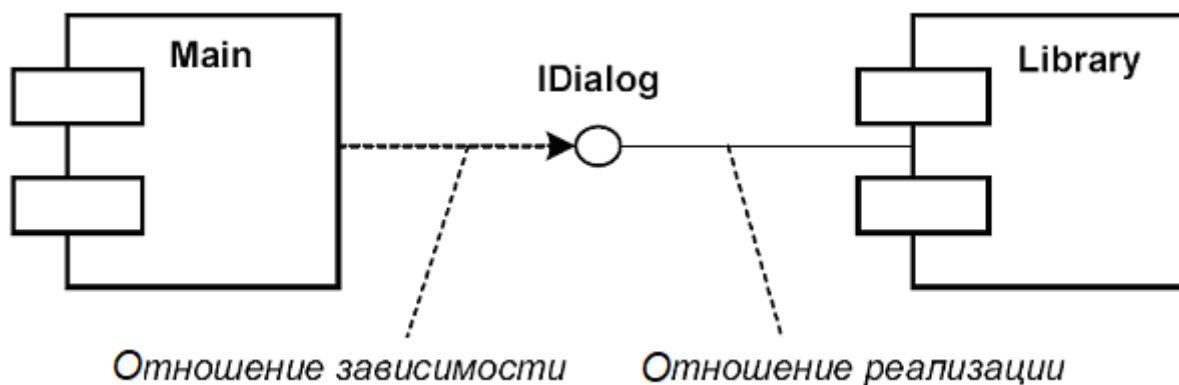


Рис. 2.23. Фрагмент диаграммы компонентов с отношениями зависимости и реализации

Диаграмма, отражающая взаимодействие компонентов разных типов, нагляднее представляется с использованием стереотипных элементов (пиктограмм). На рис. 2.24 показан пример такой модели со стереотипными элементами типа «source», «library», «form», «document», «table».

На рис. 2.25 показан пример диаграммы компонентов ПС управления банкоматом, отражающий её физическую реализацию на уровне компонентов ПО.

2.1.13. Диаграммы развертывания

Диаграмма развертывания, или применения (deployment diagram) – это еще один из двух видов диаграмм, используемых при моделировании физических аспектов объектно-ориентированной системы.

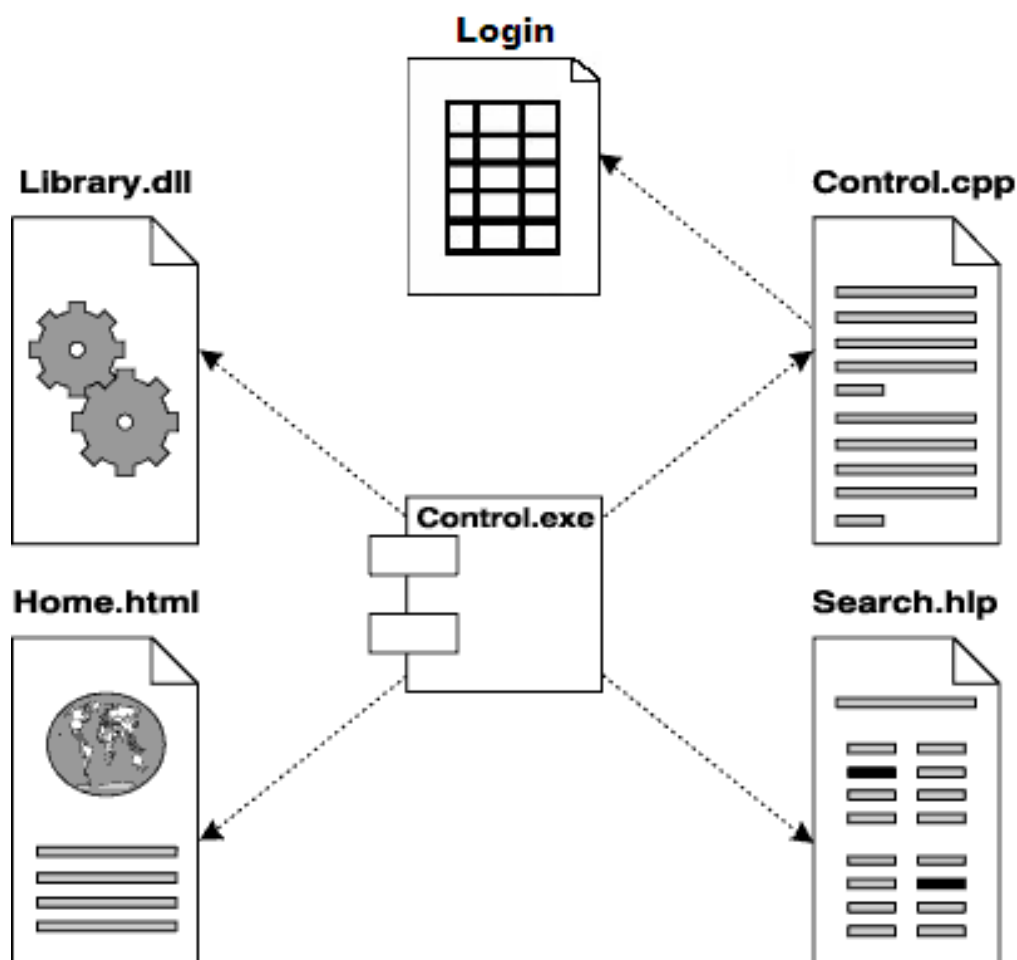


Рис. 2.24. Зависимость между компонентами приложения с использованием стереотипных элементов

Диаграмма развертывания (deployment diagram) – это диаграмма физической реализации системы, которая показывает конфигурацию узлов, где производится обработка информации, и то, какие компоненты размещены на каждом узле.

Диаграммы развертывания используются для моделирования статического вида системы с точки зрения развертывания. В основном под этим понимается моделирование топологии аппаратных средств, на которых выполняется система. Диаграммы развертывания важны для визуализации, специфицирования и документирования встроенных, клиент-серверных и распределенных систем.

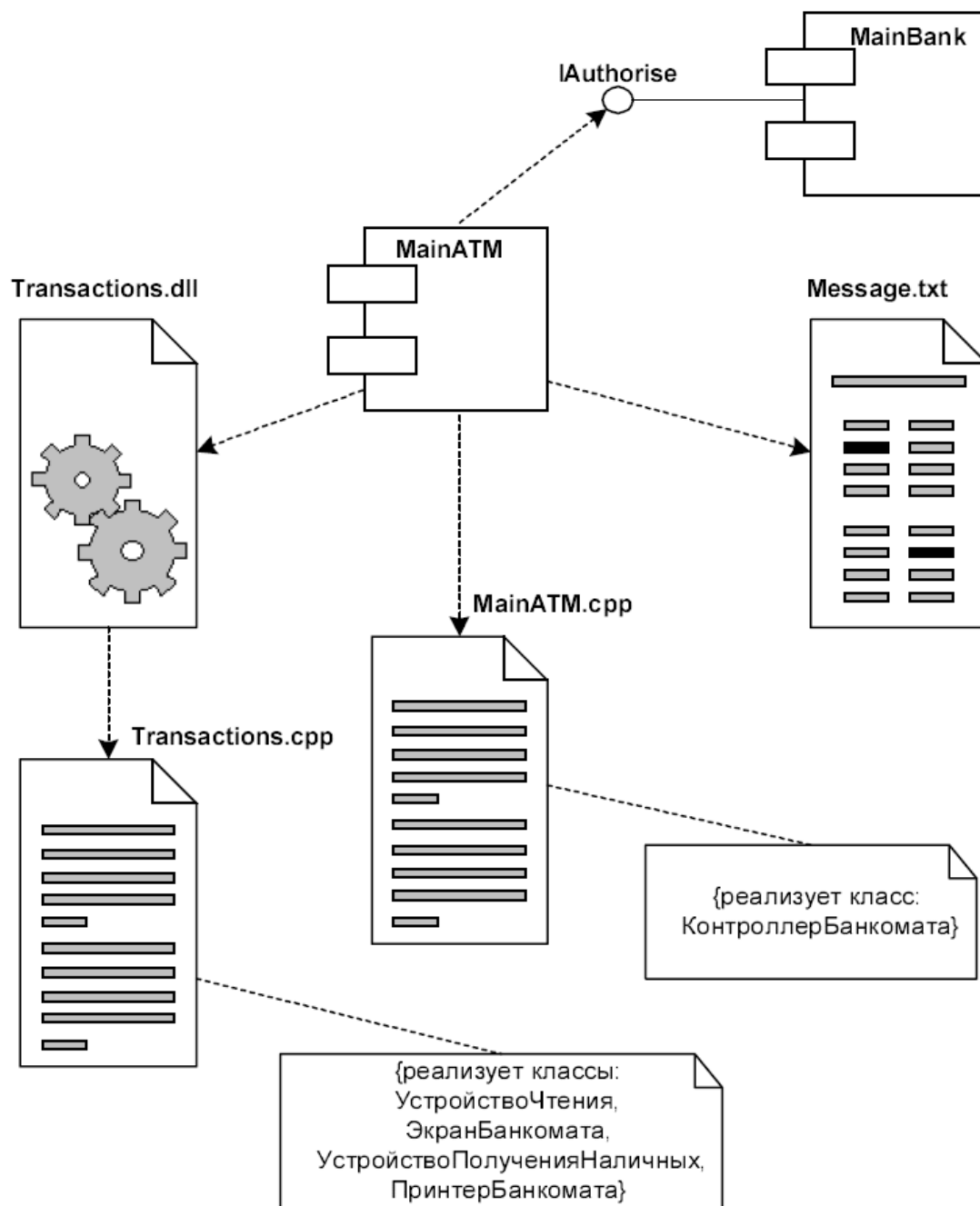


Рис. 2.25. Диаграмма компонентов программной системы управления банкоматом

Для графического представления элементов диаграмм развертывания используются параллелепипеды. Внутри параллелепипеда записывается имя и стереотип. Возможные типы компонентов показаны в табл. 2.5.

Таблица 2.5

Типы компонентов программного приложения

Стереотип	Тип элемента
«processor»	Ресурсоемкий узел
«device»	Нересурсоемкий узел (устройство)
«sensor»	Измерительное устройство (датчик)
«printer»	Печатающее устройство (принтер)
«net»	Сетевая передающая среда
«database»	База данных
«node»	Узел, не попадающий под тип, указанный в этой таблице

На рис. 2.26 показан фрагмент соединения типа «Клиент – Сервер» посредством локальной вычислительной сети.

Кроме соединений на диаграмме развертывания могут присутствовать отношения зависимости между узлом и размещаемыми на нем компонентами.

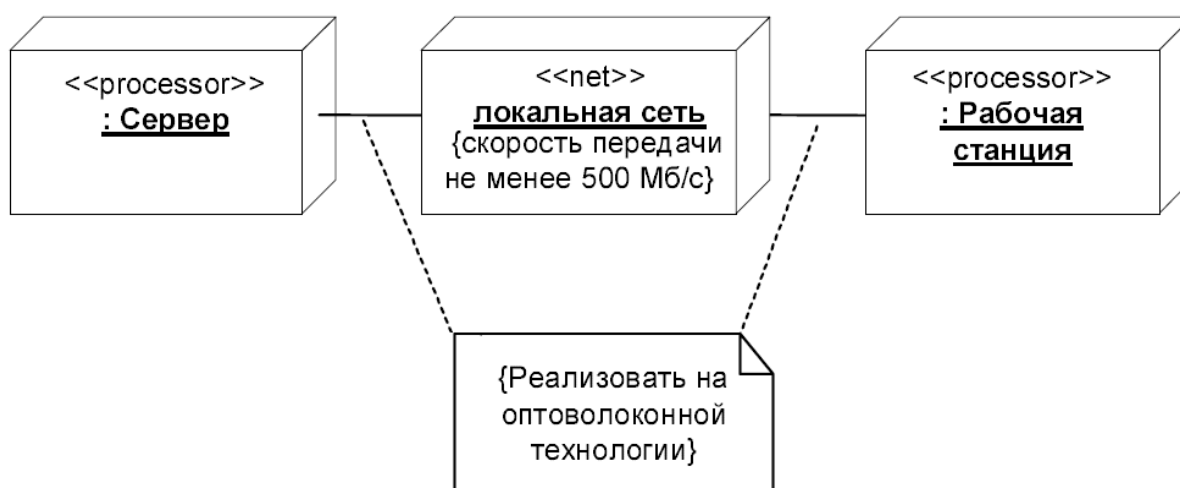


Рис. 2.26. Фрагмент диаграммы развертывания с соединениями между узлами

Подобный способ является альтернативой вложенному изображению компонентов внутри символа узла, что не всегда удобно, поскольку делает этот символ излишне объемным. Поэтому при большом количестве развернутых на узле компонентов соответствующую информацию можно представить в форме отношения зависимости, которая обозначается пунктирной линией (рис. 2.27).

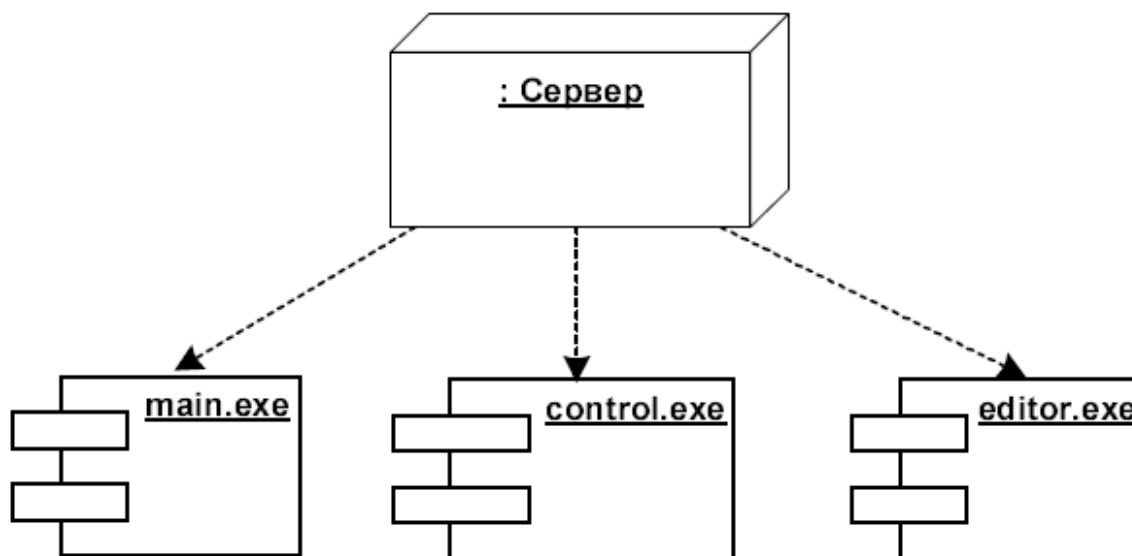


Рис. 2.27. Диаграмма развертывания с отношением зависимости между узлом и развернутыми на нем компонентами

2.1.14. Рациональный унифицированный процесс

Процесс построения отдельных типов диаграмм UML имеет свои особенности, тесно связанные с семантикой элементов этих диаграмм. Сам процесс ООП в контексте языка UML получил специальное название – рациональный унифицированный процесс (Rational Unified Process, RUP). Концепция RUP и основные ее элементы разработаны А. Джекобсоном в ходе его работы над языком UML.

Унифицированный процесс – это процесс, управляемый прецедентами, которые отражают сценарии взаимодействия пользователей с ПС и взгляд пользователей на систему снаружи.

Суть концепции RUP заключается в последовательной декомпозиции процесса ООП на отдельные этапы. На каждом этапе осуществляется разработка соответствующих типов канонических диаграмм модели системы.

Этап 1. Строятся логические представления статической модели структуры системы (Кто? Что? Сколько?).

Этап 2. Строятся логические представления динамической модели поведения системы (Как? Когда?).

Этап 3. Строятся физические представления модели системы (Что? Где? Куда?).

Таким образом, процесс ООП можно представить как поуровневый спуск от наиболее общих моделей и представлений концептуального уровня к более частным и детальным представлениям логического и физического уровней (рис. 2.28). При этом на каждом из этапов ООП данные модели последовательно дополняются всё большим количеством деталей, что позволяет им более адекватно отражать различные аспекты конкретной реализации сложной системы.

В результате реализации RUP должны быть построены канонические диаграммы на языке UML. При этом последовательность их разработки чаще всего совпадает с их последовательной нумерацией. Более детально основные этапы RUP показаны на рис. 2.29.



Рис. 2.28. Общая схема взаимосвязей моделей и представлений сложной системы в процессе объектно-ориентированного анализа и проектирования



Рис. 2.29. Схема применения концепции RUP при разработке ПС

Одним из важнейших этапов разработки согласно концепции RUP, является этап определения требований, который заключается в сборе всех возможных пожеланий заказчика к работе системы. Позднее эти данные должны быть систематизированы и структурированы. На этом же этапе в ходе интервью с будущими пользователями и изучения документов аналитики должны собрать как можно больше требований к будущей системе. Вместе с тем, как уже неоднократно отмечалось ранее, пользователи часто сами не представляют, что они должны получить в итоге. Для облегчения этого процесса аналитики используют диаграммы прецедентов (п. 2.1.6) и диаграммы классов (п. 2.1.8). Именно эти диаграммы строятся в первую очередь. Логично вначале построить диаграмму вариантов использования, а затем для каждого варианта или для ПС в целом строить диаграмму классов. Логика RUP в плане построения диаграмм UML показана на рис. 2.30.

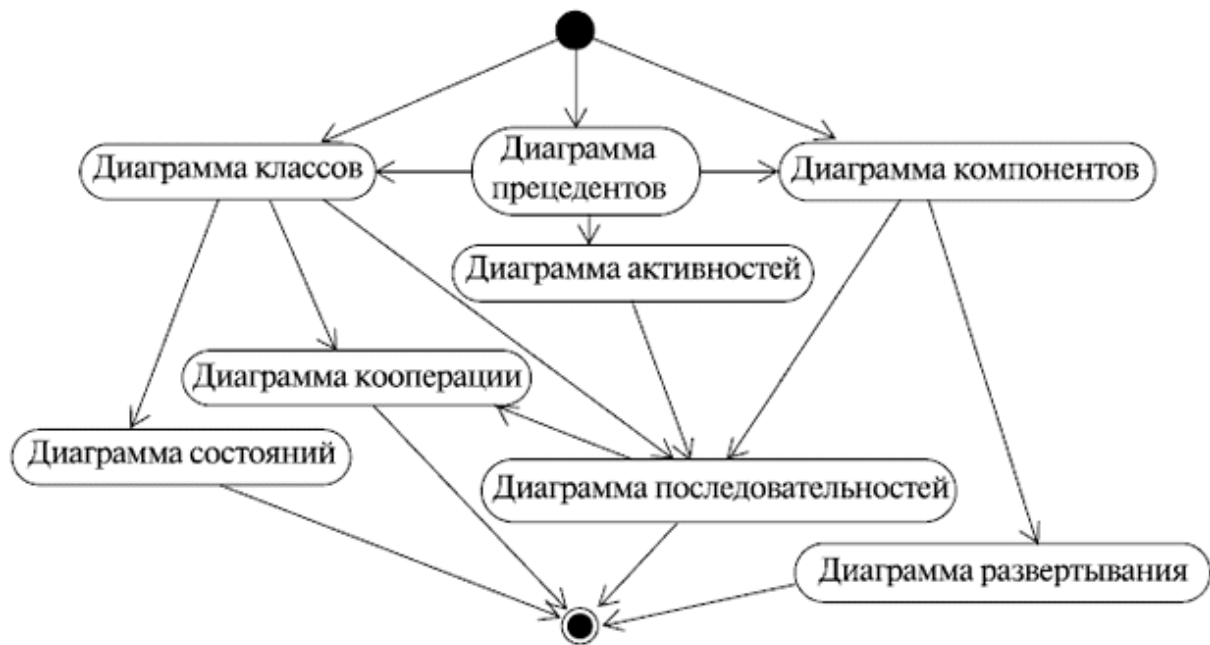


Рис. 2.30. Стандартный процесс RUP при построении модели ПС

Далее для каждого варианта или для ПС в целом строится диаграмма активностей (п. 2.1.10). Классы в их взаимодействии моделируются диаграммами взаимодействия (п. 2.1.11), а классы с наиболее нетривиальным поведением моделируются диаграммами состояний (п. 2.1.9). Последними строятся диаграммы физической реализации ПС. Логическая структура построения модели ПС на этапах её жизненного цикла показана на рис. 2.31.

Процесс RUP обширен и содержит рекомендации по ведению различных программных проектов, начиная от создания программ группой разработчиков в несколько человек и до распределенных программных проектов, объединяющих тысячи человек на разных континентах. Однако, несмотря на их колоссальную разницу, методы применения моделей, создаваемых при помощи UML, будут одни и те же. Диаграммы UML, создаваемые на различных этапах разработки, неотделимы от остальных артефактов программного проекта и часто являются связующим звеном между отдельными процессами (этапами) RUP.



Рис. 2.31. Логическая структура построения модели ПС на этапах

Решение о применении конкретных диаграмм зависит от поставленного в компании процесса разработки, который, хотя и называется унифицированным, однако не есть нечто статичное. Компания Rational не только предлагает его улучшать и дорабатывать, но и предоставляет специальные средства внесения изменений в базу данных RUP.

2.2. CASE-СРЕДСТВА

В современном обществе программное обеспечение играет важную роль в различных сферах деятельности, таких как бизнес, образование, наука и т.д. Разработка ПО представляет собой сложный процесс, требующий совместной работы большого числа специалистов. Для эффективного выполнения этого процесса используются разные методы и инструменты, включая CASE-средства.

CASE-средства предназначены для автоматизации и поддержки различных аспектов разработки ПС, облегчают процесс их создания, модификации и поддержки. Они предоставляют разработчикам средства для моделирования, анализа, проектирования и документирования системы. Использование CASE-средств сокращает время и ресурсы, а также повышает надежность и гибкость системы и улучшает взаимодействие между участниками команды.

2.2.1. Характеристика и обзор рынка CASE-средств

CASE-средства реализуют CASE-технологии создания и сопровождения ПС. Термин CASE (Computer Aided Software Engineering) используется в настоящее время в весьма широком смысле. Первоначальное значение термина CASE было ограничено вопросами автоматизации разработки только ПО. В настоящее время использование CASE-средств приобрело новый смысл, охватывающий процесс разработки сложных ПС в целом.

CASE-средства – это программные средства, поддерживающие процессы создания и сопровождения ПС, в том числе анализ и формулировку требований, проектирование специального ПО (программных приложений) и баз данных, генерацию кода, тестирование, документирование, обеспечение качества, конфигурационное управление и управление проектом, а также другие вспомогательные процессы.

CASE-средства вместе с системным ПО и техническими средствами образуют полную среду разработки ПС.

Современный рынок насчитывает более 300 CASE-средств, наиболее мощные из которых используются практически всеми ведущими компаниями-разработчиками ПО. Современные CASE-средства охватывают обширную область поддержки многочисленных технологий проектирования ПС: от простых средств анализа и документирования до полномасштабных средств автоматизации. Наиболее трудоемкими этапами ЖЦ ПС являются этапы анализа и проектирования, в процессе которых CASE-средства обеспечивают качество принимаемых технических решений и подготовку проектной документации. Графические средства моделирования предметной области позволяют разработчикам в наглядном виде изучать существующую ПС, перестраивать ее в соответствии с поставленными целями и имеющимися ограничениями.

Обычно к CASE-средствам относят любое программное средство, автоматизирующее ту или иную совокупность процессов жизненного цикла ПО, а также обладающее следующими основными характерными особенностями:

- мощные графические средства для описания и документирования ПС, обеспечивающие удобный интерфейс с разработчиком и развивающие его творческие возможности;
- интеграция отдельных компонентов CASE-средств, обеспечивающая управляемость процессом разработки ПС;
- использование специальным образом организованного хранилища проектных метаданных (репозитория).

Интегрированное CASE-средство (или комплекс средств, поддерживающих полный ЖЦ ПО) содержит следующие компоненты:

- репозиторий, являющийся основой CASE-средства, во многом обеспечивающий его эффективность, версионный контроль, синхронизацию поступления информации от разработчиков при групповой разработке, контроль метаданных на полноту и непротиворечивость;
- графические средства анализа и проектирования, обеспечивающие создание и редактирование иерархически связанных диаграмм (DFD, ERD и др.), образующих модели ПС;
- средства разработки приложений, в том числе генераторы кодов, IDE, средства отладки и т.п.;
- средства конфигурационного управления;
- средства документирования;
- средства тестирования;
- средства управления проектом;
- средства реинжиниринга.

2.2.2. Классификация CASE-средств

Все современные CASE-средства могут быть классифицированы в основном по типам и категориям. Сегодня российский рынок программного обеспечения располагает следующими наиболее развитыми CASE-средствами:

- Vantage Team Builder (Westmount I-CASE);
- Designer/2000;
- Silverrun;

- ERwin+BPwin;
- S-Designor;
- CASE.Аналитик.

Кроме того, на рынке постоянно появляются как новые для отечественных пользователей системы (например, CASE /4/0, PRO-IV, System Architect, Visible Analyst Workbench, EasyCASE), так и новые версии и модификации перечисленных систем.

Рассмотрим подробную классификацию CASE-средств по указанным признакам.

Классификация по типам отражает функциональную ориентацию CASE-средств на те или иные процессы ЖЦ. В основном она совпадает с компонентным составом CASE-средств и включает следующие основные типы:

- средства анализа (Upper CASE), предназначенные для построения и анализа моделей предметной области (Design/IDEF (Meta Software), BPwin (Logic Works));

- средства анализа и проектирования (Middle CASE), поддерживающие наиболее распространенные методологии проектирования; используют для создания проектных спецификаций (Vantage Team Builder (Cayenne), Designer/2000 (ORACLE), Silverrun (CSA), PRO-IV (McDonnell Douglas), CASE.Аналитик (МакроПроджект)). Выходом таких средств являются спецификации компонентов и интерфейсов системы, архитектуры системы, алгоритмов и структур данных;

- средства проектирования баз данных, обеспечивающие моделирование данных и генерацию схем баз данных (как правило, на языке SQL) для наиболее распространенных СУБД. К ним относятся ERwin (Logic Works), S-Designor (SDP) и DataBase Designer (ORACLE). Средства проектирования баз данных имеются также в составе CASE-средств Vantage Team Builder, Designer/2000, Silverrun и PRO-IV;

- средства разработки приложений. К ним относятся средства 4GL (Uniface (Compuware), JAM (JYACC), PowerBuilder (Sybase), Developer/2000 (ORACLE), New Era (Informix), SQL Windows (Gupta), Delphi (Borland) и др.) и генераторы кодов,

входящие в состав Vantage Team Builder, PRO-IV и частично – в Silverrun;

- средства реинжиниринга, обеспечивающие анализ программных кодов и схем баз данных и формирование на их основе различных моделей и проектных спецификаций;

- средства анализа схем БД и формирования ERD, которые входят в состав Vantage Team Builder, PRO-IV, Silverrun, Designer/2000, ERwin и S-Designor;

- средства анализа программных кодов – наибольшее распространение получили объектно-ориентированные CASE-средства, обеспечивающие реинжиниринг программ на языке C++ (Rational Rose (Rational Software), Object Team (Cayenne));

- средства планирования и управления проектом (SE Companion, Microsoft Project и др.);

- средства конфигурационного управления (PVCS (Intersolv));

- средства тестирования (Quality Works (Segue Software));

- средства документирования (SoDA (Rational Software)).

Классификация по категориям определяет степень интегрированности по выполняемым функциям и включает такие средства, как:

- отдельные локальные средства, решающие небольшие автономные задачи (tools);

- набор частично интегрированных средств, охватывающих большинство этапов жизненного цикла ПС (toolkit);

- полностью интегрированные средства, поддерживающие весь ЖЦ ПС и связанные общим репозиторием.

Помимо этого, CASE-средства можно классифицировать по следующим признакам:

- применяемым методологиям и моделям систем и БД: выделяют простые и гибридные CASE-средства;

- степени интегрированности с СУБД: выделяют слабо и глубоко интегрированные CASE-средства;

- доступным платформам: выделяют моноплатформенные и кроссплатформенные CASE-средства.

2.2.3. Факторы выбора CASE-средств

Выбор CASE-средств в программном процессе организации – это важное решение, которое требует тщательного анализа и оценивания нескольких факторов. Рассмотрим основные факторы, которые следует учесть при выборе средств автоматизации разработки ПС.

1. Цели и потребности организации. Первым и основным фактором является понимание целей и потребностей организации. Необходимо определить, какие функции и возможности нужны для поддержки процессов разработки и управления программным обеспечением. Это может быть моделирование и анализ системы, управление требованиями, автоматизация тестирования и т. д.

2. Функциональность CASE-средств. Каждое CASE-средство имеет свои особенности и функции. Важно провести анализ и сравнение различных средств автоматизации разработки программ для определения, какое из них лучше соответствует потребностям организации. Можно рассмотреть возможности создания диаграмм, генерации кода, анализа и отладки системы, поддержки специфических языков программирования и т. д.

3. Интеграция существующей инфраструктуры. При выборе CASE-средства необходимо учитывать совместимость и возможность интеграции с уже используемыми инструментами и платформами разработки. Это может включать интеграцию с IDE (Integrated Development Environment), системами управления версиями кода, инструментами для автоматической сборки и развертывания и т. д.

4. Стоимость и доступность. Фактор стоимости является важным при выборе CASE-средств. Сюда входит стоимость лицензий, обслуживания, обучения персонала и поддержки со стороны поставщика, а также доступность средств автоматизации разработки программ и их совместимость с операционными системами и аппаратным обеспечением, используемыми в организации.

5. Обучение и поддержка. При выборе CASE-средств необходимо обеспечить обучение сотрудников и поддержку со стороны поставщика. Важна доступность программных курсов и ресурсов для обучения, а также возможность получения технической поддержки в случае проблем или вопросов.

6. Надежность и стабильность поставщика, надежность и стабильность поставщика CASE-средства, длительность его работы на рынке и опыт в области разработки средств автоматизации разработки программ. Также важны отзывы от других пользователей.

Учитывая все эти факторы, организация может принять информированное решение о выборе подходящего CASE-средства, которое будет наиболее эффективно поддерживать и оптимизировать процессы разработки ПС.

2.2.4. Оценка и выбор CASE-средств

Модель процесса оценивания и выбора (рис. 2.32) описывает наиболее общую ситуацию, а также показывает зависимость между ними.

Оценивание и выбор могут выполняться независимо друг от друга или вместе, причем каждый из этих процессов требует применения определенных критериев.

Процесс может преследовать несколько целей, включая одну или более из следующих:

- оценивание нескольких CASE-средств и выбор одного или более из них;
- оценивание одного CASE-средства или более и сохранение результатов для последующего использования;
- выбор одного CASE-средства или более с использованием предыдущих оценок.

Входной информацией для процесса оценивания служат:

- цели, предположения и ограничения, которые могут уточняться в ходе процесса;

- потребности пользователей, отражающие количественные и качественные требования пользователей к CASE-средствам;
- критерии, определяющие набор параметров, в соответствии с которыми производится оценивание и принятие решения о выборе;
- формализованные результаты оценок одного средства или более;
- рекомендуемое решение (обычно либо решение о выборе, либо дальнейшее оценивание).



Рис. 2.32. Модель процесса оценки и выбора CASE-средства

Результаты оценивания могут включать результаты предыдущих оценок. При этом не следует забывать, что набор

критериев, использовавшихся при предыдущем оценивании, должен быть совместимым с текущим набором. Конкретный вариант реализации процесса (оценка и выбор, оценка для будущего выбора или выбор, основанный на предыдущих оценках) определяется перечисленными ранее целями.

Процесс оценки и / или выбора может быть начат только тогда, когда лицо, группа или организация полностью определила конкретные потребности и формализовала их в виде количественных и качественных требований в заданной предметной области.

Определение списка критериев основано на требованиях и включает:

- выбор основных критериев для использования из приведенного далее перечня;
- определение дополнительных критериев;
- определение области использования критериев (оценка, выбор или оба процесса);
- определение одной метрики или более для каждого критерия оценивания;
- назначение веса каждому критерию при выборе.

2.2.5. Критерии оценивания и выбора

Критерии формируют базис для процессов оценивания и выбора и могут принимать различные формы, в том числе:

- числовые меры в широком диапазоне значений, например, объем требуемой памяти;
- числовые меры в ограниченном диапазоне значений, например, простота освоения, выраженная в баллах от 1 до 5;
- двоичные меры (истина / ложь, да / нет), например, способность генерации документации в формате Postscript;
- меры, которые могут принимать одно или более из конечных множеств значений, например, платформы, для которых поддерживается CASE-средство.

Типичный процесс оценки и / или выбора может использовать набор критериев различных типов.

Структура набора критериев приведена на рис. 2.33. Каждый критерий должен быть выбран и адаптирован экспертом с учетом особенностей конкретного процесса. В большинстве случаев только некоторые из множества описанных далее критериев оказываются приемлемыми для использования. Выбор и уточнение набора используемых критериев является критическим шагом в процессе оценки и / или выбора.

Критерии первого класса предназначены для определения характеристик, выражающих функциональность CASE-средства. Они в свою очередь подразделяются на ряд групп и подгрупп.



Рис. 2.33. Структура набора критериев оценивания и выбора CASE-средств

1. Среда функционирования. Определяет набор процессов ЖЦ, которые поддерживают CASE-средство. Примерами таких процессов являются анализ требований, проектирование, реализация, тестирование и оценка, сопровождение, обеспечение качества, управление конфигурацией и управление проектом, причем они зависят от принятой пользователем модели ЖЦ.

2. Функции, ориентированные на фазы жизненного цикла. Ядро программного процесса можно разбить на 3 блока (рис. 2.34), которые обеспечивают прослеживание трассировки элементов на основных этапах жизненного цикла проекта.

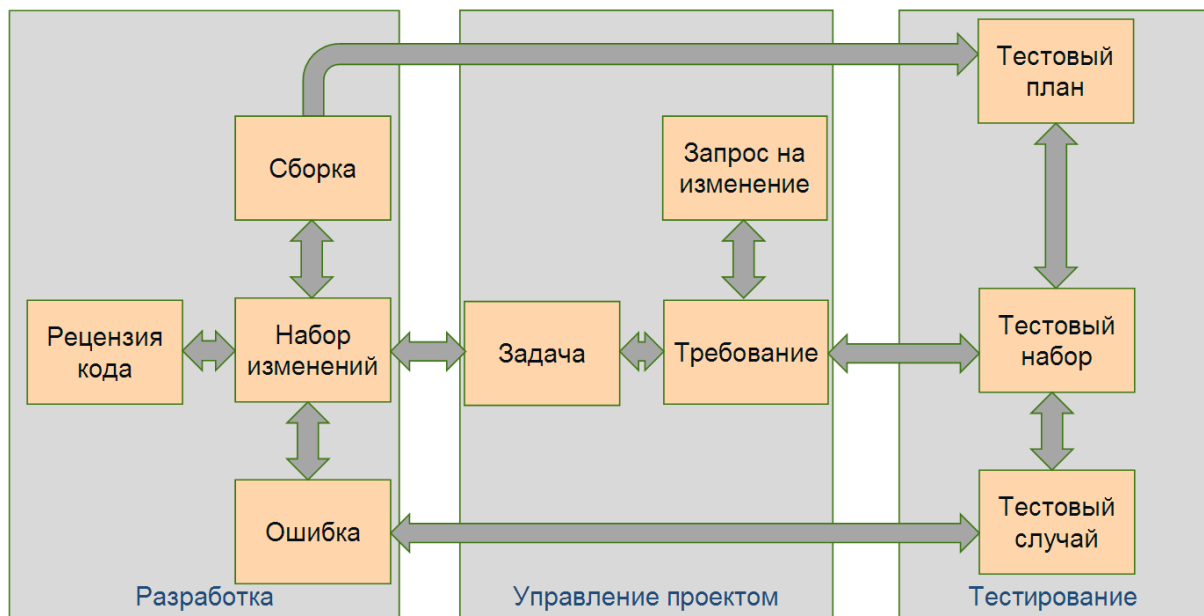


Рис. 2.34. Декомпозиция программного процесса по функциональному признаку

Разработка и тестирование – центральные основные этапы, которые базируются на корректно поставленном техническом задании и разработанных проектных решениях. Но необходимо ими ещё и управлять, поэтому есть организационный процесс ЖЦ – управление проектом.

В процессе разработки выполняется кодирование. Набор изменений сохраняется в репозитории. Согласно методологии DevOps должна производиться рецензия кода (со стороны коллег). На этапе тестирования могут быть выявлены ошибки, но

в конечном итоге результатом разработки является сборка, которая передается на этап тестирования, где формируется тестовый план. В соответствии с этим планом создается некоторый тестовый набор, состоящий из тестовых случаев, которые выявляют ошибки и передают их обратно в разработку.

Подобные действия реализуются на фазе «Управление проектом»: на основе задач формируется требование, запрос на изменение тоже может иметь место. В идеальном мире изменение требований должно сразу отображаться и на разработке, и на тестировании. Таким образом осуществляется трассировка элементов, что приводит к повышению качества продукта.

2.2.6. Технология внедрения CASE-средств

При выборе и внедрении CASE-средств организация должна учитывать ряд факторов, таких как интеграция существующей инфраструктуры, стоимость и доступность, обучение и поддержка со стороны поставщика, его надежность и стабильность. Важно определить цели и потребности организации, а также сравнить функциональность различных CASE-средств.

Выбор и внедрение подходящего CASE-средства может значительно повысить эффективность и качество программного процесса. Однако важно тщательно анализировать и оценивать преимущества, недостатки и факторы выбора в контексте конкретных потребностей организации. Правильное решение позволит достичь успеха в разработке программного обеспечения и достижении бизнес-целей.

Внедрение CASE-средств (в широком смысле) – процесс, направленный от оценки первоначальных потребностей до полномасштабного использования CASE-средств в различных подразделениях организации-пользователя.

Процесс внедрения CASE-средств состоит из следующих этапов:

- определение потребностей в CASE-средствах;
- оценка и выбор CASE-средств;

- выполнение пилотного проекта;
- практическое внедрение CASE-средств.

Процесс успешного внедрения CASE-средств не ограничивается только их использованием. На самом деле он охватывает планирование и реализацию множества технических, организационных, структурных процессов, изменений в общей культуре организации и основан на четком понимании возможностей CASE-средств.

На способ внедрения CASE-средств может повлиять специфика конкретной ситуации. Например, если заказчик предпочитает конкретное средство, или оно оговаривается требованиями контракта, этапы внедрения должны соответствовать такому predetermined выбору. В иных ситуациях особенности заказчика и / или проекта могут привести к внесению соответствующих корректив в процесс внедрения CASE-средства. К таким особенностям относятся:

- относительная простота или сложность CASE-средства;
- степень согласованности или конфликтности с существующими в организации процессами;
- требуемая степень интеграции с другими средствами;
- опыт и квалификация пользователей.

Рассмотрим детальнее процедуру определения потребностей в CASE-средствах. Данный этап (рис. 2.35) включает достижение понимания потребностей организации и технологии последующего процесса внедрения CASE-средств. Он должен привести к выделению тех областей деятельности организации, в которых применение CASE-средств может принести реальную пользу. Результатом данного этапа является документ, определяющий стратегию внедрения CASE-средств.

Успешное внедрение CASE-средств должно обеспечить заказчику такие выгоды:

- высокий уровень технологической поддержки процессов разработки и сопровождения ПС;
- положительное воздействие на некоторые или все из следующих факторов: производительность, качество продукции, соблюдение стандартов, документирование;

– приемлемый уровень отдачи от инвестиций в CASE-средства.

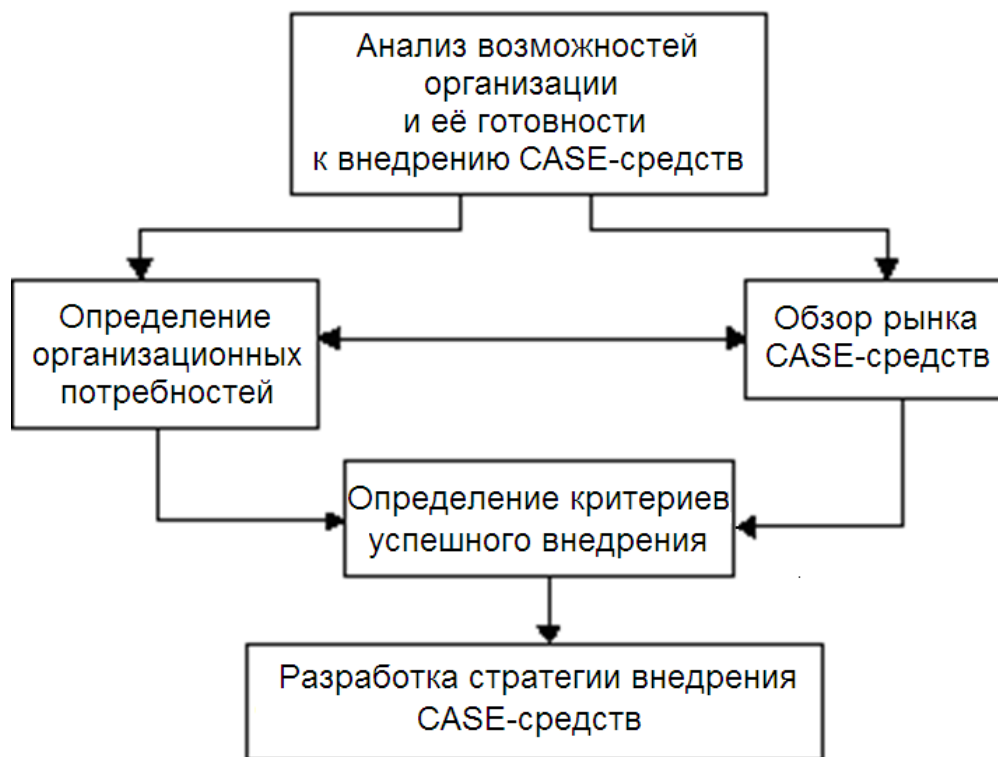


Рис. 2.35. Определение потребностей в CASE-средствах

Использование CASE-средств в разработке ПС может значительно ускорить процесс работы и повысить эффективность, освободив разработчиков от рутинных задач. Однако перед выбором и внедрением CASE-средств необходимо тщательно анализировать их потребности и учитывать специфику организации. Несмотря на некоторые недостатки, современные CASE-средства постоянно улучшаются и совершенствуются. Чтобы успешно внедрить CASE-средства, нужно тщательно оценить их преимущества и недостатки в контексте конкретных потребностей организации.

2.2.7. Преимущества использования CASE-средств

Использование CASE-средств имеет некоторые преимущества.

1. Увеличение производительности разработчиков. Это одно из ключевых преимуществ использования средств автоматизации разработки ПО. CASE-средства позволяют автоматизировать рутинные задачи, такие как генерация кода, проверка синтаксиса, отладка и документирование. Это существенно увеличивает производительность и позволяет выполнять работу более эффективно.

2. Улучшение качества ПО. С помощью средств автоматизации разработки программ можно создавать графические модели, диаграммы и схемы, которые помогают выявить потенциальные проблемы и ошибки на ранних стадиях разработки. Это способствует повышению качества программного обеспечения, снижению количества дефектов и обеспечению большей надежности и стабильности системы.

3. Сокращение затрат на разработку ПО. CASE-средства автоматизируют ряд процессов, таких как генерация кода, тестирование и документирование, что снижает количество ошибок и улучшает эффективность разработки. Это позволяет компаниям сократить расходы на разработку программного обеспечения и сосредоточиться на более важных аспектах бизнеса (инновации, улучшение конкурентоспособности и др.).

4. Упрощение сопровождения ПО. CASE-средства предоставляют возможность создания и поддержки документации, что облегчает понимание структуры и функциональности системы. Они также предоставляют инструменты для отслеживания изменений и управления версиями ПО, что значительно упрощает задачи сопровождения, обновления и модификации системы в будущем.

Таким образом, преимущества использования CASE-средств включают повышение эффективности и качества разработки, улучшение коммуникации между участниками проекта, автоматизацию рутины и сокращение времени разработки. Они позволяют разработчикам визуализировать и анализировать системы, создавать модели и диаграммы, генерировать код и документацию, а также управлять требованиями и версиями, автоматизировать рутинные задачи и ускорить процесс работы,

что в свою очередь позволяет разработчикам сосредоточиться на более сложных аспектах. Всё это в совокупности приводит к повышению эффективности и общей производительности, улучшению качества ПО и снижению затрат на разработку и сопровождение ПС.

2.2.8. Недостатки использования CASE-средств

Несмотря на множество преимуществ, использование CASE-средств в программном процессе сопряжено с определенными недостатками. Понимание этих недостатков является важным аспектом для принятия информированных решений о внедрении CASE-средств в организации.

1. Сложность внедрения и обучения персонала. Это один из основных недостатков. CASE-средства обычно имеют сложный интерфейс и требуют дополнительного времени и усилий для освоения их функциональности. Внедрение средств автоматизации разработки программ также может потребовать изменений в рабочих процессах и структуре организации, что может вызвать сопротивление со стороны сотрудников. Необходимо уделить должное внимание обучению и поддержке пользователей при внедрении средств автоматизации разработки программ, чтобы они смогли эффективно использовать их потенциал.

2. Ограничения функциональности. CASE-средства могут иметь ограниченную функциональность в некоторых областях разработки ПО. Некоторые типы проектов или специфические требования могут выходить за пределы возможностей предлагаемых инструментов CASE-средств. Это ограничение может привести к необходимости дополнительных настроек, настраиваемых расширений или даже использования альтернативных инструментов для определенных аспектов разработки. Важно тщательно оценить функциональные возможности CASE-средств и их соответствие конкретным требованиям проекта.

3. Зависимость от поставщика. При использовании CASE-средств возникает зависимость от поставщика данного CASE-средства. Компании, разрабатывающие CASE-средства, могут изменять свою стратегию разработки или даже прекратить поддержку и развитие продукта. Это может привести к необходимости перехода на другое средство автоматизации разработки или использованию альтернативных решений, что требует дополнительных затрат времени, ресурсов и усилий. Важно учитывать долгосрочную перспективу при выборе CASE-средств и оценивании их стабильности и поддержки со стороны поставщика.

4. Интеграция с существующими системами. Интеграция CASE-средств с уже существующими системами и инфраструктурой организации может быть вызовом. CASE-средства могут требовать совместимости с определенными платформами, языками программирования или базами данных, что потребует дополнительных усилий для настройки и интеграции. Кроме того, интеграция CASE-средств может потребовать согласования с другими командами и структурами организации, что затянет процесс и усложнит его выполнение. Важно учитывать факторы интеграции при выборе и внедрении CASE-средств.

5. Стоимость и лицензирование. Использование CASE-средств может быть связано с дополнительными затратами на приобретение лицензий и поддержку софта. CASE-средства имеют различные модели лицензирования, такие как ежегодные платежи, платы за использование определенного количества пользователей или расширенные функциональные возможности. Это может привести к значительным затратам для организации, особенно для больших команд или компаний. Важно тщательно оценить стоимость и модель лицензирования CASE-средств перед их выбором и внедрением.

ЗАКЛЮЧЕНИЕ

Материал данного учебного пособия сформирован на основе бакалаврских курсов «Программная инженерия» и «Технологии проектирования программного обеспечения», а также магистерского курса «Технологии проектирования и сопровождения программных систем», читаемых авторами на протяжении последних лет на факультете КТиПМ КубГУ. Сочетание теоретической базы по данным дисциплинам с опытом авторов в применении изложенных знаний в учебном процессе и на производстве позволило создать учебное пособие, ориентирующее студентов на реализацию полного жизненного цикла программной системы, включая качественную реализацию всех её обеспечивающих компонентов, конфигурирования в единый хорошо документированный программно-аппаратный комплекс и внедрение в сферу применения.

Глава 1 пособия освещает полный жизненный цикл программной системы от возникновения идеи ее создания до введения в эксплуатацию готового продукта и составлена таким образом, чтобы обеспечить учебно-методическую поддержку обучающемуся в любом из аспектов изучения: проектирования, разработки, тестирования, внедрения и сопровождения ПС.

Существенным дополнением к этому материалу стало изложенное в главе 2 подробное рассмотрение инструментальных средств проектирования ПС, в частности, языка визуального моделирования программного обеспечения UML. Широкое использование UML в программном процессе во многом является залогом успеха проекта, что подтверждено более чем 40 годами общемировой практики его применения в программной инженерии и при разработке компьютерных информационных систем разной сложности. Универсальная нотация UML позволяет использовать его на всех этапах жизненного цикла ПС, включая начальные этапы обследования объекта информатизации и постановки задачи. Кроме того, использование для автоматизированного проектирования ПС инструментального ПО

типа «CASE» может существенно сократить затраты на проектирование.

Учебное пособие будет полезно не только студентам, но и специалистам разных областей, интересующихся методикой создания и исследования информационных и программных систем.

РЕКОМЕНДУЕМАЯ ЛИТЕРАТУРА

- Бабич А. В. Введение в UML: учеб. пособие. М., 2022.
- Богданова Е.Н., Бедердинова О.И. Комплексный анализ и моделирование бизнес-процессов производственного предприятия: учеб. пособие. М., 2022.
- Босуэлл Д., Фаучер Т. Читаемый код или Программирование как искусство: монография. СПб., 2012.
- Буч Г. Объектно-ориентированный анализ и проектирование с примерами приложений на C++. М., 1999.
- Гагарина Л.Г., Кокорева Е.В., Виснадул Б.Д. Программная инженерия: учеб. пособие; под ред. Л.Г. Гагариной. М., 2012.
- Герасимов Б.Н. Реинжиниринг процессов организации. М., 2020.
- Горюнова Н.Д., Ковылкин Д.Ю., Никитина Л.Н. Управление бизнес-процессами: учеб. пособие. СПб., 2019.
- ГОСТ 19.701–90 (ИСО 5807-85). Схемы алгоритмов, программ, данных и систем. М., 1990.
- ГОСТ 34.601–90. Автоматизированные системы. Стадии создания. Информационная технология. Комплекс стандартов и руководящих документов на автоматизированные системы. М., 1991.
- ГОСТ 34.602–89. Техническое задание на создание автоматизированной системы. Информационная технология. Комплекс стандартов и руководящих документов на АС. М., 1991.
- ГОСТ Р ИСО/МЭК 12207–99. Государственный стандарт Российской Федерации. Информационная технология. Процессы жизненного цикла программных средств. М., 1999.
- ГОСТ Р ИСО/МЭК 15504-1-2009. Информационные технологии. Оценка процессов. Ч. 1. Концепция и словарь. М., 2010.
- Ахен Д., Клауз А., Тернер Р. CMMI: Комплексный подход к совершенствованию процессов. Практическое введение в модель. М., 2005.
- Елиферов В.Г., Репин В.В. Бизнес-процессы. Регламентация и управление: учеб. пособие. М., 2012.

Ехлаков Ю.П. Основы программной инженерии: учеб. пособие. Томск, 2019.

Ехлаков Ю.П. Планирование и организация вывода программного продукта на рынок: методические указания по лабораторным работам и организации самостоятельной работы. Томск, 2017.

Зариковская Н. В. Технология программирования: учеб. пособие. Томск, 2018.

Кугаевских А.В. Проектирование информационных систем. Системная и бизнес-аналитика: учеб. пособие. Новосибирск, 2018.

Леоненков А.В. Самоучитель UML. СПб., 2004.

Леоненков А.В. Объектно-ориентированный анализ и проектирование с использованием UML и IBM Rational Rose: учеб. пособие. М., 2020.

Липаев В.В. Программная инженерия сложных заказных программных продуктов: учеб. пособие. М., 2014.

Мартин Р. Чистый код: создание, анализ и рефакторинг. СПб., 2013.

Мартишин С.А., Симонов В.Л., Храпченко М.В. Проектирование и реализация баз данных в СУБД MySQL с использованием MySQL Workbench. Методы и средства проектирования информационных систем и технологий. Инструментальные средства информационных систем: учеб. пособие. М., 2014.

Мостовой Я.А. Управление программными проектами: учеб. пособие. Самара, 2016.

Муртазина М. Ш. Управление проектами в сфере информационных технологий: учеб. пособие. Новосибирск, 2022.

Носова Л.С. Case-технологии и язык UML: учеб. пособие. Челябинск; Саратов, 2019.

Полетайкин А.Н., Добровольская Н.Ю. Методология и технология разработки программных систем: методы и модели программной инженерии: учеб. пособие. Краснодар, 2025.

Полетайкин А.Н. Социальные и экономические информационные системы. Законы функционирования и принципы построения: учеб. пособие. Новосибирск, 2016.

Полетайкин А.Н. Учебно-методическое пособие по выполнению лабораторных работ по дисциплине «Программная инженерия». Ч. 1. Реализация жизненного цикла программного обеспечения. Новосибирск, 2016.

Суханов М.Б. Программная инженерия: учеб. пособие. СПб., 2018.

Шеер А.В. Индустрия 4.0: от прорывной бизнес-модели к автоматизации бизнес-процессов. М., 2020.

Шлаев Д.В. Моделирование бизнес-процессов с использованием нотаций UML: учеб. пособие. Ставрополь, 2023.

ПРИЛОЖЕНИЯ

Пример характеристики объекта информатизации

Объектом информатизации является ПАО Банк «ФК Открытие». Банк «Открытие» – крупнейший частный банк в России и четвертый по размеру активов среди всех российских банковских групп. Работает на финансовом рынке с 1993 г.¹

Активы банка и его дочерних компаний по международным стандартам финансовой отчетности (МСФО) на 30 июня 2016 г. составили 3 087,8 млрд р., собственный капитал – 217,8 млрд р.

«Открытие» – универсальный коммерческий банк с устойчивой диверсифицированной структурой бизнеса и качественным управлением капиталом.

Банк развивает следующие ключевые направления бизнеса:

- корпоративный;
- инвестиционный;
- розничный;
- малый и средний бизнес;
- приват банкинг.

Особое внимание «Открытие» уделяет высокотехнологичным сервисам. Так, в рамках проекта «Рокетбанк» предлагается полностью дистанционный сервис для физических лиц, а в рамках проекта «Точка» – полностью дистанционное обслуживание для предпринимателей.

Банк «Открытие» был сформирован в результате интеграции более чем десяти банков различного масштаба, в том числе таких крупных федеральных, как НОМОС-БАНК, Ханты-Мансийский банк и банк «Петрокоммерц».

Банк «Открытие» входит в список десяти системообразующих кредитных организаций, утвержденный Центральным банком, а также в список крупнейших компаний мира FORBES Global 2000.

¹ Банк «Финансовая корпорация Открытие». URL: <https://www.open.ru>

Надежность банка подтверждена рейтингами международных агентств S&P Global (BB-) и Moody's Investors Service (Ba3).

Клиентская база объединенного банка насчитывает свыше 30 000 корпоративных клиентов, 165 000 клиентов малого бизнеса и около 3,2 млн физических лиц, в том числе премиальных клиентов. 470 отделений банка различного формата расположены в 53 экономически значимых регионах России. Значительная часть бизнеса сосредоточена в Москве, Санкт-Петербурге, Тюменской области, Екатеринбурге, Новосибирской области, Хабаровском крае, Волгоградской области.

Ключевым акционером банка «Открытие» является «Открытие Холдинг», который владеет 64,7% голосующих акций (см. рисунок). Бенефициарами «Открытие Холдинг» являются: Вадим Беляев, Группа «ИФД Капиталь», Банк «ВТБ», Группа «ИСТ», Рубен Аганбегян, Александр Мамут. Бумаги банка также находятся в свободном обращении на Московской бирже.



Структура ОАО «Открытие Холдинг»

Главой Наблюдательного совета банка «Открытие» является Дмитрий Ромаев, председателем Правления – Евгений Данкевич.

Банк «Открытие» активно участвует в реализации социально значимых проектов, поддерживая такие направления, как наука (Политехнический музей), образование (Московский государственный университет им. Ломоносова, Европейский университет в Санкт-Петербурге), культура (Малый театр, сотрудничество с автором мультфильма «Ёжик в тумане» Юрием Норштейном), здравоохранение (Фонд помощи хосписам «Вера»), спорт (стратегическое партнерство с ФК «Спартак-Москва», Югорский лыжный марафон).

Для информатизации было выбрано отделение банка в г. Новосибирске, располагающееся по адресу: 630102, г. Новосибирск, ул. Кирова, д. 44.

Контактный телефон: (383) 325-10-85.

Управляющий филиалом: Ермилов Валерий Николаевич.

Главный бухгалтер: Жданов Алексей Владимирович.

Филиал обслуживает как частные, так и юридические лица.

Услуги частным лицам включают:

- срочные переводы по системе «Western Union» в рублях (по России) и в валюте в любую точку мира;
- прием коммунальных платежей;
- оплата услуг операторов мобильной связи;
- оплата кредитов, полученных в других банках;
- автокредитование;
- кредиты по пластиковым картам;
- открытие и ведение текущих рублёвых и валютных счетов;
- предоставление индивидуальных банковских сейфов;
- оформление и обслуживание пластиковых карт;
- денежные вклады;
- ипотечное кредитование;
- инвестиции;
- страхование;
- пенсионный калькулятор;
- сейфовые ячейки;

- операции с иностранной валютой и чеками.
- Услуги корпоративным клиентам включают:
- расчётно-кассовое обслуживание, в том числе по системе «Банк-Клиент»;
 - широкий выбор кредитных продуктов (кредитные линии, овердрафт, вексельные кредиты, лизинг);
 - оперативное проведение валютно-финансовых операций;
 - вклады и депозиты для корпоративных клиентов;
 - организация процедуры выплаты зарплаты с помощью пластиковых карт;
 - оформление пластиковых корпоративных карт международных платёжных систем;
 - консалтинговые услуги;
 - дистанционное обслуживание;
 - инвестиции.

Процессами информатизации являются: расчёт ежемесячного платежа для физических лиц и оценка их кредитоспособности при ипотечном кредитовании.

Ипотечное кредитование – долгосрочный кредит, предоставляемый юридическому или физическому лицу банками под залог недвижимости (земли, производственных и жилых зданий, помещений, сооружений). Самый распространенный вариант использования ипотеки в России – это покупка физическим лицом квартиры в кредит. Закладывается при этом, как правило, вновь покупаемое жилье, хотя можно заложить и уже имеющуюся в собственности квартиру².

Цель информатизируемых процессов: рассчитать ежемесячный платёж и проверить кредитоспособность заёмщика.

Ограничения на операции: возраст клиента должен быть не менее 18 лет и не более 65 лет на момент выплаты последнего платежа.

² IPOhelp. URL: <http://www.ipohelp.ru/manual.html>

Пример технического задания на разработку мобильного приложения

Основные разделы ТЗ

1. Словарь терминов. Описание основных объектов и сокращений.
2. Назначение разработки. Отражает суть создаваемого приложения или комплекса.
3. Технические условия / требования. Описывает рамки создаваемого приложения.
4. Логика работы.
5. Интерфейс приложения.
6. Функциональные требования.

Словарь терминов

Термин	Описание
Мобильное приложение	Мобильное приложение – это специально разработанное приложение под конкретную мобильную платформу (iOS, Android, Windows Phone)
Нативное мобильное приложение	Приложение можно назвать «родным» для операционных систем – Android, IOS, WinPhone. Такие мобильные приложения пишутся на языках программирования, утвержденных разработчиками программного обеспечения под каждую конкретную платформу, а потому органично встраиваются в сами операционные системы. Приложения загружаются через магазины приложений (App Store, Google Play и т.д.) и соответствуют требованиям этих магазинов
Администратор	Лицо, осуществляющее от имени Заказчика информационную поддержку мобильного приложения
Дизайн мобильного приложения	Уникальные для конкретного мобильного приложения структура, графическое оформление и способы представления информации

Назначение разработки

Данный раздел должен донести до исполнителя цель создания приложения (на примере разработки мобильного приложения туристического сервиса).

Назначение мобильного приложения

1. Предоставление полной информации о местах курортной зоны.
2. Возможность осуществлять бронирование развлечений (экскурсий, снаряжения и т.д.).
3. Возможность осуществлять бронирование проживания (тур, отели, дома отдыха, курорты и т.д.).
4. Возможность осуществлять бронирование транспорта (авиабилетов, ж/д билетов, такси, автобусов).
5. Возможность осуществлять бронирование питания (столик в ресторане, вызов кейтеринг и т.д.).

Цель создания мобильного приложения:

- агрегирование сервисов курортной зоны России в одной месте;
- возможность бронирования путешествий «в один клик»;
- обмен сообщениями и отзывами между пользователями.

Технические требования

Рассмотрим технические условия для мобильного приложения.

- Платформа работы приложения: IOS / Android.
- Приложение IOS:
 - совместимость с ОС: IOS 8.0 и старше;
 - поддержка устройств: iPhone 5S+, iPad2+, iPad Air+, iPad mini +;
 - верстка iPhone Книжная: Да;
 - верстка iPhone Альбомная: Адаптивная от книжной.
- Приложения Android:
 - совместимость с Android: Android 4.4. и старше;
 - верстка телефон книжная: Да;
 - верстка телефон альбомная: Да;
 - верстка планшет Книжная: Адаптивная от телефона;

- верстка планшет Альбомная: Адаптивная от телефона.
- Сервер:
- совместимый вебхостинг на базе Apache2 + PHP5 + MySQL.

В зависимости от условий – описание сервера может описывать требования по обеспечению необходимой нагрузки.

Если приложения создаются под определенные требования (безопасность, секретность и т.д.), то они также описываются в произвольной форме.

Логика работы

Раздел логики работы является ключевым для всего технического задания. Здесь описываются основные алгоритмы получения и преобразования информации. В процессе описания основных бизнес-процессов для приложения, следует упомянуть и связанные бизнес-процессы компании. Подробное изложение позволяет получить обобщенное представление и цели постановки задачи на разработку приложения.

При оформлении раздела рекомендуется использовать графические блок-схемы, наглядное описание, какие-либо снимки и т.д., чтобы у программиста собралось наглядное описание процесса работы приложения. В тестовом описании желательно системное мышление и причинно-следственные связи.

Каждый экран должен быть подробно описан, как по назначению, так и по функционалу (рис. 1). Сначала описывается задача экрана, условия перехода и возможные ограничения. Затем переходим к функционалу экрана, ссылаясь на макет и обозначенные на нем элементы интерфейса.

Интерфейс приложения

Интерфейс должен быть описан и показан в графическом виде. Если в приложении должны присутствовать элементы атрибутики фирменного стиля, то подобные медиаматериалы должны передаваться на цифровом носителе или размещены на *файлообменнике* с указанием прямой ссылки на файл. Корпоративные цвета описываются в формате HEX (например, #4177ab).

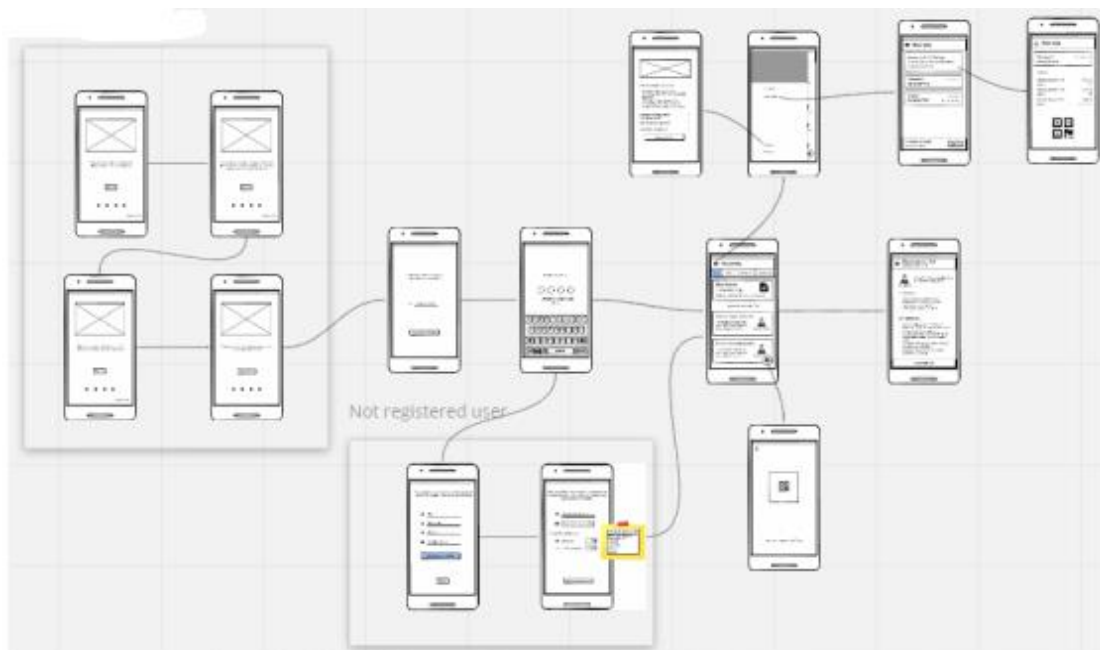


Рис. 1. Логика работы

Требования к дизайну мобильного приложения.

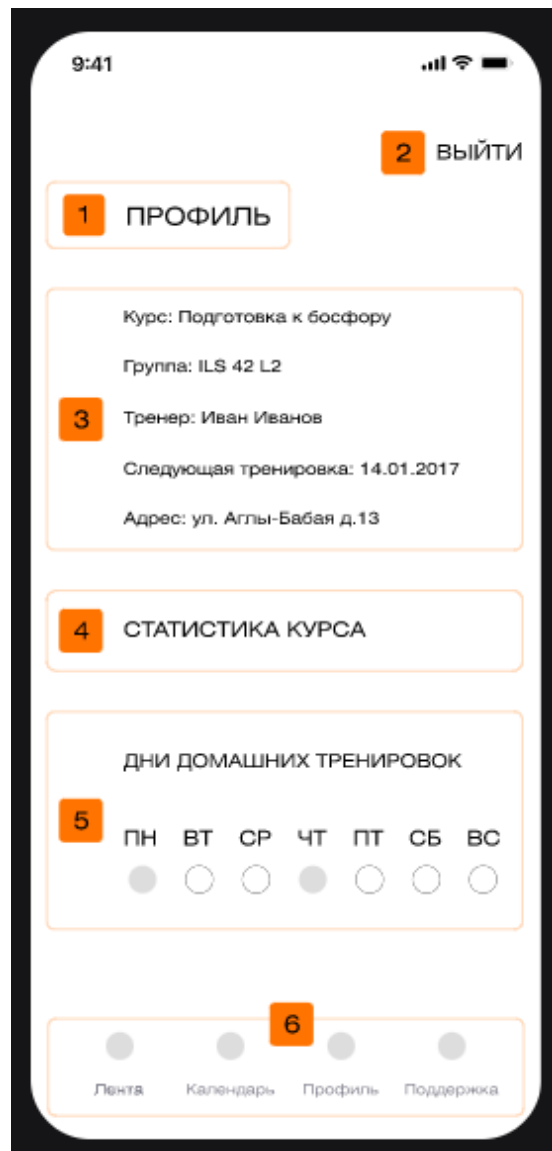
При разработке мобильного приложения должны быть использованы преимущественно светлые и контрастные цветовые решения. Оформление должно быть разработано в достаточно консервативном ключе.

На страницах не должно быть большого объема текста.

В дизайне мобильного приложения не должны присутствовать:

- мелькающие баннеры;
- много сливающегося текста;
- тёмные и агрессивные цветовые сочетания и графические решения.

Желательно приложить блок-схему переходов между экранами с описанием переходов. При необходимости включения эффектов анимации они должны быть описаны четко, техническими терминами. Если уже подготовлены макеты экранов, то можно включить их в эту схему или сослаться на название конкретного макета. В случаях, когда на макете присутствует много элементов управления (кнопок), то желательно явно прорисовывать от какой кнопки к какому экрану должен производиться переход (рис. 2).



Наименование элемента	Описание
Заголовок экрана	Содержит название экрана «Профиль»
Кнопка «Выйти»	При нажатии на кнопку, происходит logout
Блок с описанием курса	Текстовое описание курса
Кнопка «Дни домашних тренировок»	При нажатии на кнопку переход к экрану «Статистика курса»
Блок «Дни домашних тренировок»	Отображает, в какие дни есть тренировки
Навигационные кнопки	При нажатии на кнопки происходит переход к одноименным страницам

Рис. 2. Описание экрана

Описание интерфейса должно точно соответствовать логике работы.

Порядок утверждения дизайн-концепции.

Под дизайн-концепцией понимается вариант оформления главной страницы и графическая оболочка внутренних страниц, демонстрирующие общее визуальное (композиционное, цветовое, шрифтовое, навигационное) решение основных страниц мобильного приложения. Дизайн-концепция представляется в виде файла (нескольких файлов) в растровом формате или в распечатке по согласованию сторон.

Если представленная Исполнителем дизайн-концепция удовлетворяет Заказчика, он должен утвердить ее в течение пяти рабочих дней с момента представления. При этом он может направить Исполнителю список частных доработок, не затрагивающих общую структуру страниц и их стилевое решение. Указанные доработки производятся параллельно с разработкой программных модулей мобильного приложения. Внесение изменений в дизайн-концепцию после ее приемки допускается только по дополнительному соглашению сторон. Если представленная концепция не удовлетворяет требованиям Заказчика, последний предоставляет мотивированный отказ от принятия концепции с указанием деталей, которые послужили препятствием для принятия концепции и более четкой формулировкой требований.

В этом случае Исполнитель разрабатывает второй вариант дизайн-концепции (дорабатывает, вносит изменения). Обязательства по разработке второго варианта дизайн-концепции Исполнитель принимает только после согласования и подписания дополнительного соглашения о продлении этапа разработки дизайн-концепции на срок не менее пяти рабочих дней. Дополнительные (третий и последующие) варианты разрабатываются Исполнителем за отдельную плату на основании дополнительных соглашений.

Сервер API.

Данный раздел описывается в том случае, если приложение сетевое.

Описание сервера API должно включать состав протокола, а также методы взаимодействия с системами, которые не были описаны в разделе «Логика работы».

Минимальное описание с применением формата JSON в архитектуре REST:

Авторизация

URL: *https://site.ru/api/login*

Метод передачи входных параметров : POST

username – логин пользователя

password – md5-хэш контантенации логина и пароля

autologin – автовход (значение null/on)

Формат выходных данных : JSON

Пример

```
{«result»:»succefull»,»session»:»12345678901234",»use  
r_id»:12344}
```

где

result – вид ответа

succefull – удачный вход

login_error – логин не найден

password_error – ошибка пароля

locked – пользователь заблокирован

try_error – превышено количество попыток входа за 10 минут

session – идентификатор сессии пользователя

user_id – идентификатор пользователя

Подобное описание должно сопровождаться на каждую функцию сервера API. Помимо описания функций, должно сопровождаться общим блоком ошибок работы, например: сервер данных недоступен, ошибка формата API, бан за превышение квоты, ошибка сессии и т.п.

Функциональные требования (на примере разработки мобильного приложения туристического сервиса).

Классы пользователей

1. Гость – неавторизованный пользователь, обладает правами:

– регистрация в мобильном приложении;

- авторизация: ввод аутентификационных данных;
- авторизованный пользователь обладает правами:
 - а) статические разделы – просмотр;
 - б) разделы новостей – просмотр;
 - с) новости – просмотр;
 - г) раздел бронирование билетов – просмотр;
- подписка на рассылки и уведомления;
- личный кабинет: информация о пользователе – просмотр, редактирование собственной информации; статистика писем – просмотр собственных писем; список рассылок и уведомлений – просмотр, редактирование, удаление.

2. Администратор – пользователь, авторизованный в интерфейсе мобильного приложения (полный доступ ко всем функциональным возможностям администрирования мобильного приложения):

- статические разделы – просмотр, добавление, редактирование, удаление;
- разделы новостей – просмотр, добавление, редактирование, удаление;
- новости – просмотр, добавление, редактирование, удаление;
- статьи – просмотр, добавление, редактирование, удаление;
- разделы – просмотр, добавление, редактирование, удаление.

Требования к представлению мобильного приложения.

Требования к представлению главной страницы мобильного приложения.

Главная страница мобильного приложения должна содержать графическую часть, навигационное меню сайта, а также контентную область для того, чтобы посетитель мобильного приложения с первой страницы мог получить вводную информацию об актуальных развлечениях, транспорте, питании и путевки, а также ознакомиться с последними новостями.

В нижней части экрана должно располагаться горизонтальное меню и включать разделы: Транспорт, Питание, Развлечение, Проживание.

Контентная часть главной странице мобильного приложения должна включать:

- в верхней части поиск по всем страницам мобильного приложения;
- слайдер на всю ширину экрана, который будет включать картинки;
- вкладки: транспорт, развлечение, питание, путевки, новости, фото и видеогалерея, форум;
- недавно просмотренные вкладки;
- самые популярные направления.

Требования к отображению раздела «Проживание».

При переходе пользователя в раздел «Проживание» в верхней части экрана:

- должен присутствовать поиск (в котором пользователь может задать город, в который он хочет поехать);
- ниже должна быть возможность выбрать дату заезда и дату отъезда;
- рядом должна быть опция выбора количества взрослых, количества номеров и количества детей;
- разделы с отелями, хостелами, домами, гостиницами;
- в остальной части экрана слева должна отображаться фотография места, рядом с правой стороны должны быть написано название места, количество звездочек, отзывы, соотношение цены и качества, спецпредложение;
- с правой стороны должна отображаться цена, агент (который предоставил информацию) и кнопка «посмотреть».
- Требования к внутренним страницам раздела «Проживание»:
 - стрелка, чтобы вернуться на страницу разделов. Должна присутствовать опция «поделиться с друзьями» и «добавить в избранное»;

- во всю ширину должен располагаться слайдер с фотографиями данного места с правой стороны должна располагаться кнопка «поделиться» и «сохранить в закладки»;
- название места, количество звездочек, количество отзывов, выбор даты заезда и отъезда (с выбором дат на карте);
- ниже должна быть написана цена и спецпредложение. Чуть ниже должна присутствовать информация об отеле (хостеле, дома отдыхе и т.д.).

Авторизация.

Пользователи могут авторизоваться в мобильном приложении, в форме авторизации должны присутствовать:

- текстовое поле для ввода логина пользователя;
- кнопка отправки формы.

Данные для доступа (авторизации):

- логин – адрес электронной почты пользователя;
- пароль – строка, содержащая 8 символов, состоящая из A-z, 0-9.

Ниже формы располагается ссылка: «Забыли пароль».

Форма «Забыли пароль» содержит поля: Email адрес пользователя, указанный при регистрации. При неудачной попытке авторизации – появляется приглашение для повторной попытки авторизоваться с формой авторизации.

Требования к видам обеспечения.

1. Требования к информационному обеспечению.

Требования к хранению данных.

Все данные мобильного приложения должны храниться в структурированном виде под управлением реляционной СУБД. Исключения составляют файлы данных, предназначенные для просмотра и скачивания (изображения, видео, документы и т.п.). Такие файлы сохраняются в файловой системе, а в БД размещаются ссылки на них.

2. Требования к языкам программирования.

Для реализации статических страниц и шаблонов должны использоваться языки HTML 4.0 и CSS. Исходный код должен разрабатываться в соответствии со стандартами W3C (HTML

4.0). Для реализации интерактивных элементов клиентской части должны использоваться языки JavaScript и DHTML.

Для разработки мобильного приложения должен быть использован ReactJS.

3. Требования к иллюстрациям.

Все рисунки и фото объемом более 1 kb (кроме элементов дизайна страницы) должны быть выполнены с замещающим текстом. Все рисунки должны быть в формате gif или jpg.

4. Требования к программному обеспечению.

Серверная часть:

- операционная система семейства Unix (Linux, FreeBSD и т.д.);
- веб-сервер Apache 1.3.18 и выше;
- Nginx, модуль mod_accel для Apache;
- набор библиотек и утилит ffmpeg;
- PHP 4.2.0 и выше (должен быть собран как модуль Apache);
- СУБД MySQL 4.1.14 и выше (предпочтительно: поддержка формата InnoDB);
- модули PHP: Mcrypt, FTP, ffmpeg-php;
- библиотеки PHP: Smarty, GeoIP;
- возможность доступа к localhost по FTP протоколу;
- два пользователя БД.

Клиентская часть:

- Adobe Flash Player версии 9 и выше. Мобильное приложение должно быть работоспособно (информация должна быть доступна).

5. Требования к техническому обеспечению.

Серверная часть:

- компьютер с процессором Xeon 4 ядерный;
- ГГц (рекомендуется от 3 ГГц);
- оперативная память 4 Гб;
- место на жестком диске от 1 Гб.

Точные технические характеристики сервера будут уточнены после завершения системы и обширного тестирования всех модулей.

Клиентская часть:

- смартфон с процессором i3 ГГц (рекомендуется от 1,5 ГГц);

- оперативная память 256 Мб (рекомендуется от 512 Мб).

6. Порядок предоставления дистрибутива.

По окончании разработки Исполнитель должен предоставить Заказчику дистрибутив системы в составе:

- архив с исходными кодами всех программных модулей и разделов мобильного приложения;

- дампы проектной базы данных с актуальной информацией.

Дистрибутив предоставляется на CD-диске в виде файлового архива.

7. Дополнительные требования.

Требования к производительности.

Работа любого скрипта не должна превышать 60 с при условии нагрузки на сервер не более 500000 обращений к страницам портала в сутки.

Требования к безопасности.

Требуется защитить исходный код общей части мобильного приложения. Не должно быть возможности считать php-код скриптов. Требуется разграничение доступа.

Пароли пользователей хранятся в зашифрованном виде. Перехват данных на уровне протокола tcp возможен.

На уровне СУБД должно быть реализовано разграничение доступа к данным в БД.

Требования к надежности.

Система может быть недоступна не более чем 24 часа в год. Резервирование данных осуществляет хостинг-провайдер. У администратора мобильного приложения должна быть возможность выгрузить и загрузить копию мобильного приложения.

**Перечень нефункциональных требований
(согласно ГОСТ Р ИСО/МЭК 9126-93)**

1. Целостность – способность системы противостоять (сохранять информационное содержание и однозначность интерпретации смысла) изменениям, искажениям или порче при возникновении сбоев или ошибок.

2. Рациональность – подразумевает, что при функционировании оптимальным образом используются имеющиеся в распоряжении системы ресурсы: время, оборудование (память), люди.

3. Численность задействованного персонала – количество лиц, задействованных в эксплуатации и обслуживании системы.

4. Работоспособность – способность системы выполнять свои функции с эксплуатационными показателями не ниже заданных значений.

5. Производительность – характеристика системы, отражающая ее способность производить определенный объем работ в единицу времени, например, пропускная способность, время ответа, доступность, число продуктов, полученная прибыль, быстроедействие.

6. Гибкость – возможность модификации обеспечивающей части системы, обычно возникает по двум причинам: чтобы отразить в системе изменение требований или чтобы исправить ошибки, внесённые ранее в процессе разработки. Гибкость заключается в возможности адаптации, наращивания, изменения средств.

7. Открытость – прозрачность функциональной части системы и возможность ее модификации без нарушения процесса функционирования.

8. Защищенность (безопасность) – способность обеспечения защиты данных от разрушения, искажения или преднамеренных фальсификаций злоумышленником. Характеризует возможное отсутствие риска, связанного с нанесением некоторого ущерба.

9. Потенциальная управляемость – возможность компенсировать возмущение быстрее, чем успеют измениться эти возмущения.

10. Наблюдаемость основных параметров управляемого процесса – характеризует степень и глубину отслеживания параметров функционирования системы и основных характеристик объекта управления на всех циклах и этапах работы системы / объекта.

11. Надёжность – свойство системы сохранять во времени в установленных пределах значения всех параметров, характеризующих способность системы выполнять требуемые функции в заданных режимах и эксплуатации.

12. Степень оперативности и надёжности управления – характеризует способность системы и субъекта управления выполнять поставленные задачи в сроки, обеспечивающие эффективное управление объектом. Управление является оперативным, если время, затрачиваемое на решение системой задач, в совокупности со временем, необходимым персоналу на подготовку к реализации управленческих решений, не будет превышать критического времени, диктуемого условиями сложившейся ситуации на объекте.

13. Безотказность – свойство системы сохранять работоспособность в течение требуемого интервала времени непрерывно без вынужденных перерывов. Безотказность является наиболее важной компонентой надёжности, так как она отражает способность длительное время функционировать без отказов. Один из показателей надёжности системы.

14. Живучесть – свойство системы сохранять работоспособность в условиях возмущающих воздействий внешней среды и отказов компонентов системы с минимальной частотой отказов, а в случае их возникновения эффективно восстанавливать утраченные функции и ресурсы.

15. Прогрессивность компьютерных и информационных технологий, задействованных в системе.

16. Наглядность интерфейса – должен быть удобным, интуитивно понятным и продуманным с точки зрения

инженерной психологии, эргономики и методов технической эстетики.

17. Долговечность – свойство системы сохранять работоспособность до наступления предельного состояния. Зависит от долговечности технических средств и подверженности системы моральному старению.

18. Сопровождаемость – отражает возможность проведения конкретных изменений (модификаций) системы. Изменение может включать исправления, усовершенствования или адаптацию компонентов системы к изменениям в окружающей обстановке, требованиях и условиях функционирования. Эффективность – получение от функционирования системы существенного технико-экономического, социального или другого эффекта.

19. Стоимость – овеществленный в товаре труд разработчиков. Определяется количеством труда, материала, энергии и информации, затраченных на производство товаров, и измеряется количеством, например эквивалента рабочего времени.

20. Эффективность – получение от функционирования системы существенного технико-экономического, социального или другого эффекта.

21. Мобильность – возможность перенесения системы из одного окружения в другое. Окружающая обстановка может включать организационное, техническое или программное окружение, а также условия эксплуатации.

22. Наличие технической поддержки. Введенная в эксплуатацию готовая система требует определенной технической поддержки. Это обусловлено, прежде всего, динамичностью информационных процессов: совершенствованием документооборота, появлением дополнительных структур данных и автоматизированных функций, что стало обычным явлением для любых развивающихся систем.

Состав функциональных подсистем ПС

Таблица Г.1

Задачи функциональных подсистем ПС управления машиностроительным предприятием (И – информационная, В – вычислительная, У – управляющая)

Наименование задачи (автоматизированной функции)	Класс
<i>Подсистема «Портфель заказов»</i>	
1. Ведение базы потребителей и заказов готовой продукции.	И
2. Разработка годовой программы выпуска изделий с разделением ее по кварталам.	И
3. Расчет объема производства и реализации продукции.	В
4. Расчет производственной мощности подразделений и всего предприятия.	В
5. Расчет плана по труду и заработной плате.	В
6. Расчет себестоимости по элементам калькуляции продукции и элементам затрат по подразделениям и предприятию в целом.	В
7. Расчет сметы затрат на производство.	В
8. Расчет фондов экономического стимулирования.	В
9. Расчет фактической прибыли и рентабельности производства.	В
10. Расчет финансового плана.	В
11. Расчет нормальных оборотных средств предприятия.	В
<i>Подсистема «Техническая подготовка производства»</i>	
1. Планирование технической подготовки производства (конструкторской и технологической) и учет выполнения плана.	И
2. Разработка сетевого графика подготовки производства новых изделий.	И
3. Планирование и учет обеспечения производства технологической оснасткой и инструментом.	И
4. Расчет экономической эффективности изготовления изделий.	В
5. Проектирование и конструирование изделий.	У
6. Проектирование комплекса инженерных ресурсов.	У
7. Расчет производственной мощности технологического оборудования.	В

8. Проектирование технологических процессов изготовления изделий основного производства.	У
9. Разработка спецификаций изделий и сборочных единиц.	У
10. Расчеты сводной трудоемкости изготовления изделий по видам работ и профессиям, по подразделениям и предприятию.	В
11. Расчет материалов на изделие по предприятию и его подразделениям.	В
<i>Подсистема «Оперативное планирование и управление производством»</i>	
1. Расчет календарно-плановых нормативов.	В
2. Распределение квартальной программы изготовления изделий по месяцам.	И
3. Разработка месячных производственных программ для цехов и участков.	У
4. Разработка сменно-суточных программ для цехов и участков.	У
5. Расчеты ресурсов, необходимых для выполнения плановых заданий.	В
6. Определение очередности выполнения работ и формирования сменно-суточных заданий производственным подразделениям и отдельным рабочим местам.	И
7. Оперативный учет хода выполнения плана производства.	И
8. Расчет и регулирование величины нормативных заделов деталей и объема незавершенного производства.	В
9. Оперативный учет обеспеченности производства материалами и анализ их использования.	И
10. Оперативный учет, регулирование производства и анализ использования оборудования.	И
11. Оперативный технико-экономический анализ итогов работы подразделений и всего предприятия в целом.	В
<i>Подсистема «Логистическое управление запасами (материально-техническое снабжение)»</i>	
1. Расчет потребности в материалах и комплектующих на годовую программу и планируемые периоды.	В
2. Расчет нормативных запасов материалов и комплектующих (складских и в незавершенном производстве) и их регулирование.	В
3. Оперативный учет поставок материалов, комплектующих и учет обеспеченности ими производства.	И
4. Определение очередности поставок новых партий материалов и комплектующих.	И / В

5. Анализ использования материалов и комплектующих на производстве.	И
6. Анализ активности и надежности поставщиков и заказчиков.	И
<i>Подсистема «Бухгалтерский учет»</i>	
1. Учет основных средств.	И
2. Учет поступления и использования материалов.	И
3. Учет движения материальных ценностей.	И
4. Начисление заработной платы работникам предприятия и ее учет.	И
5. Учет затрат на производство.	И
6. Учет готовой продукции.	И
7. Учет подотчетных лиц.	И
8. Составление отчетной калькуляции себестоимости продукции.	И
9. Учет транспортных затрат.	И
10. Учет денежных, расчетных и кредитных операций.	И
11. Составление сводного баланса основной деятельности предприятия.	В
<i>Подсистема «Финансовое обеспечение и сбыт продукции»</i>	
1. Расчет прибыли и рентабельности производства.	В
2. Расчеты с бюджетом.	В
3. Составление кассового плана на определенный период.	И
4. Составление баланса доходов, расходов и других расчетов финансового плана.	В
5. Расчет плана реализации готовой продукции.	В
6. Разработка плана отгрузки готовой продукции.	В
7. Учет хода выполнения плана отгрузки готовой продукции.	И
8. Расчет потребности в транспортных средствах.	В
9. Составление отчетной документации о выполнении плана реализации продукции и финансового плана.	И
10. Учет выполнения договоров с заказчиками.	И
<i>Подсистема «Управление внутризаводским транспортом»</i>	
1. Учет подвижного состава.	И
2. Учет технического состояния подвижного состава.	И
3. Мониторинг пассажиропотока.	И
4. Определение оптимального расписания следования транспорта.	В
5. Принятие решений по оптимизации транспортного обеспечения.	У

<i>Подсистема «Учет кадров»</i>	
1. Ведение учета состава работающих в цехах, отделах по возрасту, стажу работы, образованию, квалификации (профессиям и разрядам).	И
2. Ведение учета состава ушедших с работы по причинам увольнения.	И
3. Формирование данных об отпусках, больничных и медосмотрах.	И
4. Формирование данных о повышении квалификации.	И / У
5. Прогнозирование движения кадров и подготовка кадров.	И / У
6. Формирование данных о составе, работающих по подразделениям предприятия.	И

Таблица Г.2

Задачи функциональных подсистем автоматизированной системы управления дорожным движением
(И – информационная, В – вычислительная, У – управляющая)

№	Наименование автоматизированной функции	Класс
1. Подсистема мониторинга параметров транспортных потоков (инициируется локальной частью АСУДД)		
1.	Опрос детекторов транспорта (предполагает съем информации с детекторов).	И
2.	Преобразование сигналов с детекторов транспорта к виду, необходимому для обработки в ЭВМ.	В
3.	Вычисление значений параметров ТП на основе определенных методов.	В
4.	Сохранение полученных данных о ТП в блок данных транспортных потоков дорожного контроллера (ДК).	И
5.	Подготовка базовых параметров ТП для передачи в ЦУП.	В
2. Подсистема автоматического управления локальным СО (инициируется локальной частью АСУДД)		
1.	Подготовка управляющих воздействий на основе блока данных управляющей информации.	В
2.	Вывод управляющих воздействий на объект.	У
3.	Диагностирование технического состояния технических устройств СО.	У
4.	Сохранение полученных данных о техническом состоянии оборудования СО в блок диагностических данных ДК.	И
5.	Подготовка диагностических данных для передачи в ЦУП.	В

3. Подсистема связи ЦУП с локальными СО (инициируется центральной частью АСУДД)		
1.	Опрос локальных контроллеров на предмет готовности данных.	И
2.	Копирование блоков данных из локальных ДК в центральный контроллер.	И
3.	Копирование блоков данных из центрального контроллера в локальные ДК.	И
4.	Синхронизация процессов обмена данных между локальными ДК и центральным контроллером.	У
5.	Диагностирование соединений локальных ДК и центрального контроллера.	У
4. Подсистема распределенного управления дорожным движением (инициируется центральной частью АСУДД)		
1.	Выбор программы централизованного управления дорожным движением: адаптивное, сетевое, координированное, смешанное.	И
2.	Определение оптимальных временных параметров работы светофорных объектов на определенном участке УДС.	В
3.	Сегментация участка УДС на автономные сегменты и определение ведущих контроллеров сегментов.	В
4.	Формирование блока данных управляющей информации для передачи ведущим контроллерам сегментов.	И
5.	Подготовка данных для визуализации распределенного управления на мониторах ЦУП.	В
5. Подсистема диспетчерского управления дорожным движением (инициируется центральной частью АСУДД)		
1.	Переключение режимов функционирования СО и распределенного управления в локальное управление или ручной режим.	У
2.	Управление определенным локальным СО в ручном режиме.	У
3.	Реализация диспетчерских функций централизованного управления: корректировка программ управления, сегментации, управляющих параметров и т. д.	У
4.	Визуализация функционирования АСУДД на мнемосхемах и векторной карте УДС.	И
5.	Проектирование новых режимов работы локальных СО.	У
6. Подсистема диспетчеризации (инициируется центральной частью АСУДД)		
1.	Регистрация событий, происходящих в АСУДД.	И

2.	Прогнозирование возможных последствий зарегистрированных событий в АСУДД.	В
3.	Формирование сообщений (предупреждающие, информационные, служебные, аварийные) оперативному персоналу, в т.ч. диспетчеру, о зарегистрированных событиях и их возможных последствиях.	И
4.	Архивирование сообщений в журнале сообщений.	И
5.	Очистка журнала сообщений от устаревшей информации.	И
7. Подсистема формирования отчетов (инициируется центральной частью АСУДД)		
1.	Формирование отчета о событиях, происходящих в АСУДД за данный период.	И
2.	Формирование графиков изменения во времени непрерывных величин (фактически по любым параметрам задействованных в системе).	И
3.	Формирование отчетов, содержащих производственную выходную информацию.	И

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ	3
1. ТЕХНОЛОГИЯ ПРОЕКТИРОВАНИЯ ПРОГРАММНЫХ СИСТЕМ.....	6
1.1. ОБЩИЕ СВЕДЕНИЯ О ПРОЕКТИРОВАНИИ ПРОГРАММНЫХ СИСТЕМ	6
1.1.1. Понятие проектирования	6
1.1.2. Системный подход к описанию предметной области ...	7
1.1.3. Этапы и стадии проектирования ПС	13
1.1.4. Автоматизация управления и ее цели.....	15
1.1.5. Принципы кибернетики, используемые при проектировании ПС	20
1.1.6. Проведение предпроектного обследования предприятий	23
1.1.7. Результаты предпроектного обследования	28
1.1.8. Постановка задачи на создание ПС	29
1.2. ПРОЕКТНЫЕ РЕШЕНИЯ ПО СОЗДАНИЮ ПРОГРАММНОЙ СИСТЕМЫ	32
1.2.1. Структурные подсистемы ПС	32
1.2.2. Функциональная часть ПС	33
1.2.3. Порядок решения задач по созданию функциональных подсистем: организационный аспект.....	36
1.2.4. Информационное обеспечение ПС.....	38
1.2.5. Техническое обеспечение ПС.....	42
1.2.6. Требования к комплексу технических средств ПС	45
1.2.7. Математическое обеспечение ПС	49
1.2.8. Программное обеспечение ПС.....	51
1.2.9. Технико-экономическое обоснование проектных решений.....	53
1.3. ПОДХОДЫ К ПРОЕКТИРОВАНИЮ ПС	57
1.3.1. Методологии и технологии проектирования ПС	57
1.3.2. Стандарты проектирования ПС	59
1.3.3. Классификация подходов к проектированию ПС	60
1.3.4. Блочнo-иерархический подход.....	61
1.3.5. Функционально-ориентированный подход	62
1.3.6. Объектно-ориентированный подход.....	68

1.4. ДОКУМЕНТИРОВАНИЕ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ	69
1.4.1. Классификация документации ПО	71
1.4.2. Проблемы стандартов единой системы программной документации	74
1.4.3. Актуальные вопросы при разработке документации ..	75
2. ИНСТРУМЕНТЫ АВТОМАТИЗИРОВАННОГО ПРОЕКТИРОВАНИЯ ПРОГРАММНЫХ СИСТЕМ.....	77
2.1. UML и ЕГО ПРИМЕНЕНИЕ В ПРОГРАММНОМ ПРОЦЕССЕ	77
2.1.1. OMG UML Specification 1.5.....	80
2.1.2. OMG UML Specification 2.0.....	82
2.1.3. Пакеты в UML.....	86
2.1.4. Особенности изображения диаграмм UML.....	86
2.1.5. Правила графического изображения диаграмм.....	88
2.1.6. Диаграммы вариантов использования.....	90
2.1.7. Объектно-ориентированный анализ прецедентов	97
2.1.8. Диаграммы классов	101
2.1.9. Диаграммы состояний.....	109
2.1.10. Диаграммы деятельности	117
2.1.11. Диаграммы взаимодействия	120
2.1.12. Диаграммы компонентов	125
2.1.13. Диаграммы развертывания	127
2.1.14. Рациональный унифицированный процесс	131
2.2. CASE-СРЕДСТВА	135
2.2.1. Характеристика и обзор рынка CASE-средств.....	136
2.2.2. Классификация CASE-средств.....	137
2.2.3. Факторы выбора CASE-средств.....	140
2.2.4. Оценка и выбор CASE-средств	141
2.2.5. Критерии оценивания и выбора.....	143
2.2.6. Технология внедрения CASE-средств	146
2.2.7. Преимущества использования CASE-средств.....	148
2.2.8. Недостатки использования CASE-средств	150
ЗАКЛЮЧЕНИЕ.....	152
РЕКОМЕНДУЕМАЯ ЛИТЕРАТУРА.....	154
ПРИЛОЖЕНИЯ	157

Учебное издание

Полетайкин Алексей Николаевич
Добровольская Наталья Юрьевна

ПРОЕКТИРОВАНИЕ ПРОГРАММНЫХ СИСТЕМ

Учебное пособие

Подписано в печать 19.09.2025. Выход в свет 10.10.2025.

Печать цифровая. Формат 60×84 ¹/₁₆.

Уч.-изд. л. 12,0. Тираж 500 экз. Заказ № 6234.

Кубанский государственный университет
350040, г. Краснодар, ул. Ставропольская, 149.

Издательско-полиграфический центр
Кубанского государственного университета
350040, г. Краснодар, ул. Ставропольская, 149.

